FLEETSHARE - CENTRALIZED VEHICLE RENTAL MANAGEMENT SYSTEM

By
MUHAMMAD NAIM NAJMI BIN HAZRE

Thesis submitted in partial fulfilment of the
requirement for the award of the degree of
Bachelor of Computer Science (Software Engineering) with Honours

FACULTY OF COMPUTER SCIENCE AND MATHEMATICS
UNIVERSITI MALAYSIA TERENGGANU
2026

THESIS CONFIRMATION AND APPROVAL

This is acknowledged and confirmed that the thesis entitled: **FleetShare - Centralized Vehicle Rental Management System** by **Muhammad Naim Najmi Bin Hazre** Matric No.: **S70224** have been checked and all the suggested corrections have been done. The thesis is submitted to the Faculty of Computer Science and Mathematics, Universiti Malaysia Terengganu in partial fulfilment of the requirements for the award of the degree of **Bachelor of Computer Science (Software Engineering) with Honours**.

Authorized by:

..... Date:.....
Main Supervisor
Name: Profesor Madya Ts. Dr. Amir b. Ngah
Official Stamp:

..... Date:.....
FYP Coordinator
Bachelor of Computer Science (Software Engineering) with Honours
Name: Dr. Rabiei Mamat
Official Stamp:

DECLARATION

I hereby declare that this thesis is the result of my own research except as cited in the references.

Signature	:	.....
Name	:	Muhammad Naim Najmi Bin Hazre
Matric No.	:	S70224
Date	:	

ACKNOWLEDGEMENTS

I would like to express my deepest gratitude to my main supervisor, Profesor Madya Ts. Dr. Amir b. Ngah, for his continuous guidance, invaluable feedback, and unwavering support throughout the research and development of the FleetShare system. I also extend my sincere appreciation to the commercial operators and independent vehicle owners, particularly the branch administrator of KLezCar Kuala Terengganu and En. Ahadeeyat Aqasha, whose participation in the requirement elicitation interviews provided crucial industry insights. Special thanks go to my peers, including, for their encouragement and constructive discussions that helped refine this work. Last but not least, I am profoundly grateful to my family for their endless love, patience, and moral support, which kept me motivated throughout my entire academic journey at Universiti Malaysia Terengganu.

ABSTRACT

The rapid expansion of the sharing economy has significantly transformed the transportation sector, particularly in the vehicle rental industry in Malaysia. Despite this growth, many small and medium-sized enterprises (SMEs), commercial operators, and independent vehicle owners still rely on fragmented, semi-digitized systems to manage their fleets and daily business operation. These disparate methods frequently lead to operational inefficiencies throughout booking process, scheduling conflicts, and financial tracking challenges. To address these critical gaps, FleetShare - a centralized, multi-tenant vehicle rental management system was proposed and developed. The system architecture utilizes a Model-View-Controller (MVC) by leveraging the Java Spring Boot framework, ensuring secure, scalable, and independent operations across distinct client, renter, and administrator user modules. Comprehensive requirement elicitation was conducted through semi-structured interviews with key stakeholders, including commercial branch administrators and independent vehicle owners. This process identified essential needs for automated billing, streamlined user access, and consolidated fleet tracking. The resulting platform unifies scheduling, payment processing, and multi-tenant management into a single, cohesive platform. By providing an integrated environment, FleetShare effectively mitigates the administrative burden on fleet owners while simultaneously enhancing the booking experience for renters, establishing a robust and scalable framework for modernized vehicle rental operations.

ABSTRAK

Perkembangan pesat ekonomi secara perkongsian telah mengubah sektor pengangkutan secara signifikan, khususnya dalam industri sewaan kenderaan di Malaysia. Namun begitu, di sebalik pertumbuhan ini, banyak perusahaan kecil dan sederhana (PKS), pengendali komersial, serta pemilik kenderaan persendirian masih bergantung pada sistem separa digital yang terpisah-pisah untuk mengurus armada dan operasi perniagaan harian. Pendekatan yang tidak seragam ini sering mengakibatkan ketidakcekapan dalam proses tempahan, konflik penjadualan, serta kesukaran dalam pengesanan kewangan. Bagi menangani jurang kritikal ini, FleetShare, sebuah sistem pengurusan sewaan kenderaan berpusat dan berbilang penyewa (*multi-tenant*), telah dicadangkan dan dibangunkan. Sistem ini dibina berasaskan seni bina *Model-View-Controller* (MVC) dengan memanfaatkan rangka kerja (*framework*) Java Spring Boot, bagi memastikan operasi yang selamat, berskala, dan lancar merentasi modul pengguna yang berbeza, termasuk pelanggan, penyewa, dan pentadbir sistem. Pengumpulan keperluan dijalankan secara komprehensif melalui temu bual separa berstruktur bersama pihak berkepentingan, seperti pentadbir cawangan komersial dan pemilik kenderaan persendirian. Hasilnya, beberapa keperluan utama telah dikenal pasti, termasuk pengebilan automatik, penyelarasan akses pengguna, dan pemantauan armada secara bersepadu. Platform yang dibangunkan berjaya mengintegrasikan fungsi penjadualan, pemprosesan pembayaran, serta pengurusan berbilang penyewa ke dalam satu ekosistem digital yang padu. Melalui persekitaran yang bersepadu ini, FleetShare bukan sahaja mengurangkan beban pentadbiran pemilik armada, malah meningkatkan pengalaman tempahan bagi penyewa. Secara keseluruhan, sistem ini menyediakan satu kerangka yang kukuh dan berskala untuk menyokong pemodenan operasi sewaan kenderaan.

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

Figure **Pages**

LIST OF APPENDICES

Appendix **Pages**

CHAPTER 1

INTRODUCTION

This chapter introduces the fundamental concepts and the underlying motivation for the development of FleetShare. It encompasses the project background, a detailed problem statement, the proposed solution, and the specific objectives of the study. Furthermore, it outlines the project scope and concludes with a structural overview of the entire thesis, providing readers with a clear roadmap of the research.

1.1 Project Background

The sharing economy has transformed global and local transportation sectors, particularly in Malaysia, with a focus on the vehicle rental industry. This economic model, driven by IT-facilitated peer-to-peer frameworks, maximizes the utility of underused assets (Hamari et al., 2016). As mobility demands increase in Malaysia, there has been a noticeable shift from traditional, rigid car ownership toward flexible, short-term vehicle rental services (Ken Research, 2023).

Despite this rapid digital transition among large-scale enterprises, many small and medium-sized enterprises (SMEs), commercial operators, and independent vehicle owners still lag in digital adoption (OECD, 2021). Current trends indicate that micro-operators possess valuable assets but lack the unified technological infrastructure required to manage them efficiently. Consequently, they resort to fragmented, semi-digitized methods to handle daily business operations.

Addressing this technological disparity is significant for the local economy. By providing a unified digital infrastructure, small fleet owners can compete more effectively, optimize their asset utilization, and improve customer satisfaction (Element Fleet Management, 2026). This project bridges the gap between basic commercial rental needs and advanced technological solutions by introducing a centralized, multi-tenant ecosystem tailored specifically for the Malaysian vehicle rental landscape.

1.2 Problem Statements

In an ideal vehicle rental ecosystem, fleet owners and administrators should possess the capability to manage bookings, track financial transactions, and monitor vehicle availability through a single, unified system, while renters experience an effortless booking process. However, the reality for numerous SMEs and independent owners is a heavy reliance on fragmented, semi-digitized systems. Operators frequently utilize a disjointed combination of spreadsheet software, such as Microsoft Excel and Google Sheets, for record-keeping, alongside instant messaging applications, including WhatsApp and Facebook Messenger, for customer communication and manual booking confirmations. This deficiency highlights a significant gap in the market: a distinct lack of accessible, multi-tenant management systems designed to cater specifically to the operational scale of independent and SME fleet owners.

If this reliance on unstandardized, manual processing persists, operators will continue to encounter severe negative impacts, encompassing operational inefficiencies, frequent scheduling conflicts such as double-booking or overlooked reservation dates, and significant financial tracking challenges. Furthermore, the administrative fatigue induced by manually cross-referencing payments and booking schedules drastically constrains business scalability and degrades the overall customer experience. Consequently, a critical need exists for the development of an integrated

digital solution that eliminates these manual bottlenecks, standardizes the rental process, and establishes a reliable, scalable management framework for small-to-medium fleet operators.

1.3 Problem Solution

To resolve the inefficiencies identified in the current rental landscape, this project proposes the development of FleetShare, a centralized, multi-tenant vehicle rental management system. FleetShare directly addresses the research gap by transitioning operators away from fragmented spreadsheets and messaging apps into a single, cohesive web-based platform (Laudon and Laudon, 2020). By utilizing a Model-View-Controller (MVC) architecture powered by the Java Spring Boot framework, the system provides a secure, scalable, and independent environment tailored for distinct user roles: platform administrators, fleet owners, and renters (Kramer and Schmidt, 2024).

The connection between the identified ailments and this proposed solution is direct and functional. To cure the persistent issue of scheduling conflicts and double-bookings, FleetShare implements a centralized, real-time booking and inventory tracking module. To mitigate the administrative fatigue associated with manual financial tracking, the system integrates automated payment processing and standardized billing records. By providing this integrated, multi-tenant environment, FleetShare effectively reduces the administrative burden on fleet owners while simultaneously enhancing user accessibility and streamlining the booking lifecycle for renters (Laudon and Laudon, 2020).

1.4 Objectives

Research objectives in this Final Year Project represent the specific, measurable targets established to resolve the issues outlined in the problem statement. The objectives of this project are as follows:

- To analyse the operational challenges present in current semi-digitized rental platforms, specifically addressing inefficient reservation workflows, communication breakdowns, overlapping bookings, and inadequate administrative controls.

- To design and develop a unified, multi-tenant digital platform that optimizes vehicle browsing, reservation, and booking administration for end-users as well as system administrators and fleet owners.

- To implement and validate essential system functionalities that guarantee both security and performance, including real-time status updates, role-based access control, and an optimized database schema.

1.5 Project Scope

The Project Scope establishes the boundaries of the FleetShare system, ensuring the research and development remain focused and achievable within the designated timeframe. The scope of this project is defined as follows:

- **Target Users:** The system is designed specifically for three distinct user roles: Platform Administrators (system oversight), Fleet Owners (SMEs and independent owners managing their vehicles), and Renters (customers booking vehicles).

- **System Boundaries:** The project encompasses user authentication, vehicle inventory management, real-time booking scheduling, payment record management, and a report generation module. It does *not* include advanced hardware integrations such as live GPS tracking modules or physical IoT engine immobilizers.

- **Technical Stack:** The solution uses a web-based application utilizing the Java Spring Boot framework for backend operations, ensuring robust MVC

separation, alongside a standard relational database for data persistence.

- **Operational Context:** The system is localized for the Malaysian context, accommodating local currency (MYR) and targeting operational scenarios typical of Malaysian SMEs and independent vehicle operators.

1.6 Thesis Outline

The following presents the structured outline of this thesis, organized into six chapters to establish a clear and logical progression of the research. Chapter 1 introduces the project by providing the research background, defining the problem statements, and delineating the objectives and scope of the study. Chapter 2 details the Literature Review, which critically evaluates existing systems, related works, and the technological landscape surrounding vehicle rental management. Chapter 3 discusses the Methodology, outlining the Software Development Life Cycle (SDLC) and the requirement elicitation techniques employed. Chapter 4 focuses on the System Design, detailing the architectural framework, database schemas, and interface mockups. Chapter 5 examines the Implementation and Testing phases, illustrating the system coding process and its validation against the initial requirements. Finally, Chapter 6 concludes the thesis by summarizing the overall research outcomes, evaluating the attainment of the established objectives, and proposing recommendations for future enhancements.

CHAPTER 2

LITERATURE REVIEW

This chapter reviews the foundational theories and existing literature pertinent to the development of centralized vehicle rental platforms. It explores fundamental concepts such as the Model-View-Controller (MVC) architecture, multi-tenancy, and the sharing economy. Furthermore, it critically analyzes current related works in the market to highlight the existing technological gaps that FleetShare aims to resolve.

2.1 Introduction

The introduction of Chapter 2 serves to validate the research concerning the theories, technologies, and previous works identified in relation to the developed application. Understanding the underlying technologies and business models of the modern transportation sector is necessary for designing a system that effectively addresses the pain points of small and medium-sized enterprises (SMEs) and independent fleet owners. Therefore, this section emphasizes the significance of these fundamental elements as a precursor to their detailed analysis in the following subsections, ultimately justifying the architectural and functional choices made for FleetShare.

2.2 Fundamental Theory and Concept

To establish a robust technological and analytical foundation for FleetShare, it is crucial to divide the core concepts that drive its development. In the context of this study, fundamental theory refers to the software engineering architectures and economic models utilized to construct the system(Mourad et al., 2019). The application relies heavily on modern web development paradigms to ensure it is secure, scalable, and capable of serving multiple independent business operators simultaneously. The following subsections define the specific ideas and architectures that are central to the FleetShare ecosystem.

2.2.1 The Sharing Economy and Collaborative Consumption

The sharing economy represents a transformative socio-economic framework predicated on the shared utilization of physical and human resources (Belk, 2014). It signifies a fundamental paradigm shift away from traditional, rigid ownership structures toward collaborative consumption, wherein individuals and enterprises opt to rent or borrow assets rather than purchasing them outright (Hamari et al., 2016). This transition is primarily facilitated by advanced information technology platforms that connect disparate parties, thereby reducing transaction costs and information asymmetry.

In the operational context of FleetShare, collaborative consumption serves as the foundational business philosophy. The proposed system functions as a critical digital enabler designed to bridge the gap between asset holders and consumers. By providing a centralized, multi-tenant platform, FleetShare empowers independent vehicle owners—often burdened with underutilized assets—to seamlessly integrate into the local rental market. This integration not only maximizes the operational utility of existing vehicles but also facilitates the generation of secondary revenue streams for micro-entrepreneurs (Bardhi and Eckhardt, 2012).

To conceptualize the flow of value within this ecosystem, Figure 2.1 illustrates the interaction between the primary stakeholders in the proposed collaborative consumption model.

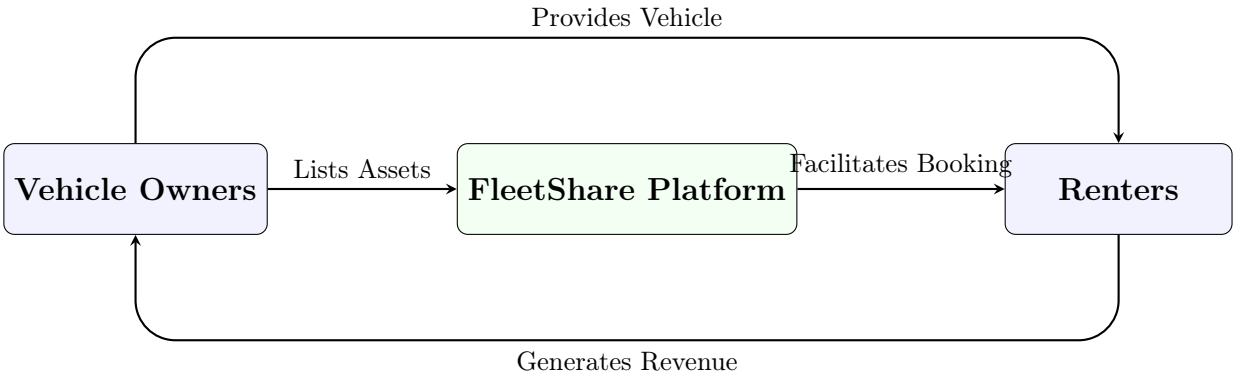


Figure 2.1: Value exchange and interaction flow in the FleetShare collaborative consumption model.

2.2.2 Multi-Tenant Software Architecture

Multi-tenancy is an architectural pattern in which a single instance of a software application serves multiple distinct user groups, known as tenants (Bezemer and Zaidman, 2010). Unlike single-tenant architectures where each customer requires a standalone database and software deployment, multi-tenancy securely partitions data within a shared infrastructure (Chong and Carraro, 2006). For FleetShare, this concept is vital. It allows various independent commercial operators and SMEs to manage their respective fleets, bookings, and financial records securely on one unified platform without their data overlapping or interfering with competitors (Seethamraju, 2015).

To visualize the architectural approach of FleetShare's multi tenancy to these systems, Figure 2.2 illustrates the standard flow of data system utilized by FleetShare.

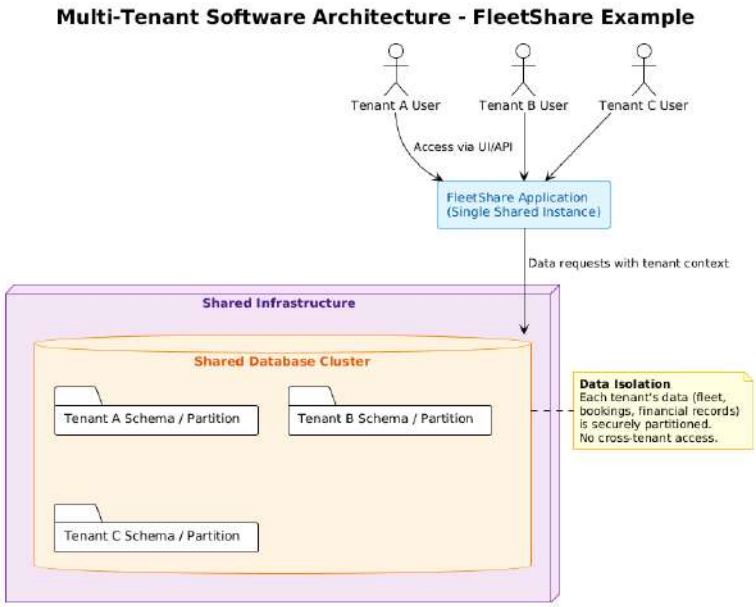


Figure 2.2: Standard Multi-Tenant Architecture Flow

2.2.3 Model-View-Controller (MVC) Design Pattern

The Model-View-Controller (MVC) is a foundational software design pattern that separates an application into three interconnected logical components (Krasner and Pope, 1988). The *Model* handles data logic and database interactions; the *View* manages the graphical user interface (UI) and user experience (UX); and the *Controller* processes user input, bridging the Model and View (Leff and Rayfield, 2001). FleetShare leverages this pattern through the Java Spring Boot framework to ensure high maintainability, allowing developers to update front-end renter interfaces without disrupting the complex backend scheduling and billing calculations (Walls, 2022).

To visualize the architectural pattern driving the proposed alternative to these systems, Figure 2.3 illustrates the standard flow of data within the selected MVC framework utilized by FleetShare.

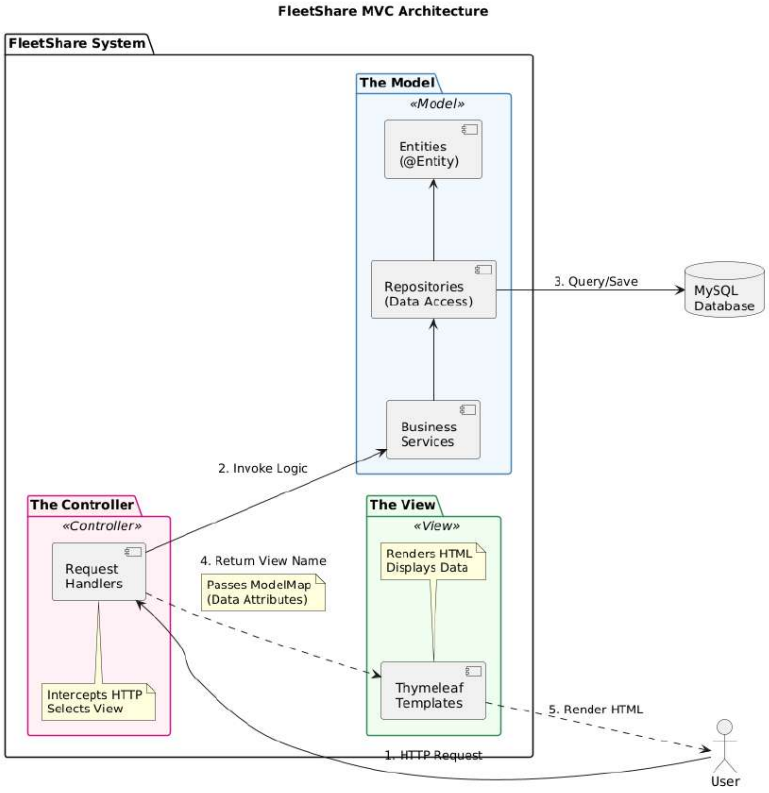


Figure 2.3: Standard Model-View-Controller (MVC) Architecture Flow

2.3 Related Works

To identify the functional gaps in the current vehicle rental market, an analysis of existing platforms was conducted. These related works represent the current standard of operations locally and globally, providing a benchmark against which FleetShare is evaluated.

2.3.1 KLezCar (Malaysia)

KLezCar operates as one of the largest car rental networks in Malaysia, utilizing a franchise-based business model. While KLezCar provides a proprietary management system for its official franchisees to track bookings, the system is strictly closed-loop. It lacks the flexibility required to accommodate external, independent fleet operators or individuals who wish to rent out personal vehicles. Consequently, local branch administrators often revert to manual tools like spreadsheets and

WhatsApp to handle edge cases or unsyncronized localized operations, indicating a gap in accessible, universal fleet management tools.

Figure 2.4: KLezCar Homepage. App screenshot

2.3.2 Hertz and Enterprise (Global Benchmarks)

Global industry leaders such as Hertz and Enterprise Rent-A-Car possess highly robust, centralized, and sophisticated IT infrastructures. These systems seamlessly handle international reservations, dynamic pricing, and vast fleet tracking. However, their technological framework is entirely proprietary and exclusive to their own massive, corporate-owned fleets. They do not operate as Software-as-a-Service (SaaS) providers. Therefore, their powerful operational models are inaccessible to the SMEs and independent owners that form the backbone of the local Malaysian rental market (Alam and Noor, 2009).

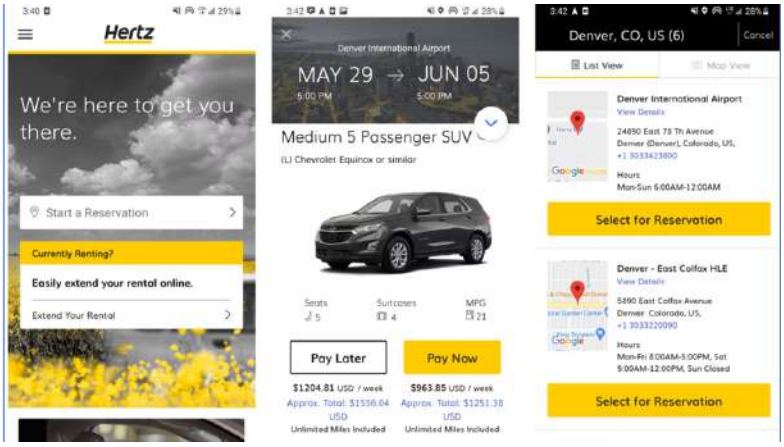


Figure 2.5: Hertz car rental mobile application homepage

2.3.3 SOCAR Malaysia (Urban Car-sharing)

SOCAR is a prominent urban car-sharing platform that has digitized the rental experience through a highly optimized mobile application. It allows users to book vehicles by the hour with keyless entry. However, SOCAR operates on a strict Business-to-Consumer (B2C) model, where the company owns or formally leases the entire active fleet. While it offers an excellent user experience, it acts as a competitor to local rental businesses rather than an enabling platform. It does not empower independent vehicle owners to manage their own independent rental operations.

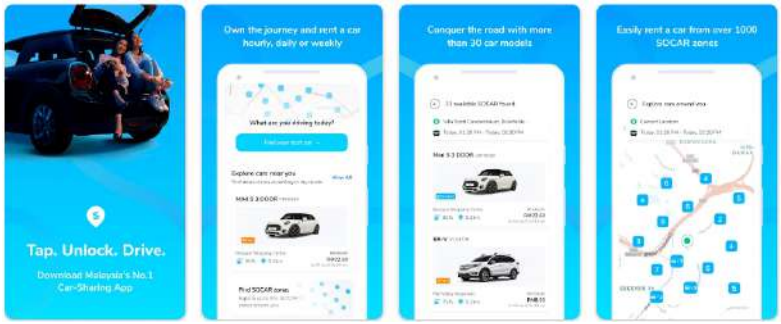


Figure 2.6: SOCAR Homepage. App screenshot

2.3.4 Comparison of Existing Systems and FleetShare

To summarize the analysis of the related works, Table 2.1 presents a direct comparison between the existing vehicle rental platforms and the proposed FleetShare system. The comparison highlights key operational and architectural differences, emphasizing how FleetShare addresses the technological gaps left by current market leaders.

Table 2.1: Comparison of Vehicle Rental Platforms

Feature	**KLezCar**	**Hertz & Enterprise**	**SOCAR Malaysia**	**FleetShare (Proposed)**
Target Audience	Official Franchisees	Global Corporate Clients	Urban B2C Consumers	SMEs & Independent Owners
System Architecture	Closed-loop System	Proprietary Enterprise	Proprietary Mobile App	Multi-tenant SaaS
Fleet Ownership	Franchise-owned	Corporate-owned	Company-owned/Leased	Independent
SME Accessibility	Low (Requires Franchise)	None	None (Competitor to SMEs)	High (Open Platform)

While existing systems focus on monopolizing fleet ownership or restricting access to proprietary networks, FleetShare is designed as an inclusive platform. It uniquely empowers independent vehicle owners and SMEs by providing them with the enterprise-grade tools necessary to digitize and manage their independent rental operations efficiently.

2.4 Discussion

The review of related works reveals a distinct polarization in the vehicle rental technology landscape. On one end of the spectrum are massive, proprietary enterprise systems (Hertz, KLezCar) that are closed to the public. On the other end are B2C aggregator platforms (SOCAR) that provide great user experiences but monopolize fleet ownership.

This polarization leaves a significant "missing middle." Small and medium-sized enterprises (SMEs) and independent owners possess the physical assets (vehicles) but lack access to an affordable, unified digital infrastructure. FleetShare tar-

gets this exact gap. By synthesizing the MVC design pattern with a multi-tenant SaaS approach, FleetShare provides a solution that is structurally akin to enterprise software but accessible to independent operators. Unlike SOCAR, FleetShare empowers the asset owner; and unlike KLezCar, it does not require formal franchising. It democratizes fleet management by offering automated billing, inventory tracking, and scheduling within a single, cohesive, and independent platform.

2.5 Summary

In summary, this chapter has detailed the foundational theories of the sharing economy, multi-tenant architectures, and the MVC design pattern, which collectively inform the technical strategy of this project. The critical analysis of related works including KLezCar, Hertz, and SOCAR demonstrated that current systems either restrict access to large corporations or monopolize the fleet entirely. This literature review confirms the necessity of the proposed FleetShare system: a centralized, multi-tenant management platform designed specifically to modernize and streamline the operational capabilities of independent vehicle rental businesses.

CHAPTER 3

METHODOLOGY

3.1 Introduction

This chapter delineates the research and development methodology employed for the FleetShare centralized vehicle rental management system. It provides a comprehensive overview of the chosen Software Development Life Cycle (SDLC) model, detailing the systematic phases from initial requirement elicitation to final system testing. Furthermore, this chapter outlines the project management strategies, including the designated milestones and the Gantt chart, which ensure the development process remains strictly aligned with the academic timeline and project scope.

3.2 Project Methodology

The FleetShare platform is developed using an waterfall model SDLC model, allowing sequential progress with refinements as needed (Larman and Basili, 2003). Development follows the standard distinction between analysis (defining what the system should do) and design (how it will be implemented) (Satzinger et al., 2015). The project comprised four major phases:

- **Phase 1 – Planning & Requirements Analysis:** Project scope was defined and requirements elicited via semi-structured interviews with stakeholders. Findings were documented in the SPMP and SRS (Wiegers and

Beatty, 2013). Key requirements included multi-tenant fleet management, real-time booking and availability, maintenance logging, secure login, flexible payments (bank transfer, QR), and role-specific dashboards.

- **Phase 2 – System Design:** The architectural blueprint was developed, including MVC architecture, a multi-tenant database schema (3NF), UI/UX wireframes, and an ERD (**?**), all formalized in the SDD (IEEE Computer Society, 2009). User flows, mockups, responsive design, and high-level module interconnections were also outlined.

- **Phase 3 – Implementation:** Backend logic was coded using Java Spring Boot. Modules for Platform Administrators, Fleet Owners, and Renters were integrated into a cohesive web environment. Integration linked Booking, Vehicle Management, and Payment modules to ensure real-time synchronization and multi-tenant isolation, followed by interoperability checks and error handling (**?**). Final adjustments aligned the system with design specifications before testing.

- **Phase 4 – Testing & Evaluation:** Unit and integration testing identified and resolved defects (Myers et al., 2011). User Acceptance Testing (UAT) then verified that the platform meets its objectives of reducing administrative burdens and streamlining vehicle booking (Cimperman, 2006).

Below is a diagram 3.1 that visualizes the classical waterfall model phases, whose sequential workflow informed the structure of the project phases described above.

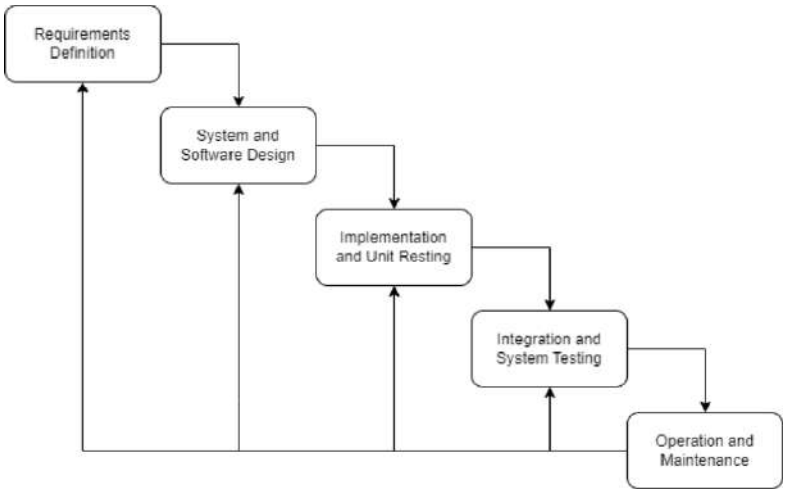


Figure 3.1: Waterfall model phases (Requirements Definition → System and Software Design → Implementation and Unit Testing → Integration and System Testing → Operation and Maintenance).

3.3 Project Planning Schedule

Effective project management relies heavily on structured scheduling and continuous progress tracking. To ensure the successful delivery of the FleetShare system within the allocated timeframe of the Final Year Project, specific milestones and a detailed Gantt chart were established.

3.3.1 Milestone

A project milestone is a crucial management tool used to mark specific, significant points along the project timeline. These milestones act as checkpoints to evaluate progress, ensure deadlines are met, and facilitate smooth transitions between the various SDLC phases. The key milestones for the development of FleetShare are outlined in Table 3.1.

Table 3.1: Key Milestones for the FleetShare Project

No.	Description	Start Date	End Date
1	Project Title Selection and Approval	01 Oct 2025	14 Oct 2025
2	Submission of Software Project Management Plan (SPMP)	15 Oct 2025	05 Nov 2025
3	Submission of Software Requirement Specification (SRS)	06 Nov 2025	10 Dec 2025
4	Submission of Software Design Document (SDD)	11 Dec 2025	15 Jan 2026
5	System Implementation and Prototype Development	16 Jan 2026	31 Mar 2026
6	Final System Testing and Submission of Thesis	01 Apr 2026	30 Apr 2026

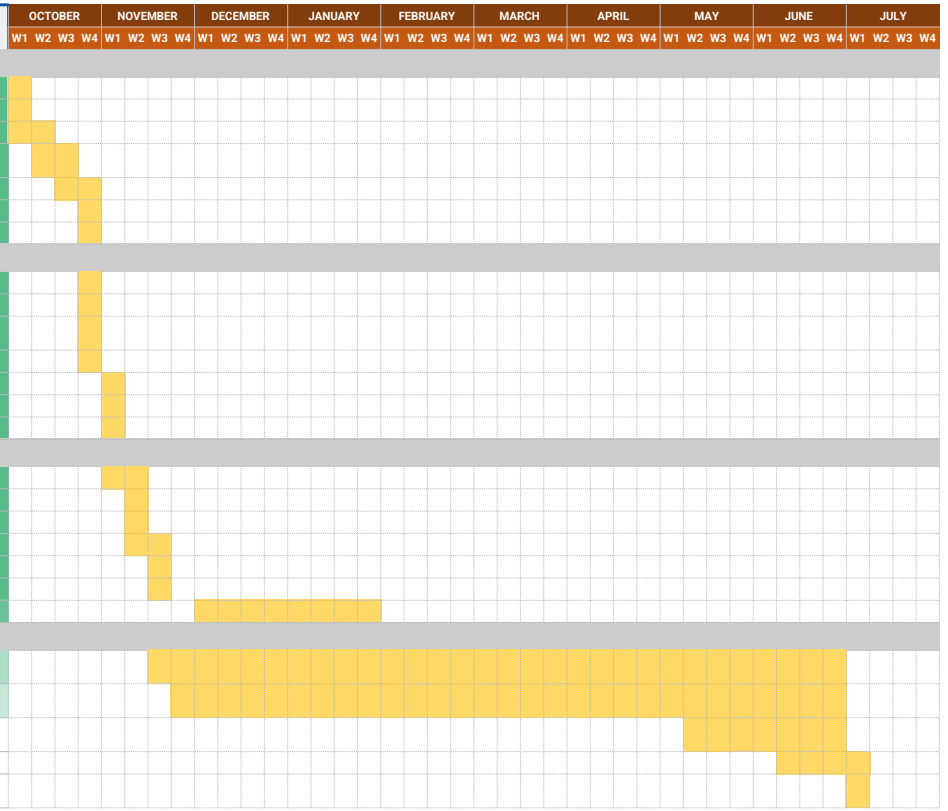
3.3.2 Gantt Chart

A Gantt chart is a chronological bar chart that visually illustrates a project schedule (Wilson, 2003). It provides a comprehensive breakdown of the tasks required to complete the FleetShare system, mapping each activity against its scheduled start and end dates. This visual representation aids in tracking concurrent tasks, managing time effectively, and ensuring that all development phases adhere to the planned milestones, as shown in Figure 3.2 (Kerzner, 2017).

Figure 3.2: Gantt Chart for the FleetShare Final Year Project

GANTT CHART : FLEETSHARE

WBS NUMBER	TASK TITLE	START DATE	DUE DATE	DURATION	PCT OF TASK COMPLETE
1	**Planning**				
1.1	Project Background	24/09/25	08/10/25	14	100
1.1.1	Project Objectives	24/09/25	02/10/25	8	100
1.2	Research	24/09/25	12/10/25	18	100
1.3	Problem Statements & Solutions	12/10/25	18/10/25	6	100
1.4	Project Scope	18/10/25	22/10/25	4	100
1.5	Methodology Selection	22/10/25	24/10/25	2	100
1.6	SPMP Submission	30/10/25	30/10/25	0	100
2	**Requirements**				
2.1	Requirements Gathering	23/10/25	25/10/25	2	100
2.2	Requirements Analysis	26/10/25	28/10/25	2	100
2.3	Functional & Non-Functional Requirements	29/10/25	30/10/25	1	100
2.4	Use Case	30/10/25	01/11/25	1	100
2.5	Activity Diagram	01/11/25	02/11/25	1	100
2.6	Sequence Diagram	03/11/25	05/11/25	2	100
2.7	SRS Submission	06/11/25	06/11/25	0	100
3	**Analysis & Design**				
3.1	Architectural Design	07/11/25	09/11/25	2	100
3.2	Normalization (0NF – 3NF)	09/11/25	11/11/25	2	100
3.3	ERD Diagram	11/11/25	13/11/25	2	100
3.4	Data Dictionary	14/11/25	16/11/25	2	100
3.5	UI/UX Design	16/11/25	19/11/25	3	100
3.6	SDD Submission	20/11/25	20/11/25	0	100
3.7	Final Documentation	1/12/2025	29/01/26	58	90
4	**Implementation**				
4.1	Coding (Development of Modules)	24/11/25	27/06/26	213	50
4.2	Module Integration (Linking All Modules)	30/11/25	28/06/26	208	35
4.3	System Optimization (UI/UX & Performance Refinement)	13/05/26	30/06/26	47	0%
4.4	User Manual Preparation	11/06/26	02/07/26	21	0%
4.5	Final Submission (System + Report)	04/07/26	04/07/26	0	0%



3.4 Summary

In summary, this chapter has detailed the methodological framework utilized to develop the FleetShare platform. By adopting an waterfall model SDLC approach, the project ensures a systematic progression through requirements gathering, system design, implementation, and rigorous testing. Furthermore, the inclusion of a structured project planning schedule, defined by clear milestones and a comprehensive Gantt chart, guarantees that the development lifecycle is managed efficiently and delivered within the academic constraints of the Final Year Project.

CHAPTER 4

SYSTEM REQUIREMENTS

4.1 Introduction

This chapter details the system requirements and the analytical modeling phases for the FleetShare platform. It outlines the requirement elicitation techniques employed to gather data from real-world stakeholders, followed by a comprehensive categorization of both functional and non-functional requirements. Finally, the chapter translates these requirements into structured UML models, including use case, activity, class, and sequence diagrams, to establish a clear blueprint for system implementation.

4.2 Requirement Elicitation Techniques

Requirement elicitation refers to the systematic search for details regarding the functions to be performed by the system and the limitations under which it must operate (Pressman and Maxim, 2020). It is the critical process of obtaining data directly from domain stakeholders to thoroughly identify business needs, operational risks, and system assumptions.

For the FleetShare project, a combination of conversational and document-centric techniques was employed to capture the nuanced needs of a multi-tenant environment:

- **Semi-Structured Interviews:** Direct conversational elicitation was con-

ducted with two primary stakeholder profiles (Hove and Anda, 2005). First, an interview with a commercial branch administrator (KLezCar Kuala Terengganu) highlighted systemic inefficiencies in utilizing disjointed spreadsheets for daily operations. Second, discussions with an independent vehicle owner (En. Ahadeeyat Aqasha) provided insights into the financial tracking and localized booking needs required by individuals renting out personal assets.

- **Existing System Analysis (Document-centric):** A comparative review of existing platforms such as SOCAR and enterprise-level rental software was conducted to identify baseline industry standards for UI/UX flow and data security (Courage and Baxter, 2005).

4.3 System Requirements

System requirements form the foundational blueprint for the proposed solution, establishing the boundaries and capabilities of the software (Pohl, 2010). They are categorized into functional requirements, which define the specific processes the system must perform, and non-functional requirements, which dictate the system's operational quality, security, and architectural constraints (Glinz, 2007).

4.3.1 Functional Requirement

Functional requirements define the specific behaviors, product features, and operations that the developers must implement to enable users to achieve their goals within the multi-tenant ecosystem (Sommerville, 2015). Given the architecture of FleetShare, these requirements are categorized by user role:

- **Renter Module:**
  - **Vehicle Browsing & Search:**
    - The system must allow users to browse and filter available vehicles based on location, dates, price range, and vehicle type.

    - The system must allow renters to view detailed vehicle information including pricing, owner details, and availability status.
  - **Booking Management:**
    - The system must allow renters to submit booking requests for available vehicles.
    - The system must validate date overlaps to prevent double-booking.
    - The system must allow renters to view their booking history with pagination.
    - The system must allow renters to cancel pending bookings.
  - **Payment Processing:**
    - The system must enable renters to process payments via the ToyyibPay gateway.
    - The system must allow renters to upload payment receipts/proofs securely.
    - The system must display available payment methods based on fleet owner configuration (FPX, Credit Card, Bank Transfer, Cash).
    - The system must allow renters to view their payment history.
  - **Profile Management:**
    - The system must allow renters to update their profile information including name, phone, and profile photo.
  - **Notifications:**
    - The system must send email notifications to renters for booking status updates, payment confirmations, and other relevant events.
- **Fleet Owner Module:**
  - **Vehicle Fleet Management:**

  - The system must enable fleet owners to add new vehicles with details (make, model, year, license plate, photos, pricing).
  - The system must allow owners to update and manage their vehicle inventory.
  - The system must allow owners to delete vehicles (with validation to prevent deletion during active bookings).
  - The system must enable owners to manage vehicle availability status (Available, Maintenance, Unavailable).
- **Booking Request Management:**
  - The system must allow owners to view incoming booking requests for their fleet.
  - The system must allow owners to approve, reject, or modify booking statuses.
  - The system must prevent owners from accessing booking data belonging to other fleet owners.
- **Maintenance Management:**
  - The system must allow owners to create, update, and track vehicle maintenance records.
  - The system must automatically update vehicle availability during maintenance (In Progress).
  - The system must provide a maintenance dashboard with KPI cards, filtering, and charts.
  - The system must maintain a historical timeline and audit trail for maintenance status changes.
- **Payment Configuration:**
  - The system must allow owners to configure ToyyibPay payment gateway (secret key, category code).

    - The system must allow owners to upload QR code images for alternative payment (DuitNow/TNG eWallet).
    - The system must allow owners to configure bank account details (bank name, account number, account holder).
    - The system must support split payments via ToyyibPay FPX for automatic commission deduction.
  - **Revenue & Reporting:**
    - The system must provide owners with revenue reports and utilization analytics.
    - The system must display vehicle utilization statistics.
  - **Profile Management:**
    - The system must allow owners to update their profile information including business details and profile photo.
  - **Notifications:**
    - The system must send email notifications to owners for new booking requests, status changes, and payment updates.
- **Platform Administrator Module:**
  - **Booking Oversight:**
    - The system must allow administrators to view all bookings across the platform.
    - The system must provide administrators with a booking management dashboard.
  - **Payment Oversight:**
    - The system must allow administrators to verify integrated payment statuses.

    - The system must display payment status logs for dispute resolution.
- **Maintenance Oversight:**
    - The system must provide a platform-wide maintenance oversight dashboard.
    - The system must allow administrators to view detailed maintenance tracking timelines for all vehicles.
- **Dispute Resolution:**
    - The system must allow administrators to access booking status audit logs (BookingStatusLog).
    - The system must allow administrators to access payment status audit logs (PaymentStatusLog).
- **Platform Reporting & Analytics:**
    - The system must provide platform-wide analytics and reporting (user counts, booking counts, revenue).
    - The system must calculate booking conversion rates and platform metrics.
- **Global Search:**
    - The system must provide a cross-entity search mechanism spanning vehicles, bookings, renters, and fleet owners.
- **AI Data Assistant:**
    - The system must provide administrators with access to the AI Data Assistant for platform analytics queries.

4.3.2 Non-functional Requirement

Non-functional requirements (NFRs) describe the system's operational capabilities and constraints, dictating *how well* the system performs rather than what it does (Dennis et al., 2021). For FleetShare, the following NFRs are critical:

- **Security and Data Partitioning:** Utilizing the MVC architecture, the system must strictly partition tenant data, ensuring that an independent fleet owner cannot query or access the bookings, vehicles, or financial records of a competing fleet owner.

- **Performance:** The system must be capable of processing real-time availability queries and concurrent booking requests without exceeding a standard response time of 3 seconds.

- **Portability:** Developed using Java Spring Boot, the system must be fully accessible via standard modern web browsers (Chrome, Safari, Edge) without requiring dedicated mobile application installations.

4.4 Requirement Analysis

Requirement analysis is the process of translating the defined functional and non-functional requirements into visual models (Boehm, 1984). These models provide a structured representation of the system's behavior, data relationships, and operational workflows (Rumbaugh et al., 2004). For the centralized vehicle rental ecosystem, Unified Modeling Language (UML) is utilized to map the interactions between the three primary actors (Renter, Fleet Owner, and Platform Administrator) and the system modules (Fowler, 2003).

4.4.1 Use Case Diagram

Use-case diagrams provide a bird's-eye view of the basic functionality and business processes of the system (Dennis et al., 2021). Figure 4.1 illustrates the interaction boundaries between Renters, Fleet Owners, and the overarching FleetShare system.

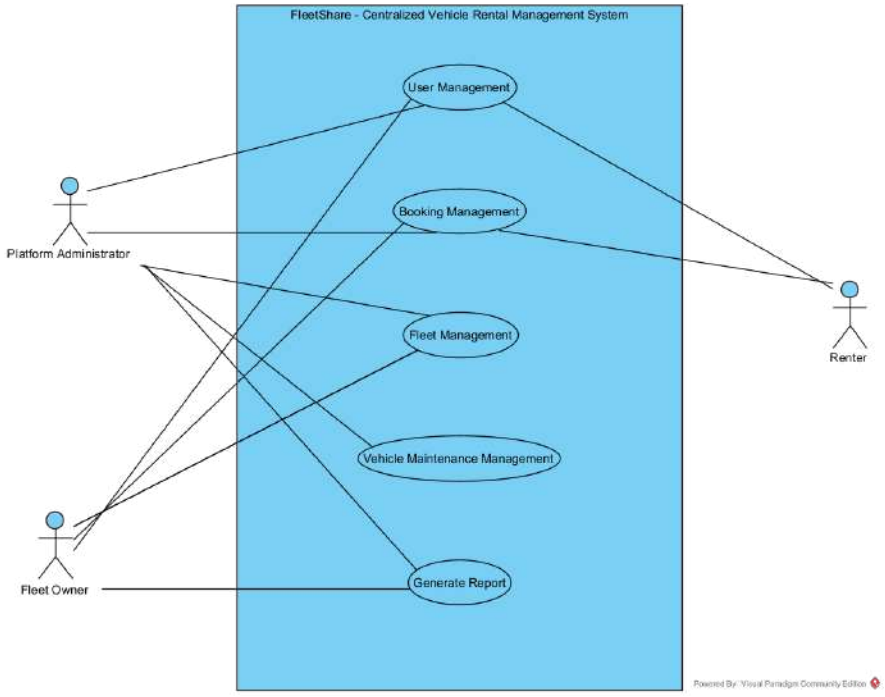


Figure 4.1: Use Case Diagram for the FleetShare System

4.4.2 Use Case Descriptions

A use case description supplements the use case diagram by providing a detailed, step-by-step textual narrative of the interaction between the primary actors and the system. It defines the primary goals, normal flow of events, and alternative workflows for each major business process (Satzinger et al., 2015). The following tables detail the core functional use cases for the FleetShare platform based on the interactions established in Figure 4.1.

Use Case: User Management

Use Case ID	UC-01
Use Case Name	User Management
Actor	Fleet Owner, Renter, Administrator
Description	Describes the lifecycle of a user account including multi-step registration, secure authentication, role-based profile management, and administrative oversight (verification and deactivation).

Continued on next page...

Precondition	User is on the login/registration page or is authenticated in the system.
Postcondition	User account is successfully created, profile data is updated, or account status is modified by an administrator.
Main Flow	1. **Registration**: User completes a 3-step form: • *Step 1 (Account)*: Enters Full Name, Email, and Password. • *Step 2 (Address)*: Enters Street Address, City, State, and Postal Code. • *Step 3 (Role)*: Selects between Renter or Vehicle Owner role. 2. **Authentication**: User logs in using Email and Password; system redirects based on role (Admin → Dashboard, Owner → Dashboard, Renter → Vehicles). 3. **Profile View**: User navigates to profile to view account details, address, and current status. 4. **Profile Update**: User updates profile image (JPG/PNG), edits personal info (Full Name/Business Name, Phone), or modifies address details. 5. **Location Setting**: Renter sets or updates geographic location using an interactive map interface. 6. **Password Management**: User changes password by providing the current password for verification.

Continued on next page...

	7. **Admin Management**: Administrator monitors the user list, verifies Fleet Owners, and updates user details. 8. **Account Deactivation**: Administrator toggles the "Active" status of an account to prevent login while preserving historical data. 9. **Logout**: User or administrator ends the session, clearing security context.
Alternative Flow	• **Business Identity**: Fleet Owners manage their profile using a Business Name which defaults to their Full Name upon registration. • **Deactivation vs Deletion**: Users are "Deactivated" rather than deleted to ensure that booking and transaction records remain consistent in reports.
Exception Flow	• **Duplicate Registration**: If the email is already in use → System rejects registration with an error. • **Invalid Credentials**: If login fails → System displays "Invalid email or password" and prevents access. • **Validation Failure**: If password is less than 8 characters or mandatory fields are missing → System prompts for correction before submission. • **Unauthorized Access**: If a user attempts to access a role-restricted URL → System redirects to an access denied page or the login screen.

Table 4.1: Use Case: User Management

Use Case: Booking and Payment Management

Use Case ID	UC-02

Continued on next page...

Use Case Name	Booking and Payment Management
Actor	Renter, Fleet Owner, Administrator
Description	Describes the end-to-end process of vehicle reservation, automated invoicing, secure payment processing (Online/Manual), and the management of the booking lifecycle.
Precondition	Renter is authenticated and has selected a vehicle; or an Owner/Admin is logged in to manage operations.
Postcondition	A booking is successfully recorded with an immutable price snapshot, and the associated payment is verified and split between the platform and the owner.
Main Flow	1. **Booking Initiation**: Renter selects rental dates; system validates availability and calculates total cost based on the current vehicle rate. 2. **Invoice Generation**: System creates a `PENDING` booking and generates an `ISSUED` invoice with a `BookingPriceSnapshot` to lock in the rental rate. 3. **Payment Selection**: Renter chooses a payment method: • *Online (ToyyibPay)*: Redirects to gateway for Card/FPX; system handles automated verification via callback. • *Bank Transfer/QR*: Renter uploads a receipt image (Proof of Payment). • *Cash*: Renter selects cash; status remains pending until physical handover.

Continued on next page...

	4. **Payment Verification**: Fleet Owner reviews uploaded receipts or physical cash and clicks "Verify Payment" in the dashboard. 5. **Status Transition**: Once verified, booking moves to `CONFIRMED` and invoice status updates to `PAID`. 6. **Rental Execution**: Owner updates status to `ACTIVE` upon vehicle pickup and `COMPLETED` upon return. 7. **Manual Booking**: Fleet Owner creates a booking manually for walk-in customers, bypassing the online search flow. 8. **Financial Reporting**: Owner views revenue dashboards; Administrator audits split payments (Commission vs. Payout).
Alternative Flow	• **Payment Retry**: If an online payment fails, the system allows the Renter to change the payment method or retry with a new transaction. • **Verification Rejection**: If a receipt is invalid, Owner marks payment as `FAILED` with a reason; system notifies Renter to re-upload. • **Status Dispute**: If an issue arises during rental, Owner/Admin sets status to `DISPUTED` for investigation.

Exception Flow	• **Date Conflict**: System prevents booking if the vehicle is already reserved for overlapping dates (status is neither `CANCELLED` nor `COMPLETED`). • **Callback Failure**: If the payment gateway callback fails, the system uses the Return URL logic as a fallback to verify the transaction. • **Invalid Modification**: System denies updates to booking dates or costs once a payment has been `VERIFIED`.

Table 4.2: Use Case: Booking and Payment Management

Use Case: Fleet Management

Use Case ID	UC-03
Use Case Name	Fleet Management
Actor	Fleet Owner
Description	Describes how fleet owners manage the lifecycle of their vehicle assets, including registration, dynamic pricing, status monitoring, and maintenance scheduling.
Precondition	Fleet Owner is authenticated and is on the Fleet Dashboard.
Postcondition	The vehicle inventory, pricing history, and maintenance logs are updated and synchronized with the booking system.
Main Flow	1. **Vehicle Registration**: Owner enters specifications (Brand, Model, Year, Registration No) and selects categories (Fuel Type, Transmission, Category). 2. **Dynamic Pricing**: Owner sets the "Rate per Day" and specifies an "Effective Date" for the price to become active. 3. **Image Management**: Owner uploads high-resolution vehicle images to the cloud/local storage.

Continued on next page...

Table 4.3: Use Case: Fleet Management (Continued)

	4. **Inventory Tracking**: Owner monitors vehicle status (AVAILABLE, MAINTENANCE, UNAVAILABLE) and updates current mileage. 5. **Maintenance Scheduling**: Owner creates a maintenance record with an estimated cost and scheduled date. 6. **Status Sync**: System validates maintenance dates against existing bookings; upon starting maintenance, system automatically sets vehicle to `MAINTENANCE` status. 7. **Record Archival**: Owner performs a soft-delete to remove a vehicle from the active listing while preserving historical transaction data.
Alternative Flow	• **Price History**: Owners can view a complete history of price changes for each vehicle to analyze rental trends. • **Maintenance Completion**: Upon completing maintenance, Owner records the final cost, and the system restores the vehicle to `AVAILABLE` status.
Exception Flow	• **Booking Conflict**: If maintenance is scheduled during an active booking → System generates a warning to alert the owner of the overlap. • **Unauthorized Edit**: If a user attempts to modify a vehicle ID not owned by them → System throws a security exception and denies access. • **Validation Error**: If the "Registration No" already exists in the system → System rejects the new vehicle entry.

Table 4.3: Use Case: Fleet Management

Use Case: Generate Report

Use Case ID	UC-04
Use Case Name	Generate Report and AI Analytics
Actor	Fleet Owner, Administrator
Description	Allows actors to generate detailed performance reports, perform period-over-period comparisons, and utilize AI-driven insights to optimize fleet operations.
Precondition	Actor is authenticated and has existing transaction or maintenance data in the system.
Postcondition	A comprehensive report (Standard or AI-generated) is displayed with visualizations and made available for multi-format export.
Main Flow	1. **Category Selection**: Owner selects a report category (Booking, Vehicle, Payment, Maintenance, or User). 2. **Parameter Filtering**: Owner applies filters including duration (Today, This Month, Last 7 Days, Custom), vehicle ID, and transaction status. 3. **Standard Generation**: System aggregates data using `ReportService` and calculates summary metrics (Total Revenue, Avg Utilization, etc.). 4. **Comparison Analysis**: System calculates period-over-period trends (Current vs. Previous Period) to show growth percentages. 5. **AI Intelligence**: Owner enters a natural language query; system builds a data context and uses an LLM to generate insights and strategic recommendations. 6. **Exporting**: Owner selects the export format (`PDF`, `CSV`, or `Excel`) and downloads the branded report.

Continued on next page...

Table 4.4: Use Case: Generate Report (Continued)

Alternative Flow	• **AI Assistant**: If the owner asks a specific question (e.g., "Top 3 earners"), the AI generates a customized table and explanation. • **No Data**: If no records match the criteria → The system displays an empty state with a "No data found" notification.
Exception Flow	• **AI Rate Limit**: If the AI API limit is reached → The system performs an automatic retry with exponential backoff before notifying the user. • **Invalid Date Range**: If the end date is before the start date → The system prompts for a valid date range correction.

Table 4.4: Use Case: Generate Report

Use Case: Vehicle Maintenance Management

Use Case ID	UC-05
Use Case Name	Predictive Maintenance and Fleet Health Management
Actor	Fleet Owner, Administrator
Description	Describes the management of vehicle health through predictive risk scoring, scheduled routine servicing, and real-time maintenance status synchronization.
Precondition	Fleet Owner is authenticated and has vehicles registered in the platform.
Postcondition	Vehicle health is monitored via risk scores, and maintenance logs are updated with synchronized vehicle availability status.

Continued on next page...

Table 4.5: Use Case: Vehicle Maintenance Management (Continued)

Main Flow	1. **Health Monitoring**: Owner views the "Fleet Health Dashboard" which displays `CRITICAL` or `HIGH` risk vehicles based on the system's predictive scoring algorithm. 2. **Maintenance Scheduling**: Owner creates a maintenance record, specifying the type (Routine, Repair, etc.), estimated cost, and scheduled date. 3. **Booking Validation**: System checks for scheduling conflicts with existing bookings and warns the owner if the vehicle is currently reserved. 4. **Work Execution**: Owner updates status to `IN_PROGRESS`; system automatically changes vehicle status to `MAINTENANCE`, removing it from the public search listing. 5. **Completion & Restoration**: Owner records the final cost and marks the record as `COMPLETED`; system restores the vehicle to `AVAILABLE` status. 6. **Audit Review**: Administrator or Owner reviews the `VehicleMaintenanceLog` to track all status changes and associated remarks.
Alternative Flow	• **Health Recommendations**: System provides specific service recommendations (e.g., "Check brake pads") based on the vehicle's mileage and age. • **Obsolete Removal**: Owner performs a soft-delete on obsolete maintenance records to clean up the dashboard while retaining data for reporting.

Continued on next page...

Table 4.5: Use Case: Vehicle Maintenance Management (Continued)

Exception Flow	• **Active Booking Conflict**: If maintenance is marked `IN_PROGRESS` while an active booking is ongoing → System generates a high-priority alert for the owner. • **Unauthorized Access**: System prevents Fleet Owners from viewing or modifying maintenance logs of vehicles belonging to other owners.

Table 4.5: Use Case: Vehicle Maintenance Management

4.4.3 Activity Diagrams

Activity diagrams are a key behavioral modeling technique in the Unified Modeling Language (UML), used to illustrate the flow of control within a system and the sequence of actions that make up a process Ambler (2004). In a sense, they open up the black box of each business process by providing a more-detailed graphical view of the underlying activities that support each business process.

The following subsections present activity diagrams for key use cases in the FleetShare system. Each diagram focuses on a specific process, illustrating the sequence of activities, decisions, and flows to ensure efficient multi-tenant vehicle rental management. These diagrams are designed to support individuals and businesses in Malaysia by streamlining fleet rental operations.

4.4.4 User Management

This activity diagram models the primary user-facing workflows within the FleetShare system: registration, authentication, and role-based dashboard navigation.

Table 4.6: Activity Diagram Details: User Management

Diagram ID	AD-01
Diagram Name	User Management
Participants	User, System
Description	Models the workflows for user access, including account registration (Renter/Owner), authentication (Login), and role-based dashboard redirection.
Precondition	User accesses the application URL.
Postcondition	User is successfully authenticated and directed to their role-specific landing page, or a new account is registered and ready for login.

Continued on next page...

Table 4.6: Activity Diagram Details: User Management (Continued)

Main Flow	1. User is at the entry point and is presented with a choice: Login or Register. 2. **Authentication (Login) Path:** • User selects "Login", enters credentials, and submits. • System validates credentials against the database. • **Role-Based Redirection:** System checks `UserRole` and redirects: – `PLATFORM_ADMIN` → Admin Dashboard. – `FLEET_OWNER` → Owner Dashboard. – `RENTER` → Vehicle Browse Page. 3. **Registration Path:** • User selects "Register" and fills in the `RegistrationDTO` (includes Role selection). • System validates data (email uniqueness, password confirmation). • System performs transactional save: Creates `User`, `Address`, and `Profile` (Renter/Owner). • System sends a welcome email and redirects the user to the Login page. 4. **Dashboard & Profile Actions:** • From the dashboard, users can "Access Own Profile" to update personal details or "Change Password". • User selects "Logout", which invalidates the session and returns them to the Login screen.
Alternative Flow	• **Direct URL Access:** If an unauthenticated user attempts to access a protected dashboard URL, the system redirects them to the Login page.

Continued on next page...

Table 4.6: Activity Diagram Details: User Management (Continued)

Exception Flow	• **Invalid Login:** If authentication fails, the system displays an "Invalid username or password" alert and remains on the login page. • **Invalid Registration:** If validation fails (e.g., email already exists), the system displays the specific `RegistrationException` message and preserves form data for correction. • **Unverified Owner:** A Fleet Owner may login but will be restricted from listing vehicles until the Admin updates their `isVerified` status.

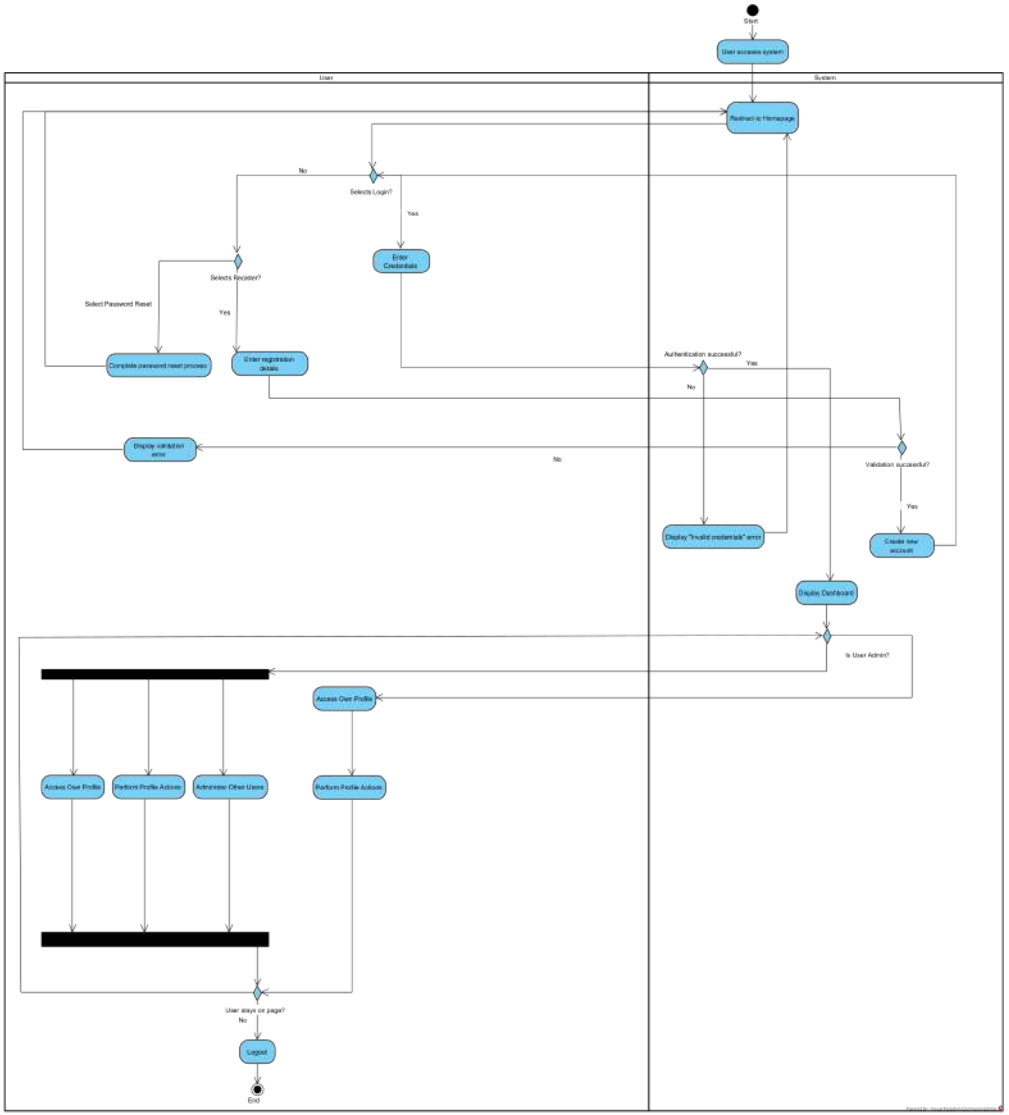


Figure 4.2: Activity Diagram for User Management

4.4.5 Fleet Management

This activity diagram models the core vehicle management workflows for a Fleet Owner within the FleetShare system, focusing on asset lifecycle and data integrity.

Table 4.7: Activity Diagram Details: Fleet Management

Diagram ID	AD-02
Diagram Name	Fleet Management
Participants	Fleet Owner, System
Description	Models the workflows for a Fleet Owner to add, view, edit, and remove vehicles, including price history tracking and secure deletion.
Precondition	A logged-in Fleet Owner navigates to the "/owner/vehicles" management section.
Postcondition	The vehicle inventory is modified (added, soft-deleted, or updated) and reflected in the dashboard.
Main Flow	1. Fleet Owner selects a management action: Add, View, Edit, or Delete. 2. **Path 1: Add New Vehicle** • Fleet Owner enters vehicle specs (Brand, Model, Plate No, etc.) and uploads an image. • System validates the `AddVehicleRequest` and stores the image. • System creates the `Vehicle` record and an initial `VehiclePriceHistory` entry.

Continued on next page...

Table 4.7: Activity Diagram Details: Fleet Management (Continued)

	3. **Path 2: View Vehicle List** • System fetches vehicles by `fleetOwnerId` and resolves the latest rates from history. • System displays the summarized fleet list. 4. **Path 3: Edit Existing Vehicle** • Fleet Owner updates details. If the "Rate per Day" is modified, the system prompts for an "Effective Date". • System updates the core vehicle record and appends a new `VehiclePriceHistory` if the rate changed. 5. **Path 4: Remove Vehicle (Secure)** • Fleet Owner selects "Delete". • **Security Check:** System prompts the owner to enter their login password. • System verifies the password and checks for active booking conflicts. • System performs a **Soft Delete**, marking the vehicle as inactive while preserving its history.
Alternative Flow	• **Rate History Management:** From the "Edit" view, owners can access a "Rate History" sub-flow to view past price changes or schedule future price updates without editing the main vehicle profile.
Exception Flow	• **Invalid Deletion Password:** If the password verification fails, the system displays an "Incorrect password" error and aborts the deletion. • **Validation Failure:** If required fields (e.g., Registration No) are missing during Add/Edit, the system redirects back with field-specific error messages.

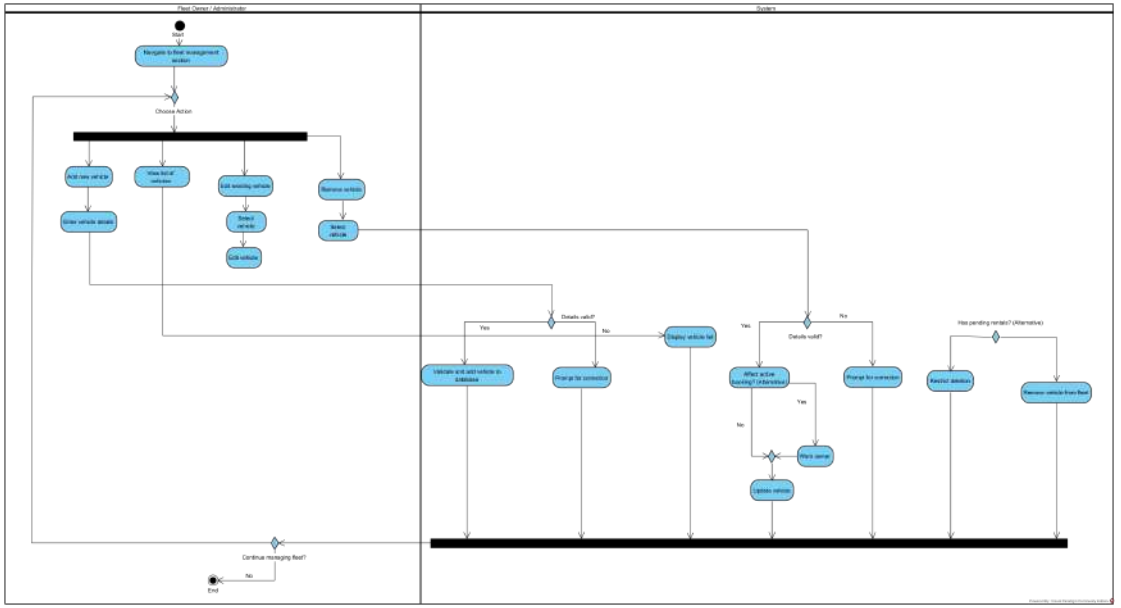


Figure 4.3: Activity Diagram for Fleet Management

4.4.6 Booking and Payment Management

This activity diagram models the end-to-end lifecycle of a vehicle reservation, from initial availability checks to final financial verification and booking confirmation.

Table 4.8: Activity Diagram Details: Booking and Payment Management

Diagram ID	AD-03
Diagram Name	Booking and Payment Management
Participants	Renter, System, Fleet Owner/Administrator
Description	Models the end-to-end workflow for vehicle reservation, integrating ToyyibPay gateway processing and manual receipt verification.
Precondition	Renter has selected a vehicle and is authenticated in the system.
Postcondition	Booking is moved to "CONFIRMED" status, the Invoice is marked "PAID", and both parties receive email notifications.
Main Flow	1. Renter selects a vehicle and specifies rental dates. 2. **System Validation:** System checks date availability and calculates the total cost using the current `VehiclePriceHistory`. 3. **Booking Initialization:** System records a "Provisional Booking" (`PENDING`) and generates a linked `Invoice` and `PriceSnapshot`. 4. Renter chooses a payment method:

Continued on next page...

Table 4.8: Activity Diagram Details: Booking and Payment Management (Continued)

	5. **Path 1: Online Payment (ToyyibPay)** • System generates a ToyyibPay bill and redirects the Renter to the gateway. • Gateway processes payment and sends a callback to the System. • If status is "Success", System automatically marks the Invoice as `PAID`. 6. **Path 2: Manual Payment (Receipt Upload)** • Renter uploads a digital receipt (Bank Transfer/QR). • System records a `Payment` with status `PENDING_VERIFICATION`. • Fleet Owner/Administrator reviews the receipt in their management dashboard. • If approved, the System marks the Payment as `VERIFIED`. 7. **Finalization:** • System updates Booking status to `CONFIRMED` and creates a `BookingStatusLog` entry. • System triggers welcome/confirmation emails to both Renter and Owner.
Alternative Flow	• **Payment Rejection:** If the Owner/Admin rejects a manual receipt, they must provide a reason. The System notifies the Renter to "Retry Payment" or "Change Payment Method".

Continued on next page...

Table 4.8: Activity Diagram Details: Booking and Payment Management (Continued)

Exception Flow	• **Vehicle Unavailable:** If the vehicle becomes booked by another user before confirmation, the System aborts the process and notifies the Renter. • **Gateway Timeout:** If ToyyibPay fails to respond, the booking remains `PENDING` for 24 hours before automatic expiration.

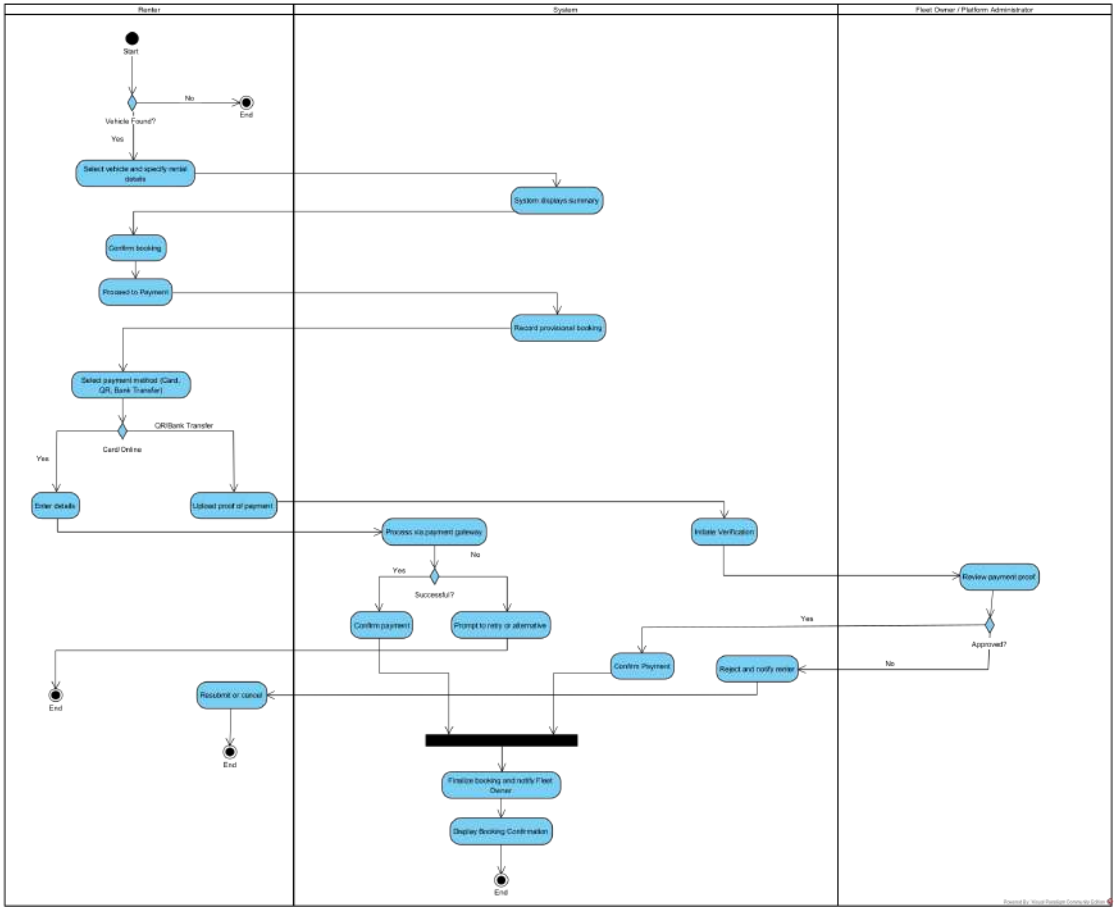


Figure 4.4: Activity Diagram for Booking Management

4.4.7 Generate Report

This activity diagram models the analytical workflow for synthesizing system data into actionable insights, covering data aggregation, visualization, and multi-format distribution.

Table 4.9: Activity Diagram Details: Generate Report

Diagram ID	AD-04
Diagram Name	Generate Report
Participants	Fleet Owner / Administrator, System
Description	Models the analytical workflow for aggregating multi-dimensional data into on-screen dashboards or downloadable files (PDF, CSV, Excel).
Precondition	A logged-in Fleet Owner or Administrator navigates to the "/reports" section of their respective portal.
Postcondition	Analytical data is visualized via charts/tables or exported as a physical document for offline use.
Main Flow	1. User selects a report category (e.g., "Revenue") and applies dynamic filters (Date Range, Vehicle Status, etc.). 2. **Data Aggregation:** System queries the relevant repositories to fetch raw records based on the filter criteria. 3. **Analytical Processing:** System calculates totals, averages, and utilization percentages. If "Comparison Mode" is enabled, it fetches data for the previous period to calculate growth metrics.

Continued on next page...

Table 4.9: Activity Diagram Details: Generate Report (Continued)

	4. **Visualization:** System generates chart data (line, bar, or pie charts) via the `ReportChartService`. 5. **Output Selection:** User chooses the presentation format: • **On-Screen View:** Renders interactive charts and summarized tables. • **Document Export:** Converts the report data into a specialized format: – **PDF:** Uses `PdfRendererBuilder` for professional layout. – **Excel/CSV:** Uses Apache POI for raw data manipulation.
Alternative Flow	• **Zero Records Found:** If no data matches the filters, the system displays a "No matching records found" empty state and prompts the user to broaden their search criteria.
Exception Flow	• **Complex Query Timeout:** If the requested report covers an extremely large data range that exceeds processing limits, the system provides a partial result or suggests a shorter duration.

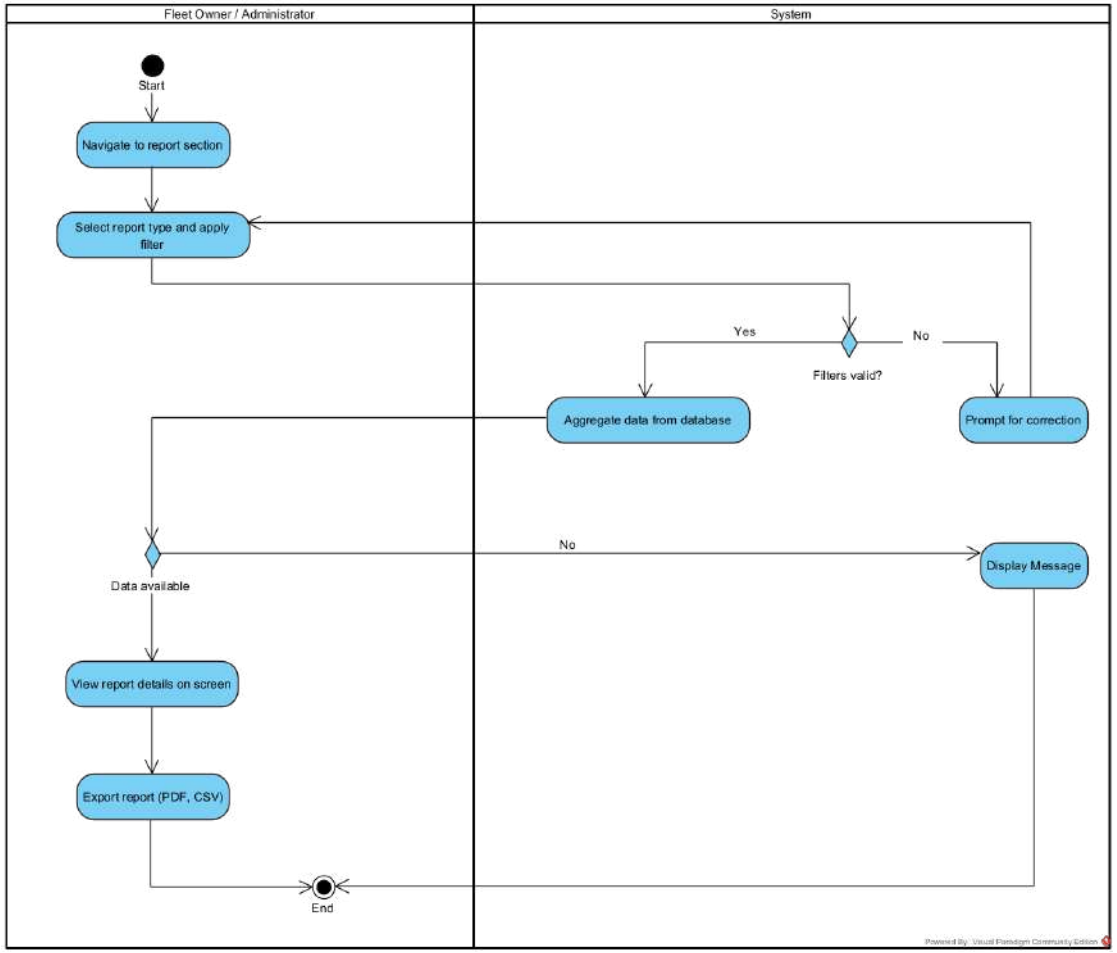


Figure 4.5: Activity Diagram for Generating Report

4.4.8 Vehicle Maintenance Management

This activity diagram models the lifecycle of vehicle health management, integrating conflict validation, automated status synchronization, and historical auditing.

Table 4.10: Activity Diagram Details: Vehicle Maintenance Management

Diagram ID	AD-05
Diagram Name	Vehicle Maintenance Management
Participants	Fleet Owner / Administrator, System
Description	Models the CRUD and analytical workflows for vehicle maintenance, featuring automated availability synchronization and booking conflict checks.
Precondition	A logged-in Fleet Owner is in the "/owner/maintenance" section.
Postcondition	Maintenance records are updated, the vehicle's availability status is synchronized, and audit logs are recorded.
Main Flow	1. User selects an action: View History, Review Stats, Create Record, Update Status, or Archive (Delete). 2. **Path 1: Create New Maintenance** • User enters maintenance details and scheduled date. • **System Validation:** System checks for overlapping bookings on the selected date. • **Conflict Warning:** If bookings exist, System warns the User but allows the save. • System saves the record and initializes the `VehicleMaintenanceLog`.

Continued on next page...

Table 4.10: Activity Diagram Details: Vehicle Maintenance Management (Continued)

	3. **Path 2: Update Status (The State Machine)** • User updates status (e.g., to `IN_PROGRESS` or `COMPLETED`). • **Status Sync:** If `IN_PROGRESS`, System sets the Vehicle status to `MAINTENANCE`. • **Cost Finalization:** On completion, User enters the `finalCost`. • System restores Vehicle to its previous status and records the transition in the log. 4. **Path 3: Review Fleet Stats** • System aggregates data to calculate "Total Maintenance Spend" and "Monthly Service Trends". • System displays a KPI dashboard with visual charts. 5. **Path 4: Delete/Archive Record** • System performs a soft delete (`isDeleted=true`) and records the deletion timestamp.
Exception Flow	• **Ownership Violation:** If a user attempts to edit a maintenance record for a vehicle they do not own, the system throws an authorization error and redirects to the dashboard. • **Invalid Cost Entry:** If a negative cost is entered, the system prompts for correction before saving.

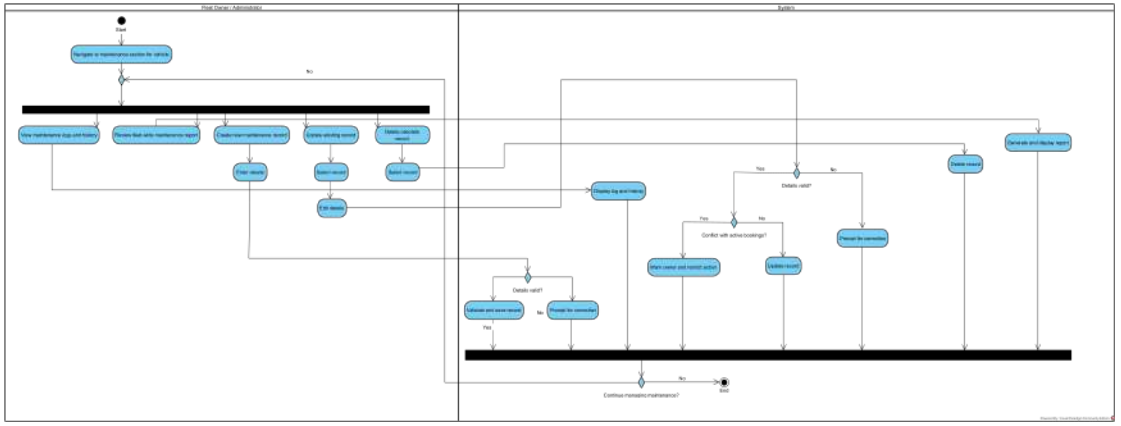


Figure 4.6: Activity Diagram for Vehicle Maintenance Management

4.4.9 Class Diagram

Class diagrams are a cornerstone of the Unified Modeling Language (UML), providing a static view of a system by modeling its classes, attributes, operations, and the relationships among them (Fowler, 2003). A class diagram is a static model that shows the classes and the relationships among them. It provides a structural view of the system by depicting classes, their attributes, methods, and the dependencies between them.

The class diagram for FleetShare illustrates the core entities involved in the multi-tenant vehicle rental management system. It highlights the structure for managing users, vehicles, bookings, payments, and maintenance. The design emphasizes strong data integrity and logical dependencies, ensuring a robust foundation for the peer-to-peer rental platform. Key classes and their relationships are described below:

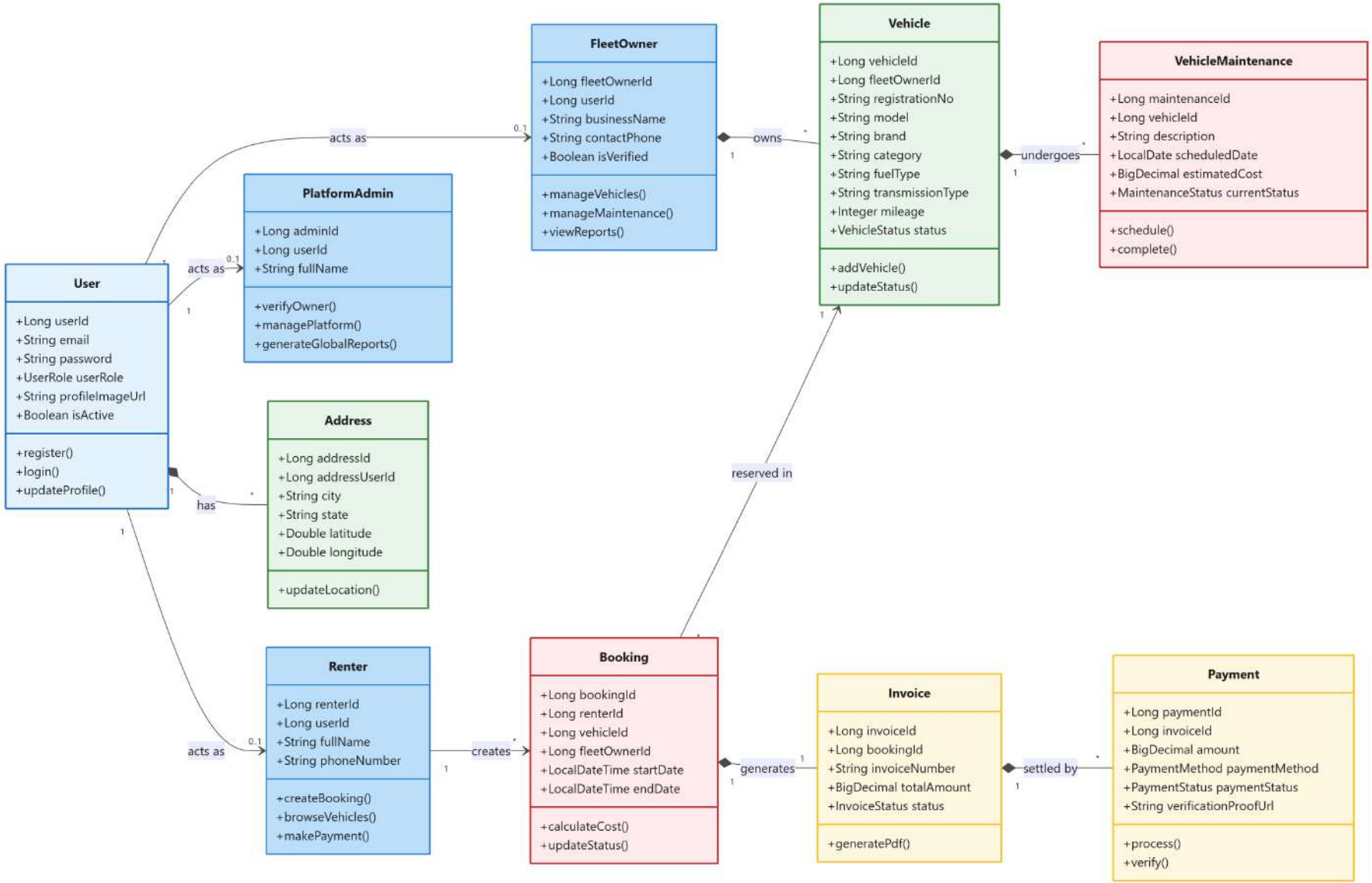


Figure 4.7: Class Diagram of FleetShare

4.4.10 Key Classes and Attributes

- **User**: The central identity and authentication entity. It stores core credentials (`email`, `password`), global status (`isActive`), and security roles (`userRole`). Unlike the previous design, it now acts as a standalone entity that role-specific profiles link to.
- **Renter**: A role-specific profile linked to a `User` via a 1:1 association. It stores customer-specific data like `fullName` and `phoneNumber`. Methods focus on the rental lifecycle, including `browseVehicles()`, `createBooking()`, and `uploadPaymentProof()`.
- **FleetOwner**: A business profile linked to a `User`. It contains `businessName`, `contactPhone`, and an `isVerified` flag managed by Administrators. It provides methods for fleet governance, such as `addVehicle()`, `manageMaintenance()`, and `generatePayoutReport()`.
- **PlatformAdmin**: An administrative profile linked to a `User`. It provides high-level oversight capabilities, including `verifyOwner()`, `monitorPayments()`, and platform-wide analytics.
- **Vehicle**: Represents a rental asset with expanded technical specifications including `brand`, `category`, `fuelType`, `transmissionType`, and `mileage`. It tracks availability through a `VehicleStatus` enum (`AVAILABLE`, `RENTED`, `MAINTENANCE`).
- **Booking**: The core operational entity connecting a `Renter`, `Vehicle`, and `FleetOwner`. It manages the reservation period (`startDate`, `endDate`) and relies on a `BookingStatusLog` for state transitions.
- **Invoice**: Generated automatically upon booking creation. It tracks the `totalAmount` and `dueDate`. It serves as the financial bridge between a reservation and its subsequent payments.

- **Payment**: Handles transaction data, supporting both automated gateway logic (via `toyyibpayBillCode`) and manual verification. It stores a `verificationProofUrl` for renters to upload receipts for admin approval.
- **Address**: A new entity dedicated to spatial data, storing `city`, `state`, and geographic coordinates (`latitude`, `longitude`). It supports historical tracking of user locations via effective dating.
- **VehicleMaintenance**: A robust module for fleet health. It tracks `maintenanceType`, `scheduledDate`, and distinguishes between `estimatedCost` and `finalCost`. It is synchronized with the `Vehicle` status to prevent rentals during repairs.

4.4.11 Relationships

The system architecture has been updated to prioritize decoupling and auditability, moving away from deep inheritance hierarchies in favor of composition and state logging.

- **One-to-One Association (Role-Based Mapping)**:
  - Instead of inheritance, `Renter`, `FleetOwner`, and `PlatformAdmin` maintain a 1:1 relationship with the `User` entity. This allows a user's authentication identity to remain distinct from their functional profile, facilitating easier role transitions and cleaner data separation.
- **Composition (Strong Ownership and State Tracking)**:
  - **FleetOwner to Vehicle (1 to 0..*)**: A `Vehicle` is strictly owned by one `FleetOwner`. The system enforces this through `fleetOwnerId` foreign keys in all vehicle-related operations.
  - **Status Logging (Compositional Audit)**: `Booking`, `Payment`, and `VehicleMaintenance` entities now utilize a composition relationship with their respective `*Log` entities (e.g., `BookingStatusLog`). These

logs are immutable records that provide a full audit trail of every status change.

- **Aggregation (Weak Ownership and Historical Persistence)**:
  - **Vehicle to Price History (1 to 0..*)**: A `Vehicle` aggregates multiple `VehiclePriceHistory` records. This allows the system to change rental rates without altering the financial data of existing or past bookings.
  - **User to Address (1 to 0..*)**: Users can maintain multiple address records over time. The system uses aggregation to ensure that even if an address is updated, the link to past transactions remains intact for legal and tax compliance.
- **Standard Associations**:
  - **Booking to Invoice (1 to 1)**: In the current implementation, each `Booking` is associated with a single `Invoice` that captures the total calculated cost at the time of reservation.
  - **Invoice to Payment (1 to 0..*)**: An `Invoice` can be associated with multiple `Payment` attempts, allowing for failed transactions or retries while maintaining a link to the original bill.
  - **Booking to Price Snapshot (1 to 1)**: To ensure data integrity, each `Booking` is linked to a `BookingPriceSnapshot`, which freezes the vehicle's rate and rental duration at the moment of booking, protecting the transaction from subsequent price fluctuations.

4.4.12 Sequence Diagrams

Sequence diagrams are used to model the interactions between actors and the objects in a system, as well as the interactions between the objects themselves; they show the sequence of interactions that take place during a particular use case or use case instance (Sommerville, 2020). Sequence diagrams are one of two types of interaction diagrams. They illustrate the objects that participate in a use case and the messages that pass between them over time for one use case. A sequence diagram is a dynamic model that shows the explicit sequence of messages that are passed between objects in a defined interaction. Because sequence diagrams emphasize the time-based ordering of the activity that takes place among a set of objects, they are very helpful for understanding real-time specifications and complex use cases.

The following subsections present sequence diagrams for key use cases in the FleetShare system. Each diagram focuses on the interactions between objects, such as users, system components, and databases, to model the dynamic behavior during multi-tenant vehicle rental operations. These diagrams support individuals and businesses in Malaysia by detailing how fleet rental processes are executed securely and efficiently.

4.4.13 User Management

This sequence diagram models the account management and administrative workflows, highlighting the interaction between the role-based controllers and the historical address tracking system.

Table 4.11: Sequence Diagram Details: User Management

Diagram ID	SD-01
Diagram Name	User Management
Participants	User, AdminController, UserManagementService, BookingService, UserRepository, AddressRepository

Continued on next page...

Table 4.11: Sequence Diagram Details: User Management (Continued)

Description	Illustrates the logic for profile synchronization, address versioning, and secure account deactivation.
Precondition	User is authenticated and possesses the required roles for the requested operation (Self or Admin).
Postcondition	User state is persisted, historical records are preserved, and active sessions are updated.
Main Flow	1. **Update Profile:** • User submits `UserDetailDTO` to the `AdminController`. • `AdminController` invokes `UserManagementService.updateUser()` • System calls `BookingService.checkActiveBookings()` if sensitive data (email) is changing. • `UserManagementService` updates the `UserRepository` and the role-specific profile. • **Address Check:** If the physical address changed, the System creates a new record in `AddressRepository` rather than updating the old one. 2. **Deactivate Account (Soft Delete):** • User/Admin requests deactivation. • `UserManagementService` verifies there are no "ACTIVE" or "CONFIRMED" rentals via `BookingService`. • System sets `isActive = false` in the `UserRepository`. • System returns a logout directive to the User. 3. **Admin User Management:** • Admin searches for users via `UserManagementService.getAllRenters()` or `getAllFleetOwners()`. • System returns batch-fetched DTOs for efficient display. • Admin applies bulk updates or individual status changes.

Continued on next page...

Table 4.11: Sequence Diagram Details: User Management (Continued)

Alternative Flow	• **Active Rental Conflict:** If deactivation is requested while a rental is "ACTIVE", the `BookingService` returns a conflict status, and the System prompts the user to resolve the rental first.
Exception Flow	• **Email Conflict:** If the new email address already exists in `UserRepository`, the system returns a `400 Bad Request` and aborts the update.

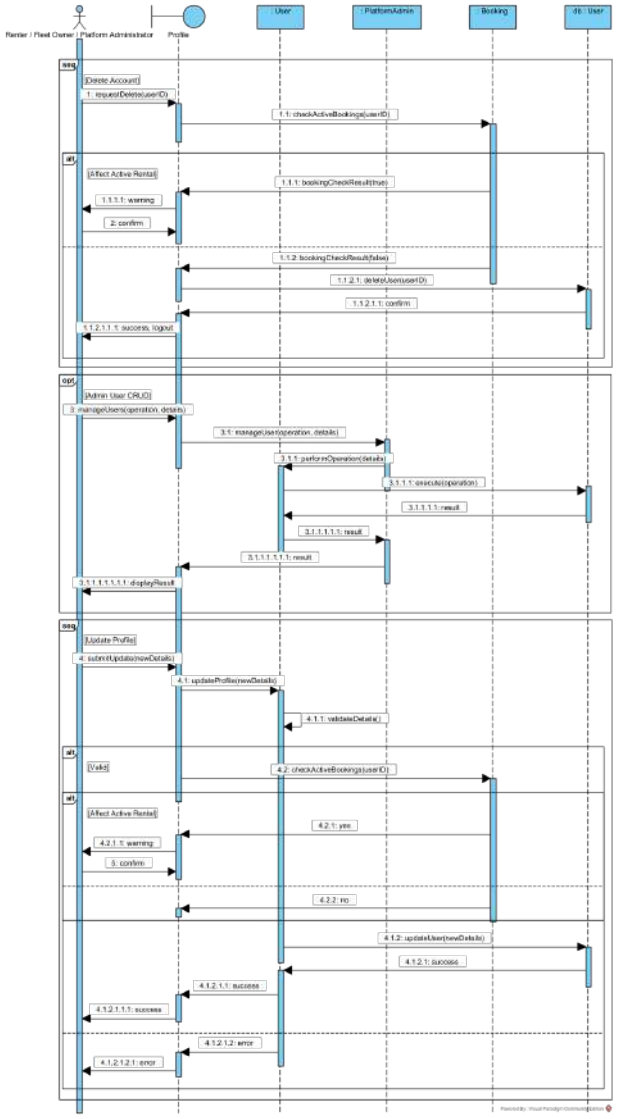


Figure 4.8: Sequence Diagram for User Management

4.4.14 Fleet Management

This sequence diagram models the lifecycle of vehicle assets, specifically focusing on the pricing versioning system and the secure archival process for removed vehicles.

Table 4.12: Sequence Diagram Details: Fleet Management

Diagram ID	SD-02
Diagram Name	Fleet Management
Participants	Fleet Owner, OwnerController, VehicleManagementService, VehicleRepository, VehiclePriceHistoryRepository
Description	Models the CRUD workflows for the vehicle fleet, featuring atomic price versioning and secure asset deactivation.
Precondition	Fleet Owner is authenticated and owns the vehicles being managed.
Postcondition	Vehicle metadata is persisted, a new price version is created (if applicable), or the vehicle is marked as deleted while preserving historical data.
Main Flow	1. **Add Vehicle:** • User submits `AddVehicleRequest` to `OwnerController`. • `OwnerController` calls `VehicleManagementService.createVehicl` • `VehicleManagementService` saves the `Vehicle` entity to the database. • **Price Initialization:** System creates the first record in `VehiclePriceHistoryRepository` using the provided rate and current timestamp.

Continued on next page...

Table 4.12: Sequence Diagram Details: Fleet Management (Continued)

	2. **Update Vehicle & Pricing:** • User submits updated details. • `VehicleManagementService` verifies that the `fleetOwnerId` matches the vehicle's owner. • **Versioning Check:** System compares the new rate with the latest price history record. • If the rate has changed, a **New Price Version** is inserted into `VehiclePriceHistoryRepository` with an effective start date. • Metadata (brand, mileage, etc.) is updated in the `VehicleRepository`. 3. **Soft Delete Vehicle:** • User requests removal of a vehicle. • System verifies there are no "ACTIVE" or "CONFIRMED" rentals in the `BookingRepository`. • `VehicleManagementService` sets `isDeleted = true` and records the `deletedAt` timestamp. • The vehicle is archived in the `VehicleRepository`.
Alternative Flow	• **Price Schedule:** If the owner provides a future `effectiveDate` during an update, the system schedules the price change by inserting a future-dated record in the `VehiclePriceHistoryRepository`.
Exception Flow	• **Unauthorized Access:** If a user attempts to manage a vehicle belonging to another owner, the service throws an exception, and the controller returns an error message.

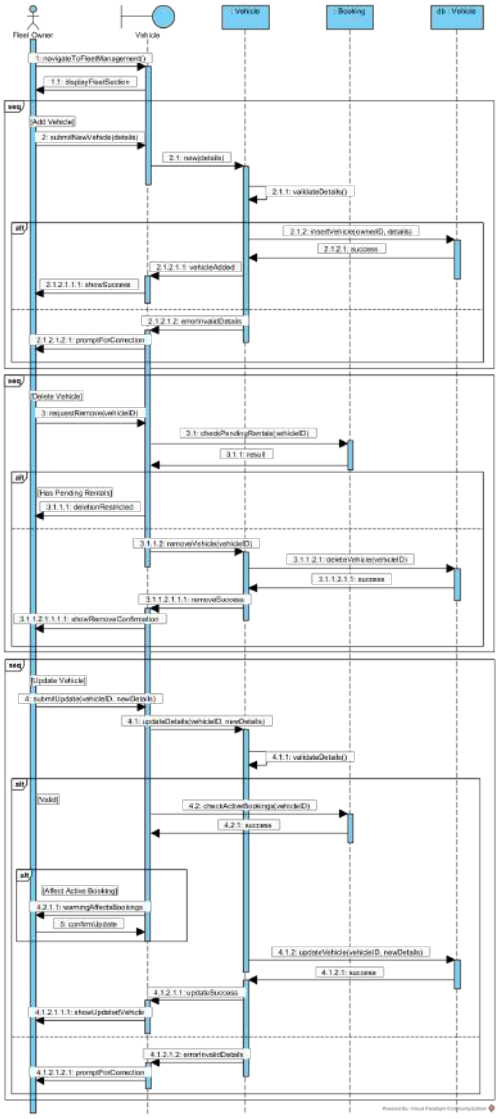


Figure 4.9: Sequence Diagram for Fleet Management

4.4.15 Booking and Payment Management

These sequence diagrams model the end-to-end lifecycle of a rental transaction, from the initial price-locked reservation to the finalization of funds via automated or manual channels.

Table 4.13: Sequence Diagram Details: Booking and Payment Management

Diagram ID	SD-03
Diagram Name	Booking and Payment Management
Participants	Renter, BookingService, InvoiceService, PaymentService, ToyyibPay Gateway, Fleet Owner, db::Repositories
Description	Models the transactional flow of reservations, including cost snapshotting, multi-channel payment processing, and administrative verification.
Precondition	Renter is authenticated; Vehicle is currently "AVAILABLE" or has no status-conflicting bookings.
Postcondition	Booking is moved to "CONFIRMED" status, funds are verified, and an audit log is created for all state transitions.
Main Flow	1. **Create Booking & Invoice:** • Renter sends `createBooking(vehicleId, dates)` to `BookingService`. • `BookingService` fetches the latest rate from `VehiclePriceHistoryRepository`. • `BookingService` creates `BookingPriceSnapshot` and the `Booking` (status: PENDING). • `InvoiceService` generates an `Invoice` (status: ISSUED).

Continued on next page...

Table 4.13: Sequence Diagram Details: Booking and Payment Management (Continued)

	2. **Payment Processing (Automated):** • Renter selects "ToyyibPay" and sends `initPayment()` to `PaymentService`. • `PaymentService` redirects Renter to the `ToyyibPay Gateway`. • After payment, `ToyyibPay` sends a `callback()` to the system. • `PaymentService` updates `Payment` to "PAID" and notifies `BookingService` to set Booking status to "CONFIRMED". 3. **Payment Processing (Manual):** • Renter uploads a receipt; `PaymentService` sets status to "VERIFICATION_PENDING". • Fleet Owner reviews receipt and calls `verifyPayment(paymentId, APPROVED)`. • System updates `Payment` to "PAID" and Booking to "CONFIRMED".
Alternative Flow	• **Payment Failure/Retry:** If the gateway returns "FAILED", the Renter is redirected back to the invoice page with an option to "Retry Payment" or change the method.
Exception Flow	• **Concurrency Conflict:** If two renters confirm the same vehicle simultaneously, the database unique constraint on booking dates triggers a rollback, and the second renter receives a "Vehicle No Longer Available" error.

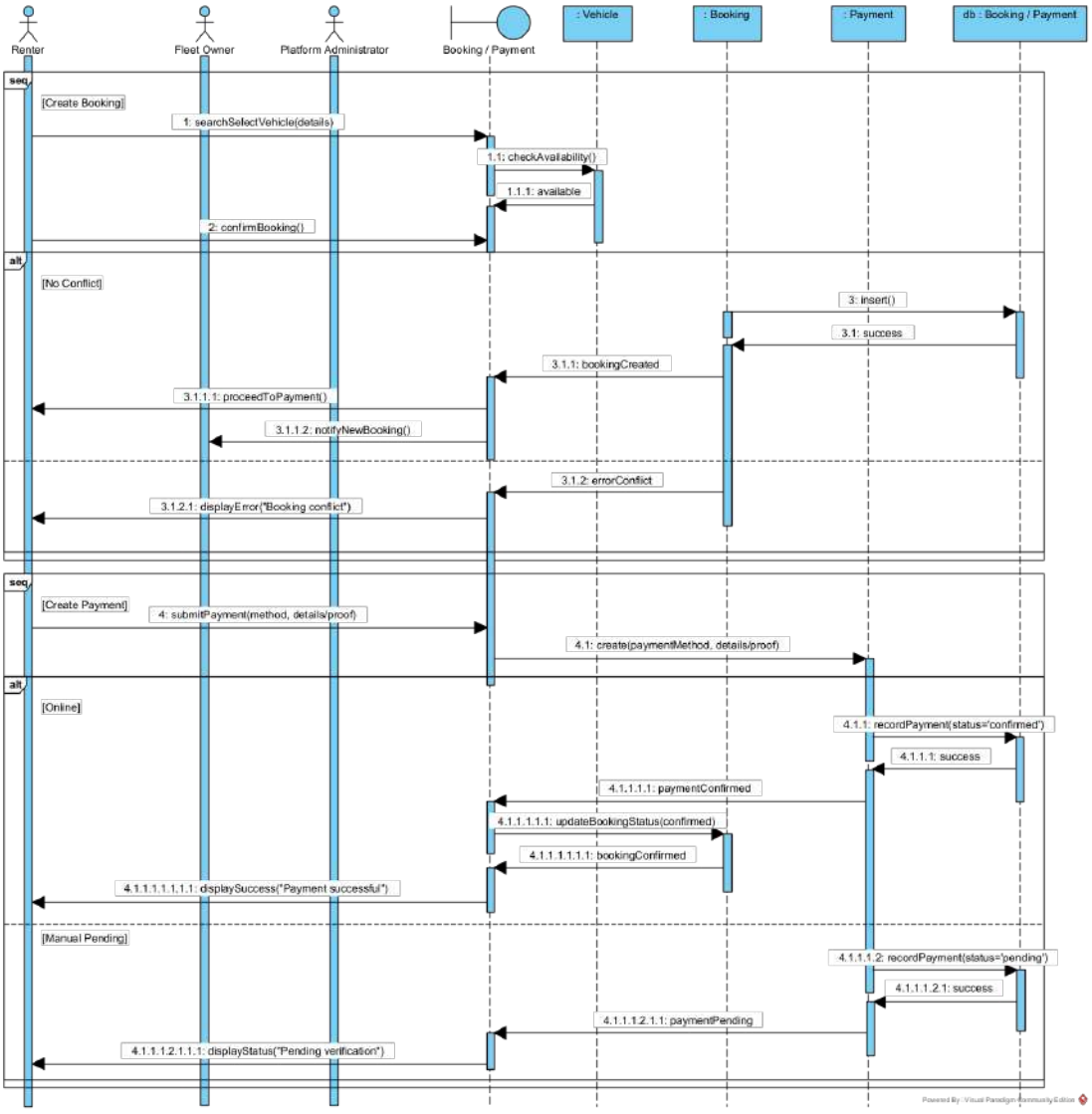


Figure 4.10: Sequence Diagram for Booking and Payment Management (Part 1: Creation)

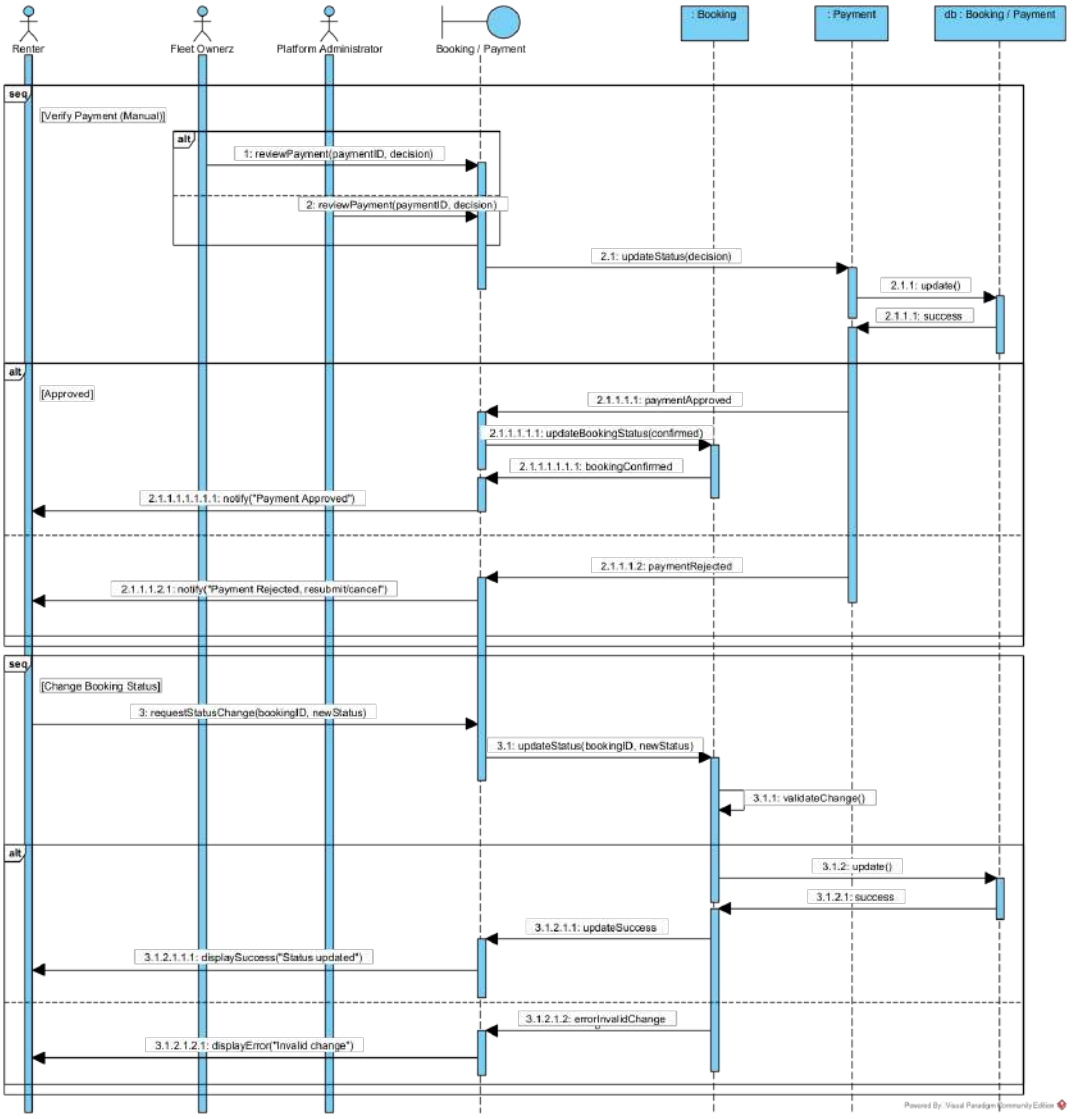


Figure 4.11: Sequence Diagram for Booking and Payment Management (Part 2: Verification and Status Change)

4.4.16 Generate Report

This sequence diagram models the analytical pipeline for converting raw transactional records into visual dashboards and professional export documents.

Table 4.14: Sequence Diagram Details: Generate Report

Diagram ID	SD-04
Diagram Name	Generate Report
Participants	User, Controller, ReportService, ReportChartService, db::Repositories, PDF/Excel Engines
Description	Illustrates the multi-stage process of data aggregation, analytical comparison, chart transformation, and file rendering.
Precondition	User is authenticated and has selected a report category and duration.
Postcondition	Analytical data is rendered on-screen via charts or delivered as a downloadable binary file.
Main Flow	1. **Preview Generation:** • User sends a `ReportRequest` (filters, category) to the `Controller`. • `Controller` invokes `ReportService.generateReportData()`. • `ReportService` performs parallel/sequential queries to `db::Repositories`. • `ReportService` calculates growth metrics (if comparison mode is ON). • `ReportService` calls `ReportChartService` to map data to chart coordinates. • `Controller` returns the aggregated JSON payload for on-screen visualization.

Continued on next page...

Table 4.14: Sequence Diagram Details: Generate Report (Continued)

	2. **Document Export:** • User clicks "Download" (PDF, Excel, or CSV). • `ReportService` initiates the specific engine: – **PDF:** Renders an HTML template and converts it via `PdfRendererBuilder`. – **Excel:** Populates an `SXSSFWorkbook` (Apache POI) for high-performance spreadsheet creation. • System returns the byte stream with appropriate `Content-Type` headers.
Alternative Flow	• **Empty Result Set:** If `Repositories` return no matching records, the `ReportService` returns a flag indicating "NO_DATA", and the `Controller` triggers a placeholder UI view.
Exception Flow	• **Format Error:** If an unsupported export format is requested, the system defaults to PDF and logs an analytical warning.

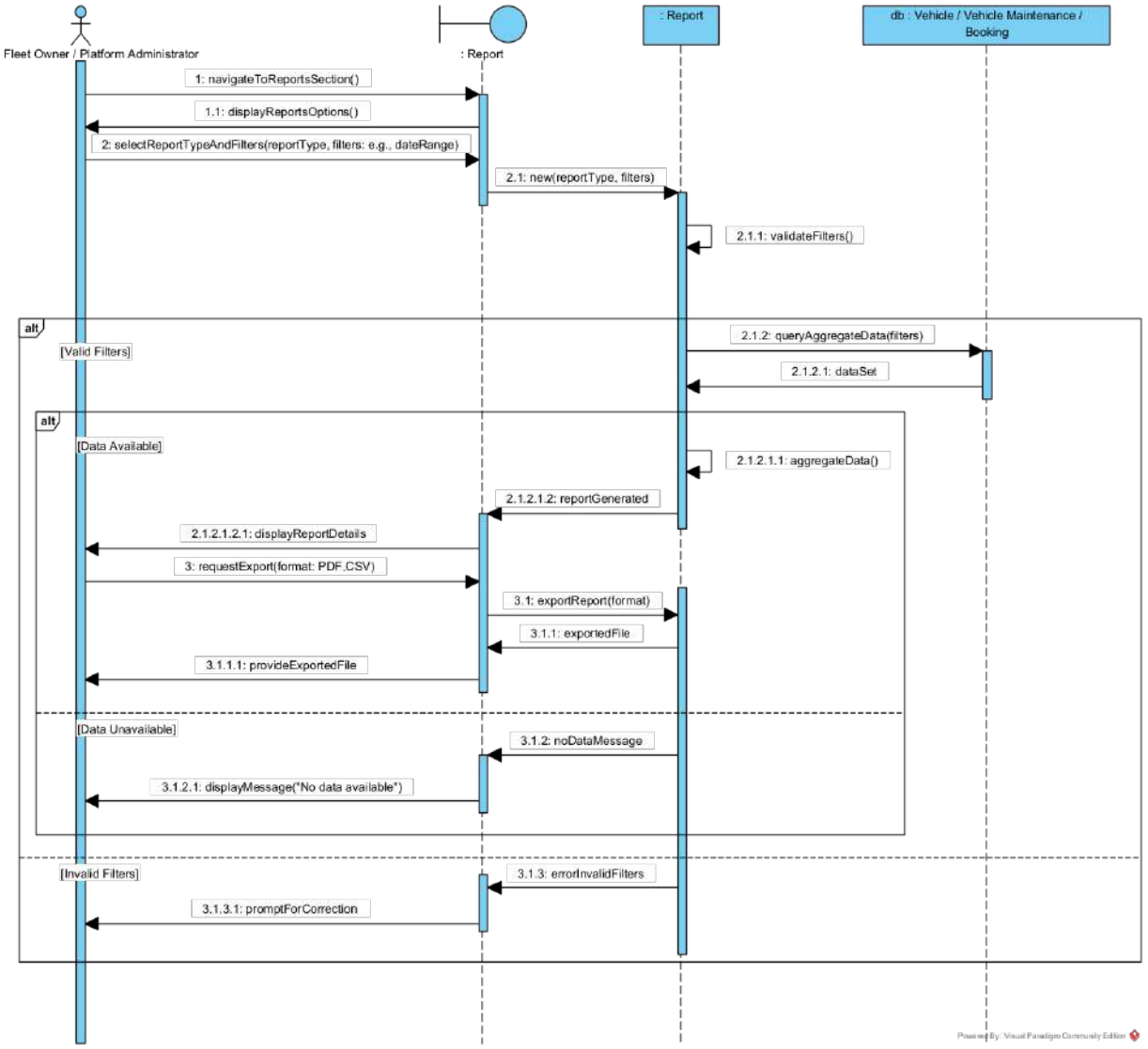


Figure 4.12: Sequence Diagram for Generate Report

4.4.17 Vehicle Maintenance Management

This sequence diagram models the lifecycle of a vehicle service event, highlighting the integration between maintenance scheduling and the automated vehicle availability system.

Table 4.15: Sequence Diagram Details: Vehicle Maintenance Management

Diagram ID	SD-05
Diagram Name	Vehicle Maintenance Management
Participants	User, OwnerController, MaintenanceService, BookingRepository, VehicleRepository, MaintenanceLogRepository
Description	Illustrates the process of service scheduling, booking conflict detection, and automated vehicle status synchronization.
Precondition	Fleet Owner is authenticated and has selected a vehicle from their managed fleet.
Postcondition	Maintenance records are persisted, audit logs are initialized, and the vehicle's availability status is updated in real-time.
Main Flow	1. **Schedule Maintenance:** • User submits `MaintenanceDTO` to `OwnerController`. • `MaintenanceService` invokes `BookingRepository.findOverlappingBookings()`. • **Conflict Handling:** If a conflict is found, the System sends a warning but proceeds with saving if the User confirms the priority. • System saves the record and calls `MaintenanceLogRepository` to init the "PENDING" state.

Continued on next page...

Table 4.15: Sequence Diagram Details: Vehicle Maintenance Management (Continued)

	2. **Execute Service (Status Sync):** • User updates status to `IN_PROGRESS`. • `MaintenanceService` calls `VehicleRepository.updateStatus(MA` • System records the actor and timestamp in the `MaintenanceLogRepository`. • The vehicle is now locked from new bookings. 3. **Complete Service:** • User submits `finalCost` and sets status to `COMPLETED`. • `MaintenanceService` restores vehicle status to `AVAILABLE` via `VehicleRepository`. • System finalizes the audit log.
Alternative Flow	• **Maintenance Cancellation:** If a service is cancelled, the `MaintenanceService` immediately restores the vehicle's availability to prevent unnecessary fleet downtime.
Exception Flow	• **Ownership Breach:** If the `userId` of the requester does not match the vehicle's `fleetOwnerId`, the service aborts the transaction and returns a `403 Forbidden`.

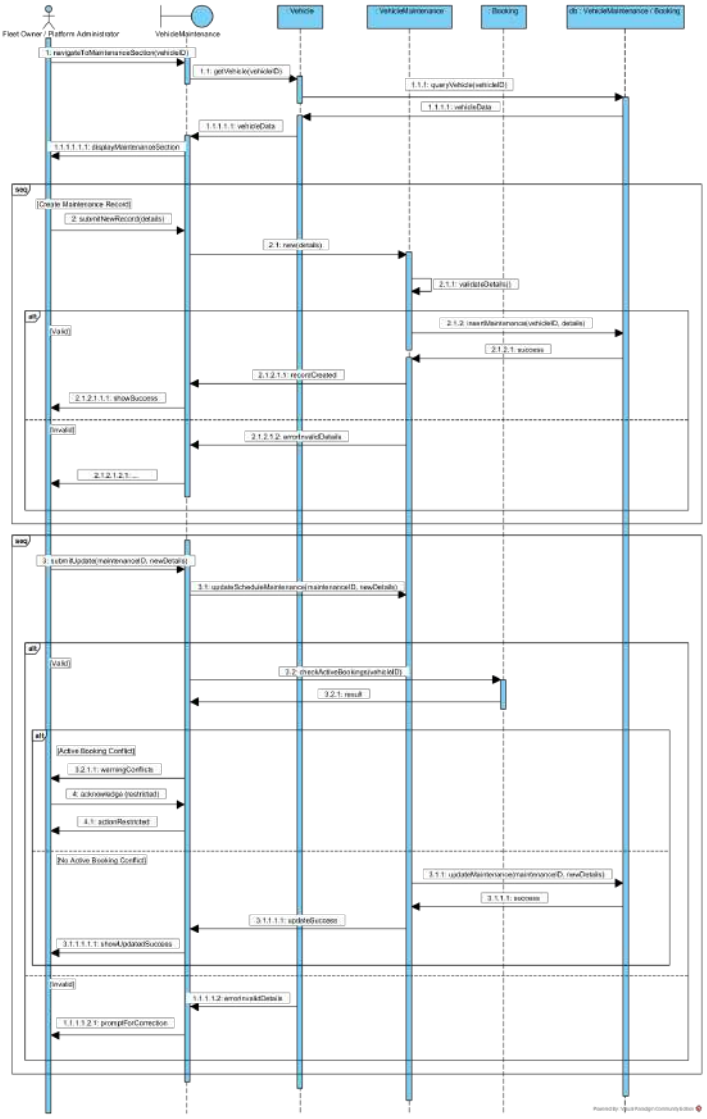


Figure 4.13: Sequence Diagram for Vehicle Maintenance Management

4.5 CRUD Matrix

The CRUD (Create, Read, Update, Delete) technique is a common tool used in systems analysis to map system interactions and validate the completeness of a design (Satzinger et al., 2015). One useful technique to identify how the underlying objects in the problem domain work together to collaborate in support of the use cases is CRUD analysis. CRUD analysis uses a CRUD matrix, in which each interaction among objects is labeled with a letter for the type of interaction: C for create, R for read or reference, U for update and D for delete. In an object-oriented approach, a class/actor-by-class/actor matrix is used. Each cell in the matrix represents the interaction between instances of the classes.

	User	Profile	Vehicle	Booking	Payment	Verification	Maintenance	Report
Fleet Owner	CRUD (own)	CRUD (own)	CRUD	CRUD	RU	RU	CRUD	CR
Renter	CRUD (own)	CRUD (own)	R	CRUD	CR	CR	-	-
Administrator	RU	RU	RU	RUD	RUD	RUD	RU	CR

Table 4.16: CRUD Matrix for FleetShare System

Table 4.16 presents the CRUD matrix for the FleetShare system, mapping the primary actors—Fleet Owner, Renter, and Administrator—against the core functional modules. Each cell defines the access level derived from the system's current Spring Boot implementation.

For instance, a "Fleet Owner" possesses full "CRUD" permissions over their "Vehicle" and "Maintenance" records, enabling them to register new cars and log service history. A "Renter" has "CR" (Create, Read) access to "Payment" and "Verification," as they are responsible for initiating transactions and uploading payment proofs for manual verification. The "Administrator" role focuses on platform governance, holding "RUD" (Read, Update, Delete) privileges over "Verification" and "Bookings" to resolve disputes and validate user credentials. Finally, the "Report" module supports "CR" for both Owners and Admins, allowing them

to generate dynamic analytics via the `ReportService` for fleet performance and platform-wide metrics respectively.

CHAPTER 5

SYSTEM DESIGN

5.1 Introduction

This chapter presents the comprehensive system design of the FleetShare platform, translating the established software requirements into a structured technical blueprint. The chapter begins by detailing the overall system architecture, specifically focusing on the Model-View-Controller (MVC) framework and the logical decomposition of modules for different multi-tenant user roles. Subsequently, it outlines the database design, illustrating the Entity-Relationship Diagram (ERD) and the comprehensive data normalization process up to the Third Normal Form (3NF). Finally, the chapter concludes with the User Interface and User Experience (UI/UX) design strategy, presenting screen prototypes and workflows that demonstrate how independent fleet owners, platform administrators, and renters will interact within the centralized environment.

5.2 System Architecture

The system architecture serves as the foundational blueprint of the FleetShare platform, defining the structural properties, subsystem interactions, and the logical distribution of computational resources (Bass et al., 2021). To fulfill the complex requirements of a centralized, multi-tenant vehicle rental ecosystem, FleetShare is engineered utilizing a **Layered N-Tier Architectural Pattern**, predominantly

driven by the Java Spring Boot framework (Walls, 2022). This approach ensures a rigid separation of concerns, thereby maximizing system scalability, security, and long-term maintainability. This architectural strategy effectively isolates tenant data and custom enterprise workflows while optimizing hardware resource utilization across the entire platform ecosystem (Bezemer and Zaidman, 2010).

5.2.1 Architectural Layers

The application logic is explicitly decoupled into four distinct layers, ensuring that modifications to the user interface do not disrupt the underlying business rules or database schemas (Fowler, 2002).

- **Presentation Layer (Thymeleaf & JS):** Handles the graphical user interface (GUI). It utilizes server-side rendering with Thymeleaf for SEO and initial load speed, combined with modern JavaScript libraries (Chart.js for analytics) to provide an interactive, dashboard-driven experience (Walls, 2022).
- **Controller Layer (Web & API):** Acts as the entry point for all system interactions. Utilizing Spring MVC, this layer handles request routing, session management via `SessionUser` objects, and input validation. It serves both standard web requests and asynchronous AJAX calls for real-time reporting (Leff and Rayfield, 2001).
- **Service Layer (Business Logic):** The "brain" of the system. This layer (e.g., `BookingService`, `MaintenanceService`) encapsulates the core business rules, manages cross-entity transactions, and orchestrates calls to external integrations like the AI engine and Payment Gateways (Alur et al., 2003).
- **Data Access Layer (JPA & DTO):** Implemented using Spring Data JPA to map Java entities to the MySQL database. This layer utilizes **Data Transfer Objects (DTOs)** to project database records into lightweight

structures for the presentation layer, enhancing performance and security (Bauer et al., 2015).

5.2.2 External Integration Infrastructure

To provide a competitive SaaS experience, the architecture incorporates three specialized integration modules (Hohpe and Woolf, 2004):

1. **AI Analytical Hub:** A service-layer integration that enables natural language querying of fleet data, allowing administrators to generate complex reports without manual filtering (Katsogiannis-Meimarakis and Koutrika, 2023).

2. **Payment Processing Engine:** A dual-path engine that handles automated ToyyibPay API callbacks for instant settlements and a manual verification pipeline for bank transfers (Lauret, 2019).

3. **Transactional Messaging:** A background `EmailService` that handles multi-party notifications for booking confirmations, payment failures, and status updates (Newman, 2021).

5.2.3 Multi-Tenant SaaS Infrastructure

FleetShare utilizes a **Shared-Schema Multi-Tenant Model** (Krebs et al., 2012). Isolation is achieved logically at the application level; every operational record—including vehicles, maintenance logs, and financial transactions—is strictly bound to a specific `fleetOwnerId` (Kabbedijk et al., 2012). The Service Layer enforces mandatory "Ownership Verification" during every transaction, ensuring that Fleet Owners are digitally partitioned from each other while utilizing the same high-performance application instance. As illustrated in the layered N-Tier architecture (Figure 5.1), this authorization logic is executed specifically within the Business Services layer

before any data access operations are performed by the Repositories. This architectural configuration maximizes database resource pooling efficiency while requiring robust application-level access control mechanisms to mitigate cross-tenant data leakage risks (Bezemer and Zaidman, 2010).

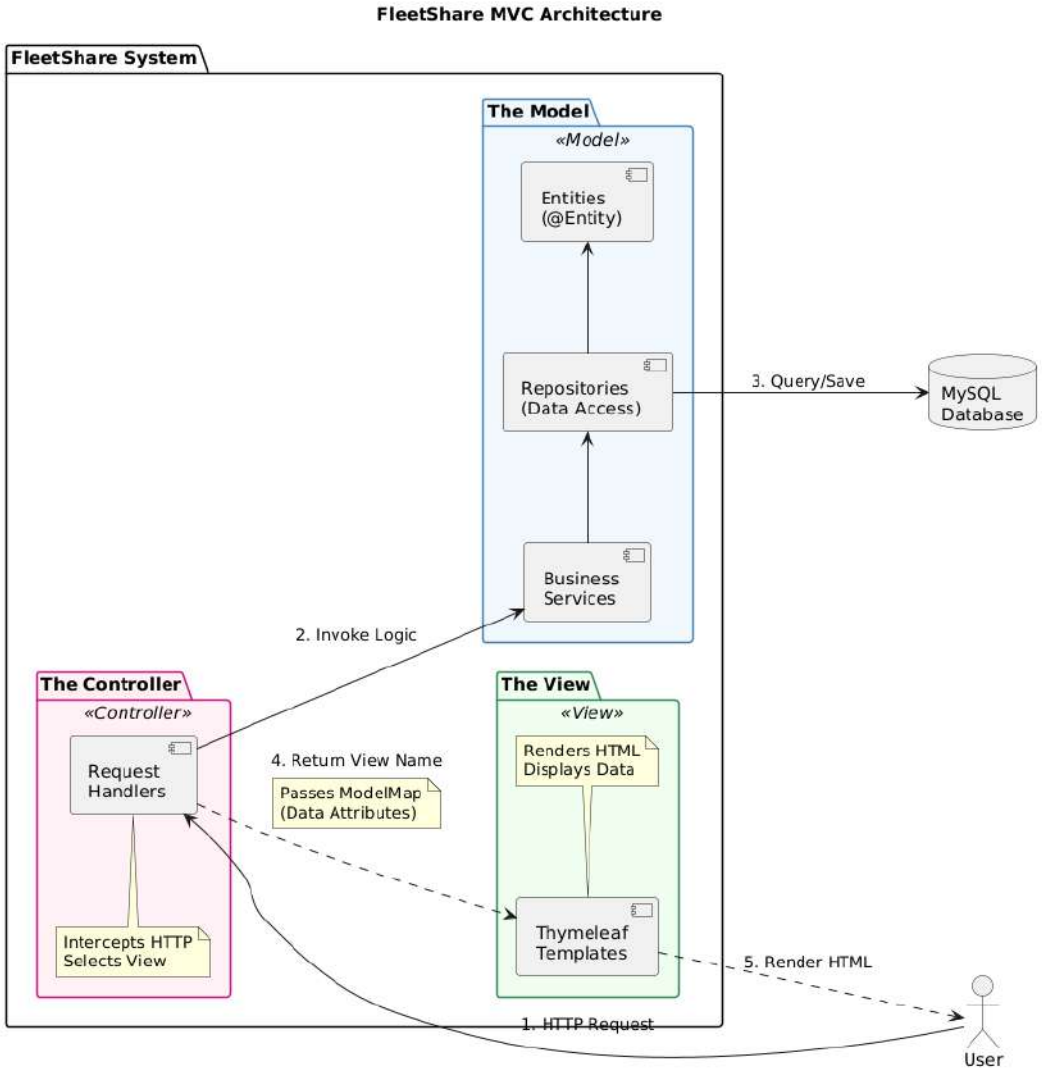


Figure 5.1: Layered N-Tier Architecture of the FleetShare Platform

5.3 Package Diagram

This document provides a comprehensive overview of the internal package structure for the FleetShare Spring Boot application (`com.najmi.fleetshare`). As illustrated in the structural breakdown of Figure 5.2, the architecture is modularly organized into distinct, decoupled layers, strictly adhering to the principles of separation of concerns and N-Tier architecture (Fowler, 2002).

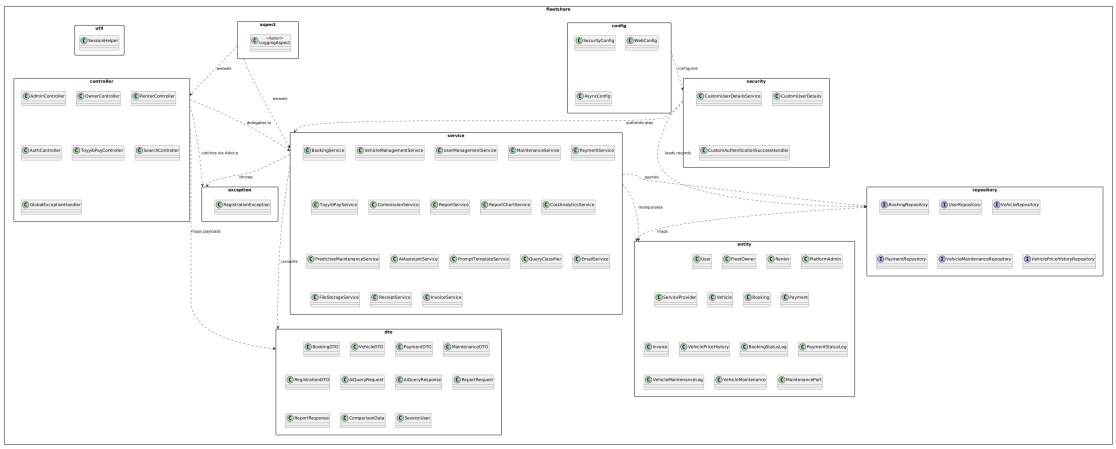


Figure 5.2: Package diagram of the FleetShare application

Package Details

1. `controller` (Web Layer)

Acts as the presentation tier, responsible for intercepting incoming HTTP requests, orchestrating input validation, and mapping requests to the appropriate business logic services (Walls, 2022). It returns either structured data responses or Thymeleaf-rendered views. The controllers are architecturally partitioned by domain actors and functional features.

- **Key Classes:**
  - `AdminController`, `OwnerController`, `RenterController`: Dashboards and management portals partitioned by user roles.

- `AuthController`: Manages authentication flows, including login, registration, and session initialization.
- `ToyyibPayController`: Exposes secure webhook endpoints for the payment gateway callbacks.
- `SearchController`: Provides the API for public vehicle discovery and filtering.
- `GlobalExceptionHandler`: Uses `@ControllerAdvice` to globally intercept and format HTTP error responses across all controllers (Walls, 2022).

2. `service` (Business Logic Layer)

Encapsulates the core business logic, workflow algorithms, and transactional boundaries. This layer acts as the intermediary engine, orchestrating complex interactions between the web controllers and the underlying data access mechanisms (Fowler, 2002).

- **Key Classes:**
  - **Core Operations:** `BookingService`, `VehicleManagementService`, `UserManagement` `MaintenanceService`.
  - **Financial Processing:** `PaymentService`, `ToyyibPayService`, `CommissionService`.
  - **Data & Analytics:** `ReportService`, `ReportChartService`, `CostAnalyticsService` `PredictiveMaintenanceService`.
  - **AI Integration:** `AiAssistantService`, `PromptTemplateService`, `QueryClassifier`
  - **Infrastructure:** `EmailService`, `FileStorageService`, `ReceiptService`, `InvoiceService`.

3. `repository` (Data Access Layer)

Comprises Spring Data JPA interfaces that provide an abstraction layer over data storage. This tier facilitates seamless CRUD operations and executes complex custom database queries using JPQL and native SQL, bridging the object-oriented domain with the MySQL database (Bauer et al., 2015).

- **Key Classes:** `UserRepository`, `VehicleRepository`, `BookingRepository`, `PaymentRepository`, `VehicleMaintenanceRepository`, `VehiclePriceHistoryRepositor`

4. `entity` (Domain Model Layer)

Forms the core domain model representing the relational database schema. These JPA-annotated classes map directly to database tables, explicitly defining structural constraints and cascading behaviors (Bauer et al., 2015).

- **Key Classes:**
  - **Actors:** `User`, `FleetOwner`, `Renter`, `PlatformAdmin`, `ServiceProvider`.
  - **Core:** `Vehicle`, `Booking`, `Payment`, `Invoice`.
  - **Tracking/Logs:** `VehiclePriceHistory`, `BookingStatusLog`, `PaymentStatusLog`, `VehicleMaintenanceLog`.
  - **Maintenance:** `VehicleMaintenance`, `MaintenancePart`.

5. `dto` (Data Transfer Objects)

Functions as lightweight, serialized payloads used to securely transfer data across subsystem boundaries. DTOs are crucial for decoupling internal database entities from external API contracts and preventing sensitive information leakage (Fowler, 2002).

- **Key Classes:**

- **Standard Payloads:** `BookingDTO`, `VehicleDTO`, `PaymentDTO`, `MaintenanceDTO`, `RegistrationDTO`.
- **AI Features:** `AiQueryRequest`, `AiQueryResponse`.
- **Analytics:** `ReportRequest`, `ReportResponse`, `ComparisonData`.
- **State Management:** `SessionUser` (Cached user state representation).

6. `security` (Security & Auth Layer)

Houses comprehensive Spring Security configurations and authentication mechanisms. This layer manages session states, enforces Role-Based Access Control (RBAC), and securely handles user credential loading (Ferraiolo et al., 2001).

- **Key Classes:**
  - `CustomUserDetailsService`: Implements standard interfaces to load user-specific data during authentication.
  - `CustomUserDetails`: Extends standard security user definitions with platform-specific fields.
  - `CustomAuthenticationSuccessHandler`: Dynamically routes users to appropriate dashboards (Admin/Owner/Renter) based on their role upon successful login.

7. `config` (Configuration Layer)

Centralizes the application-wide configuration classes. These Spring `@Configuration` components instantiate beans and define system settings (Walls, 2022).

- **Key Classes:**
  - `SecurityConfig`: Defines HTTP security rules, CORS policies, password encoders, and login/logout behaviors.

  - `WebConfig`: MVC configurations, particularly resource handlers for static assets and file uploads.
  - `AsyncConfig`: Thread pool configurations for asynchronous and non-blocking tasks (e.g., dispatching emails).

8. `exception` (Custom Error Definitions)

Defines custom business exceptions to explicitly categorize and handle domain-specific errors gracefully without exposing underlying stack traces (Walls, 2022).

- **Key Classes:**
  - `RegistrationException`: Thrown during anomalous user registration flows.

9. `aspect` (AOP Layer)

Leverages Aspect-Oriented Programming (AOP) to abstract cross-cutting concerns—such as audit logging, transaction monitoring, and execution profiling—away from the core business logic, ensuring cleaner, more maintainable codebases (Kiczales et al., 1997).

10. `util` (Utility Layer)

Contains stateless utility classes, constants, and generalized helper methods. These components provide shared functionality across the application.

- **Key Classes:**
  - `SessionHelper`: Provides static abstractions for session data extraction and formatting.

5.4 Database Design

This chapter presents the comprehensive relational database design engineered for the FleetShare platform, enabling individuals and business entities to seamlessly list, manage, and monetize their vehicle fleets. The architecture strictly adheres to relational database principles and has been systematically normalized to the Third Normal Form (3NF). As illustrated in the Entity-Relationship Diagram (ERD) in Figure 5.3, this structured normalization ensures robust data integrity, comprehensively eliminates data redundancy, and heavily optimizes query performance to support concurrent operations within a scalable, multi-tenant environment (Elmasri and Navathe, 2016).

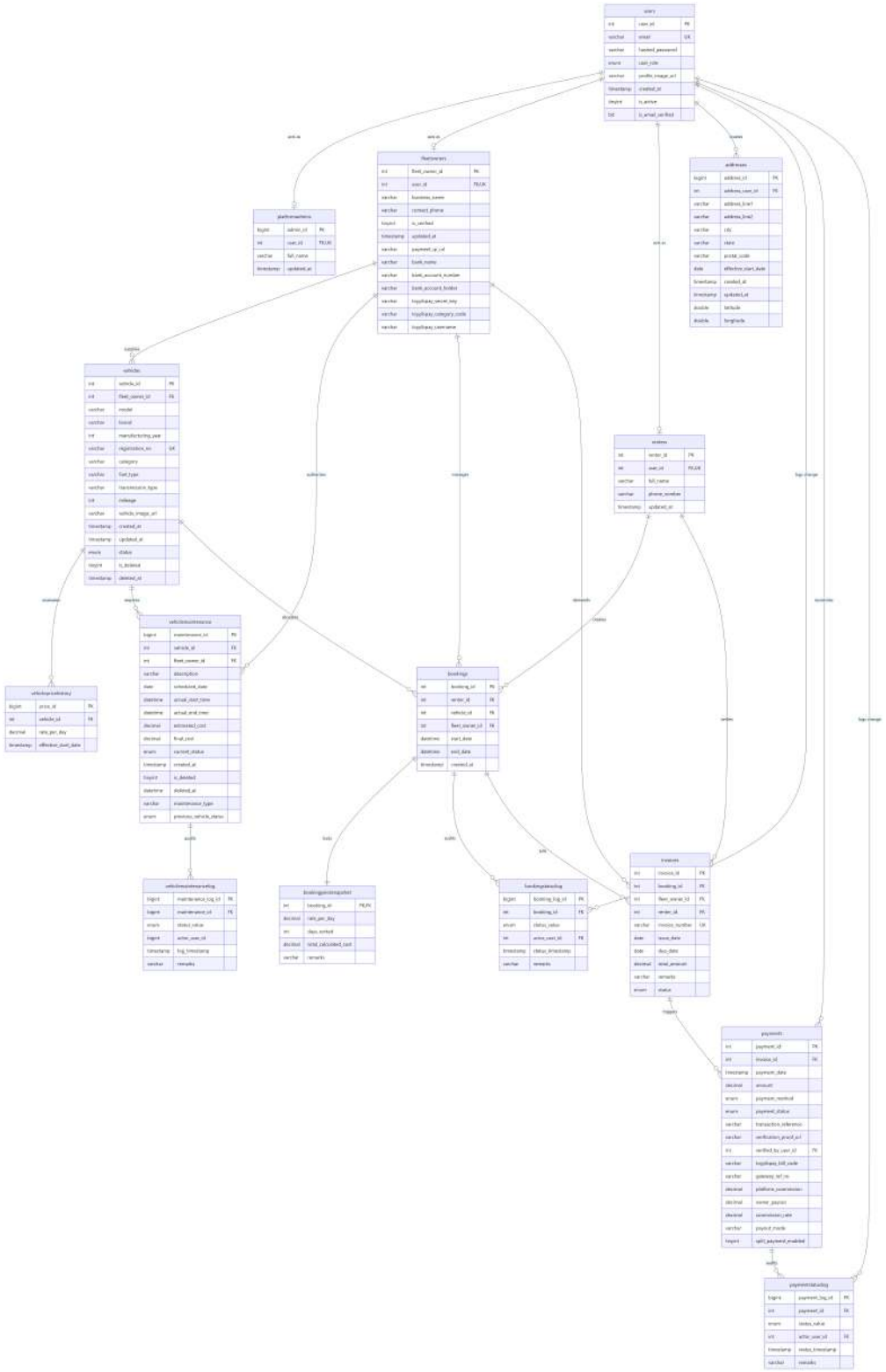


Figure 5.3: Entity-Relationship Diagram (ERD) of the FleetShare database schema

5.4.1 Normalization

Normalization is a critical database design methodology that decomposes complex, interwoven data structures into smaller, tightly cohesive relations, rendering the resulting model highly flexible and transactionally reliable (Codd, 1970). While this methodology introduces minor computational overhead due to the necessity of relational joins, it fundamentally enables scalable, long-term data management without requiring extensive structural refactoring as the application evolves (Date, 2004).

Zero Normal Form: 0NF

The unnormalized form (0NF) represents a conceptual starting point where all data for the entire application is imagined as a single, monolithic, flat table (Elmasri and Navathe, 2016). This raw data structure contains no atomic values and is characterized by severe data redundancy, insertion anomalies, and deeply nested repeating groups (*) (Silberschatz et al., 2019).

```
FleetShare_UNF (
    -- User-specific details
    user_id, email, hashed_password, user_role, profile_image_url, created_at, i
    is_email_verified,

    -- Renter-specific details
    renter_id, renter_full_name, renter_phone_number, renter_updated_at,

    -- FleetOwner-specific details
    fleet_owner_id, business_name, contact_phone, is_verified, owner_updated_at
    bank_name, bank_account_number, bank_account_holder, toyyibpay_secret_key,
    toyyibpay_category_code, toyyibpay_username,

    -- Admin-specific details
    admin_id, admin_full_name, admin_updated_at,

        -- Repeating Group: User Addresses
        (address_id, address_line1, address_line2, city, state, postal_code, lat
        longitude, effective_start_date, address_created_at, address_updated_at)

    -- Repeating Group: Fleet Owner's Vehicles
    (vehicle_id, model, brand, manufacturing_year, registration_no, category, fu
    transmission_type, mileage, vehicle_image_url, vehicle_status, vehicle_creat
    vehicle_updated_at, is_deleted, deleted_at)*,

        -- Nested Repeating Group: Vehicle Price History
            (price_id, rate_per_day, price_effective_start_date)*,
```

```
    -- Nested Repeating Group: Vehicle Maintenance
        (maintenance_id, description, maintenance_type, scheduled_date, actu
        actual_end_time, estimated_cost, final_cost, current_status, previou
        maintenance_created_at, maintenance_is_deleted, maintenance_deleted_

        -- Deep Nested Group: Maintenance Logs
            (maintenance_log_id, maintenance_status_value, maintenance_actor
            maintenance_log_timestamp, maintenance_remarks)*
)*,

-- Repeating Group: Renter's Bookings
(booking_id, start_date, end_date, booking_created_at,

    -- Nested Repeating Group: Booking Status Log
    (booking_log_id, booking_status_value, booking_status_timestamp,
    booking_actor_user_id, booking_remarks)*,

    -- Nested Attribute: Booking Price Snapshot
    (snapshot_rate_per_day, snapshot_days_rented, snapshot_total_calculated_

    -- Nested Attribute: Invoice
    (invoice_id, invoice_number, issue_date, due_date, total_amount, invoice
    invoice_remarks,

    -- Nested Repeating Group: Payments
        (payment_id, payment_date, amount, payment_method, payment_status,
        transaction_reference, verification_proof_url, verified_by_user_id,
        toyyibpay_bill_code, gateway_ref_no, platform_commission, commissio
```

```
            owner_payout, payout_mode, split_payment_enabled)*,

            -- Deep Nested Group: Payment Status Log
                (payment_log_id, payment_status_value, payment_status_timestamp
                payment_actor_user_id, payment_remarks)*
            )*
        )*
    )*
)
```

One Normal Form: 1NF

Applying the formal decomposition process to the FleetShare UNF structure results in the following comprehensive set of 1NF relations, ensuring each table possesses a defined primary key and strictly atomic attributes.

Table Name (Relation)	Primary Key (PK)
Users	user_id
Addresses	address_id
FleetOwners	fleet_owner_id
Renters	renter_id
PlatformAdmins	admin_id
Vehicles	vehicle_id
VehiclePriceHistory	price_id
VehicleMaintenance	maintenance_id
VehicleMaintenanceLog	maintenance_log_id
Bookings	booking_id
BookingStatusLog	booking_log_id
BookingPriceSnapshot	booking_id
Invoices	invoice_id
Payments	payment_id
PaymentStatusLog	payment_log_id

Second Normal Form: 2NF

An analytical review of the 1NF Relations reveals a critical architectural design choice: every single relation in the schema utilizes a single-column, surrogate primary key (e.g., *user_id*, *vehicle_id*, *booking_id*). Because partial dependencies

mathematically can only exist in tables possessing composite primary keys, it is structurally impossible for a partial dependency anomaly to occur within this schema. Therefore, the database is inherently advanced to the Second Normal Form (2NF) by design.

Third Normal Form: 3NF

The schema successfully eliminates transitive dependencies by isolating distinct entities into dedicated relations and explicitly snapshotting temporal financial data:

- **Financial Integrity (Temporal Data):** The `total_cost` of a historical booking is transitively dependent on the active `rate_per_day` of a vehicle at the specific moment of reservation. If a fleet owner modifies a vehicle's rental price in the future, it would erroneously and retroactively alter past financial records if merely joined. The schema programmatically resolves this by introducing the `BookingPriceSnapshot` table, immutably locking the `rate_per_day` and `days_rented` to the `booking_id` during checkout.
- **Status Auditing & Traceability:** Rather than perpetually overriding a singular status string in a booking, maintenance, or payment record, the system utilizes append-only log tables (`BookingStatusLog`, `PaymentStatusLog`, and `VehicleMaintenanceLog`). This rigorously isolates temporal state changes, directly linking each transition to the specific `actor_user_id` (System, Admin, Owner, or Renter) responsible for the mutation.

5.4.2 Data Dictionary

The following subsections meticulously detail the schema definition for each core entity based on the physical database implementation.

5.4.3 Users

Table 5.1: Entity: Users

Attribute	Data Type	Description / Constraints
user_id	INT	PK, AI
email	VARCHAR(255)	Unique registered email address
hashed_password	VARCHAR(255)	Secure Bcrypt hashed password
user_role	ENUM	('PLATFORM_ADMIN', 'FLEET_OWNER', 'RENTER')
profile_image_url	VARCHAR(1024)	Path URI to uploaded user avatar
is_active	TINYINT(1)	Account accessibility (Soft delete logic)
is_email_verified	BIT(1)	Email verification confirmation flag
created_at	TIMESTAMP	Record initialization timestamp

5.4.4 Fleet Owners

Table 5.2: Entity: Fleet Owners

Attribute	Data Type	Description / Constraints
fleet_owner_id	INT	PK, AI
user_id	INT	FK → Users (Unique constraint)
business_name	VARCHAR(255)	Registered business entity name
contact_phone	VARCHAR(255)	Official business contact number
payment_qr_url	VARCHAR(1024)	Path URI to DuitNow/QR code image
bank_name	VARCHAR(100)	Financial institution for manual payouts
bank_account_number	VARCHAR(50)	Registered bank account number
bank_account_holder	VARCHAR(100)	Registered name of the account holder
toyyibpay_secret_key	VARCHAR(255)	Encrypted API secret for payment gateway
toyyibpay_category_code	VARCHAR(100)	ToyyibPay assigned billing category
toyyibpay_username	VARCHAR(100)	Registered username for the gateway
is_verified	TINYINT(1)	Boolean flag indicating platform approval
updated_at	TIMESTAMP	Audit timestamp for last record update

5.4.5 Addresses

Table 5.3: Entity: Addresses

Attribute	Data Type	Description / Constraints
address_id	BIGINT	PK, AI
address_user_id	INT	FK → Users (Polymorphic ownership mapping)
address_line1	VARCHAR(255)	Primary street address
address_line2	VARCHAR(255)	Secondary address (Unit/Floor/Building)
city	VARCHAR(100)	City or municipal district
state	VARCHAR(100)	State or province
postal_code	VARCHAR(20)	Postal or ZIP code
latitude	DOUBLE	Precise GPS latitude coordinate for mapping
longitude	DOUBLE	Precise GPS longitude coordinate for mapping
effective_start_date	DATE	Temporal validity marker for address history
created_at	TIMESTAMP	Record initialization timestamp
updated_at	TIMESTAMP	Record modification timestamp

5.4.6 Vehicles

Table 5.4: Entity: Vehicles

Attribute	Data Type	Description / Constraints
vehicle_id	INT	PK, AI
fleet_owner_id	INT	FK → FleetOwners
model	VARCHAR(100)	Specific vehicle model nomenclature
brand	VARCHAR(100)	Vehicle manufacturer brand
manufacturing_year	INT	Year of production
registration_no	VARCHAR(20)	Official license plate identifier
category	VARCHAR(50)	Vehicle class (e.g., Sedan, SUV, Hatchback)
fuel_type	VARCHAR(50)	Powertrain fuel requirement (Petrol, Electric)
transmission_type	VARCHAR(50)	Gearbox mechanism (Automatic, Manual)
mileage	INT	Current odometer reading
vehicle_image_url	VARCHAR(1024)	URI to primary vehicle photograph
status	ENUM	('AVAILABLE', 'MAINTENANCE', 'RENTED', 'UNAVAILABLE')
created_at	TIMESTAMP	Record creation timestamp
updated_at	TIMESTAMP	Record modification timestamp
is_deleted	TINYINT(1)	Boolean flag for soft-delete archival
deleted_at	TIMESTAMP	Precise timestamp of soft deletion

5.4.7 Vehicle Maintenance

Table 5.5: Entity: Vehicle Maintenance

Attribute	Data Type	Description / Constraints
maintenance_id	BIGINT	PK, AI
vehicle_id	INT	FK → Vehicles
fleet_owner_id	INT	FK → FleetOwners
description	VARCHAR(1000)	Comprehensive details of service performed
maintenance_type	VARCHAR(100)	Categorization of the maintenance event
scheduled_date	DATE	Intended future date of service
actual_start_time	DATETIME	Exact timestamp when service commenced
actual_end_time	DATETIME	Exact timestamp when service concluded
estimated_cost	DECIMAL(10,2)	Projected financial expenditure
final_cost	DECIMAL(10,2)	Actual billed financial cost
current_status	ENUM	('PENDING', 'IN_PROGRESS', 'COMPLETED', 'CANCELLED')
previous_vehicle_status	ENUM	Original vehicle state prior to maintenance
created_at	TIMESTAMP	Record initialization timestamp
is_deleted	TINYINT(1)	Boolean flag for soft-delete archival
deleted_at	DATETIME	Precise timestamp of soft deletion

5.4.8 Payments

Table 5.6: Entity: Payments

Attribute	Data Type	Description / Constraints
payment_id	INT	PK, AI
invoice_id	INT	FK → Invoices
payment_date	TIMESTAMP	Timestamp when the transaction occurred
amount	DECIMAL(10,2)	Total monetary value processed
payment_method	ENUM	('CREDIT_CARD', 'BANK_TRANSFER', 'QR_PAYMENT', 'CASH')
payment_status	ENUM	('PENDING', 'VERIFIED', 'FAILED')
transaction_reference	VARCHAR(255)	Unique identifier from external payment vendor
verification_proof_url	VARCHAR(1024)	URI to uploaded manual payment receipt
verified_by_user_id	INT	FK → Users (Admin or Owner who verified)
toyyibpay_bill_code	VARCHAR(50)	Specific bill reference ID from ToyyibPay gateway
gateway_ref_no	VARCHAR(100)	Secondary gateway tracking identifier
platform_commission	DECIMAL(10,2)	Calculated commission fee retained by platform
commission_rate	DECIMAL(5,4)	The percentage rate applied for the commission
owner_payout	DECIMAL(10,2)	Net funds remitted to the Fleet Owner
payout_mode	VARCHAR(20)	Disbursement method (e.g., 'BYOK' for direct)
split_payment_enabled	TINYINT(1)	Boolean flag indicating automated fund routing

5.5 User Interface and User Experience (UI/UX) Design

This chapter outlines the advanced User Interface (UI) and User Experience (UX) design architecture engineered for the FleetShare platform. The design paradigm prioritizes intuitive navigation, universal accessibility, and operational efficiency, empowering both individual users and commercial fleet businesses to seamlessly manage and rent vehicle fleets (Nielsen, 1993). Key structural design principles include mobile-first responsive layouts leveraging the Bootstrap 5 CSS framework, explicit visual hierarchies, consistent branding via a centralized styling system, and user-centered interactions that minimize cognitive load while delivering immediate feedback (Shneiderman et al., 2016). By systematically implementing fluid presentation layouts via the Thymeleaf server-side rendering engine, the system effectively reduces user friction and preserves functional parity across both expansive desktop monitors and constrained mobile viewports (Marcotte, 2011).

5.5.1 Overview of User Interface Architecture

The FleetShare interface is structurally crafted to facilitate a frictionless, peer-to-peer vehicle rental ecosystem. From a user's perspective, the system is securely accessed via a web-based portal featuring strictly role-based layout templates (`layout:decorate`) that dynamically adapt to the authenticated user's profile (Fleet Owner, Renter, or Platform Administrator) (Shneiderman et al., 2016). As depicted in the high-level user flow diagram (Figure **??**), the layout strategy maintains contextual consistency while offering distinctly partitioned functionalities:

- **Fleet Owners** are presented with a comprehensive management portal to intuitively add vehicles, dynamically set rental pricing, and track live bookings. The dashboard incorporates rich visual Key Performance Indicators (KPIs) and interactive JavaScript-driven grid-to-table toggles to manage vast vehicle inventories effortlessly.

- **Renters** interface with a visually engaging discovery portal to browse available vehicles utilizing granular, real-time filters (e.g., manufacturing year, vehicle category, status). The interface provides detailed vehicle specifications, immersive imagery, and a streamlined, multi-step checkout workflow integrating the ToyyibPay payment gateway.

- **Administrators** utilize an overarching command center equipped with high-level data visualization tools for comprehensive user management, transactional dispute resolution, and generating system-wide, AI-assisted analytics reports.

This rigid, role-specific structural partitioning limits cognitive friction and comprehensively optimizes workflow coordination across disparate tenant actions without compromising the system's shared architecture (Few, 2013).

To further optimize the user experience, sophisticated feedback mechanisms are inherently woven into the front-end logic. This includes utilizing Material Design Icons (`mdi`) for intuitive visual clarity, dynamic success and error toasts (e.g., "Booking confirmed" or "Invalid date selected"), loading indicators during asynchronous form submissions, and contextual tooltips for complex interactive elements (Nielsen, 1993). Ultimately, the interface ensures that critical mission workflows—such as secure payment processing, vehicle handovers, and predictive report generation—are executed with minimal navigational steps to significantly enhance overall usability and user satisfaction (Norman, 2013).

5.5.2 Screen Images

This section presents screenshots of the FleetShare system interfaces. These screens are organized by module general User, Client, and Admin to demonstrate the specific user flows and functionality available to each role (Cooper et al., 2014). While actual implementation details may vary, these figures represent the core design structure and user experience. Providing clear visual representations within

technical software documentation has been shown to significantly diminish textual ambiguity and enhance system comprehension among cross-functional stakeholders (Rüping, 2003).

General User Access

This module covers the entry points for the system, including secure authentication and new account registration.

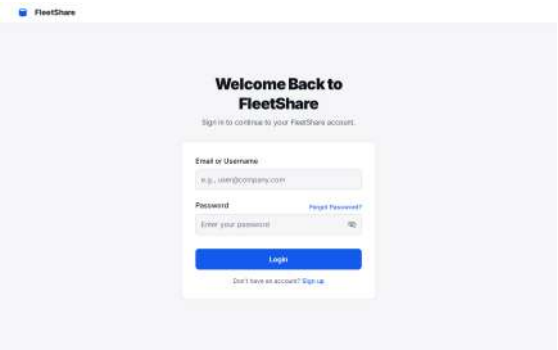


Figure 5.4: User Login: Secure entry point featuring email/password authentication.

The login interface (Figure 5.4) allows registered users to authenticate into the FleetShare platform. It features a centered form with input fields for email and password, a primary submission button, and secondary links for password recovery or new user registration.

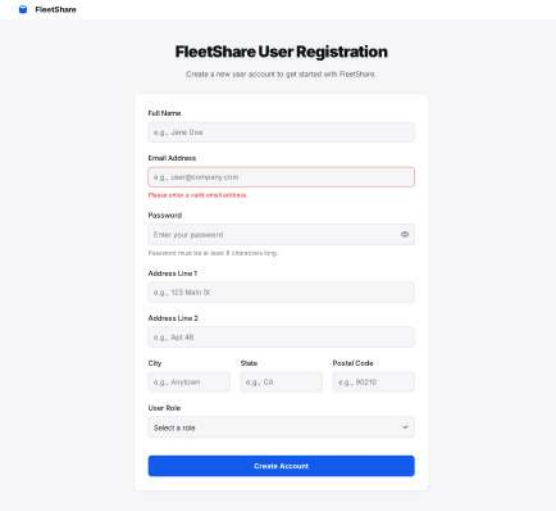


Figure 5.5: User Registration: Sign-up form for creating new FleetShare accounts.

The registration interface (Figure 5.5) facilitates new account creation by collecting personal details, contact information, and physical address data. It includes input fields for role selection and password definition, utilizing real-time validation feedback to highlight errors such as invalid email formats prior to submission.

Client Module

The Client module focuses on the renter's flow, facilitating vehicle browsing, booking management, payments, and profile updates.

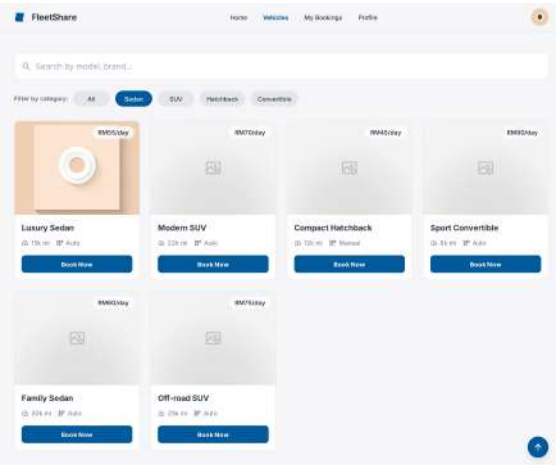


Figure 5.6: Browse Vehicle: Search interface allowing clients to filter and view available cars.

The vehicle browsing interface (Figure 5.6) enables clients to discover available vehicles using a keyword search bar and categorical filters. Results are displayed in a responsive grid layout, where each card presents key vehicle details including daily rental rate, mileage, and transmission type alongside a "Book Now" button to initiate the reservation process.

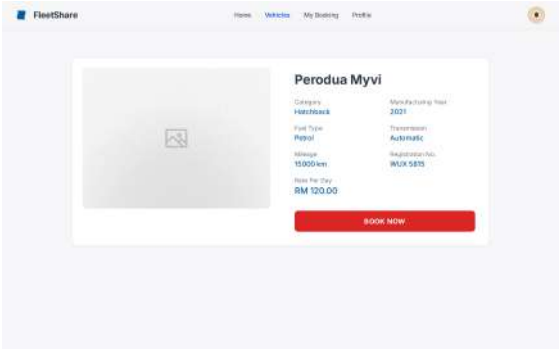


Figure 5.7: Vehicle Summary: Detailed specifications and information about a selected vehicle.

The vehicle summary interface (Figure 5.7) presents detailed specifications for a specific car selected by the user. It utilizes a split-card layout featuring a prominent vehicle image alongside key technical data, including transmission type, fuel category, and daily rental rate. A distinct call-to-action button allows users to immediately proceed to the booking confirmation phase.

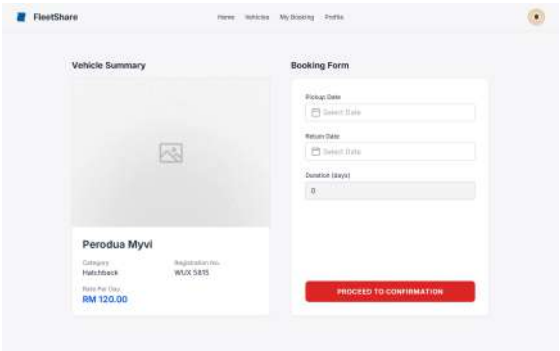


Figure 5.8: Booking Form: Interface for selecting rental dates and preferences.

The booking interface (Figure 5.8) captures reservation details while displaying a static summary of the selected vehicle to ensure booking accuracy. It features

interactive date pickers for scheduling the pickup and return, automatically calculates the total rental duration, and provides a clear call-to-action to proceed to the confirmation stage.

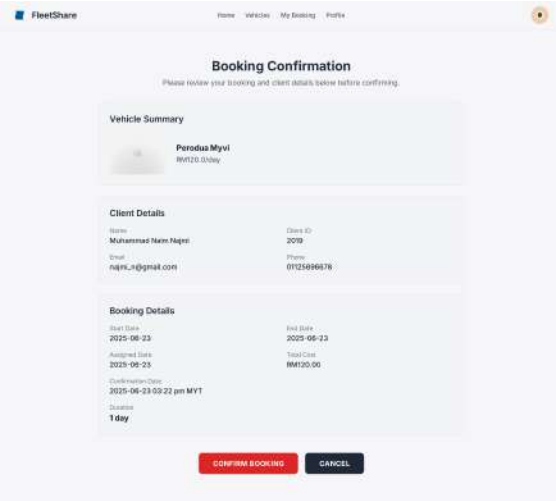


Figure 5.9: Confirm Booking: Summary review screen before proceeding to payment.

The confirmation interface (Figure 5.9) provides a comprehensive review of the reservation details prior to finalization. It aggregates information into segmented panels for the vehicle summary, client profile, and rental specifics—including the total calculated cost and duration. The layout features a primary "Confirm Booking" action to commit the transaction and a secondary "Cancel" option to abort the process.

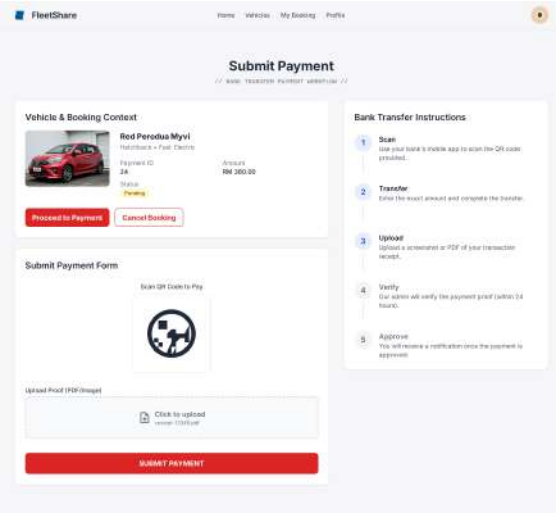


Figure 5.10: Submit Payment: Gateway selection and payment processing screen.

The payment interface (Figure 5.10) guides the user through the bank transfer workflow. It displays the booking context and payable amount alongside a generated QR code for immediate transaction. The screen includes a step-by-step instruction panel and a file upload mechanism for submitting the transaction receipt as proof of payment.

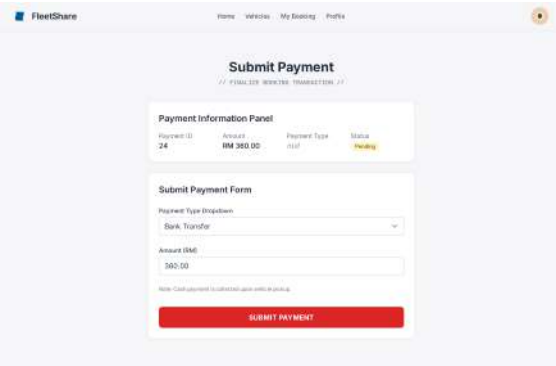


Figure 5.11: Card Payment: Detailed input form for credit/debit card transactions.

The transaction finalization interface (Figure 5.11) presents a summary of the booking payment status alongside a submission form. It features a dropdown menu for selecting the payment method, a field to verify the total amount, and a primary button to execute the payment process.

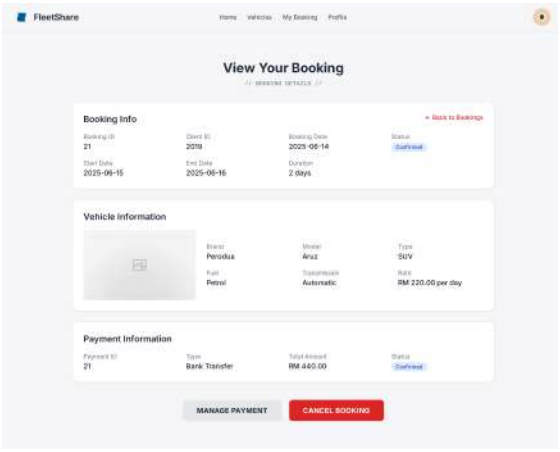


Figure 5.12: View Booking: Status overview of current and past rental reservations.

The booking details interface (Figure 5.12) consolidates reservation data into three distinct panels: booking status and timeline, vehicle specifications, and payment transaction records. It includes action buttons at the footer, allowing users to manage payment methods or initiate a cancellation request.

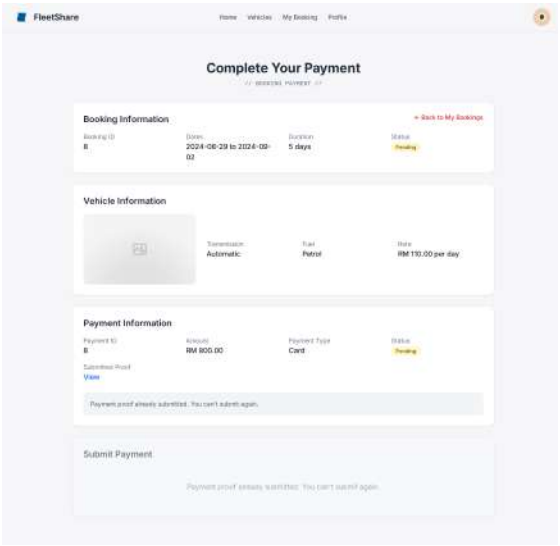


Figure 5.13: View Payment: Transaction history and receipt generation.

The payment review interface (Figure 5.13) tracks the financial status of a reservation, categorizing data into booking context, vehicle details, and transaction specifics. It implements a read-only state for submitted payments, displaying the current verification status while disabling the submission form to prevent duplicate

entries.

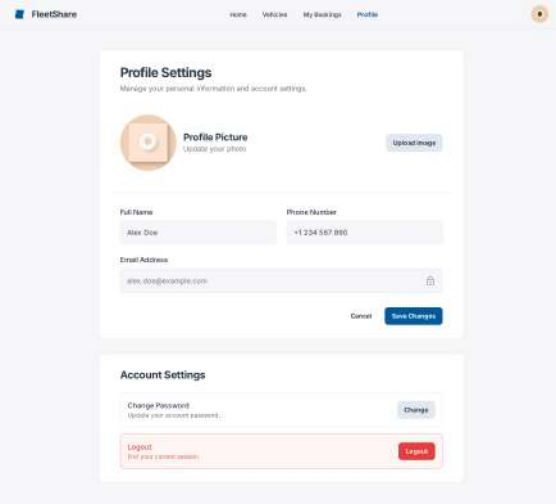


Figure 5.14: Profile Management: Settings for updating personal details and preferences.

The profile management interface (Figure 5.14) enables users to update personal information and account security settings. It features an editable form for contact details—with a read-only field for the registered email to ensure identity integrity alongside specific controls for profile picture updates, password modification, and secure session termination.

Admin Module

The Admin module provides comprehensive back-office tools for managing users, fleet inventory, bookings, maintenance, and system reporting.

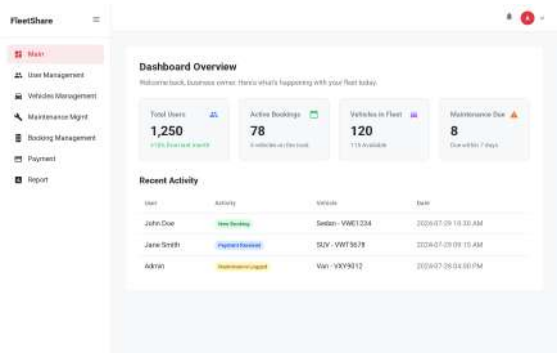


Figure 5.15: Admin Dashboard: High-level overview of system metrics and alerts.

The administrative dashboard (Figure 5.15) serves as the central command hub, offering a snapshot of fleet operations through high-level metrics. It displays statistics such as total users, active bookings, and maintenance alerts in a card layout, accompanied by a recent activity feed to monitor real-time system events.

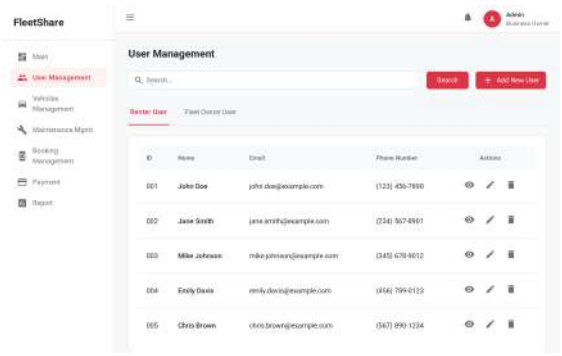


Figure 5.16: User Management: List view for monitoring and managing platform users.

The user management interface (Figure 5.16) enables administrators to oversee platform accounts through a centralized dashboard. It features a tabbed layout to distinguish between "Renter" and "Fleet Owner" profiles, accompanied by search functionality for rapid record retrieval. The data grid displays essential contact details and provides direct access to administrative actions, including viewing, editing, or deleting user records.

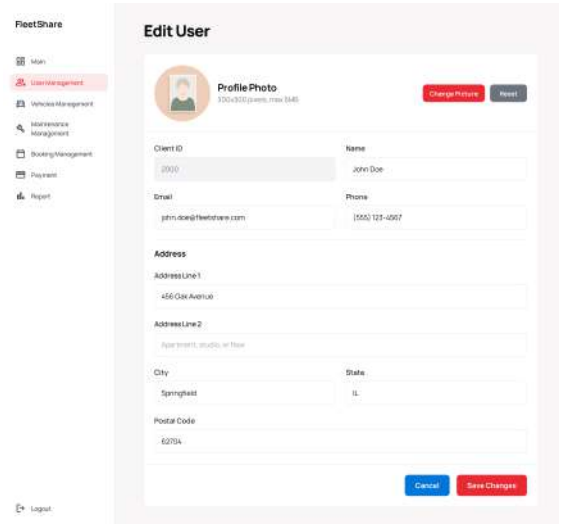


Figure 5.17: Edit User: Interface for modifying user roles and account details.

The user editing interface (Figure 5.17) permits administrators to update account-specific information. It features a profile photo management section, followed by editable fields for personal contact details and physical address. A control bar at the bottom allows for committing updates via "Save Changes" or reverting actions via "Cancel".

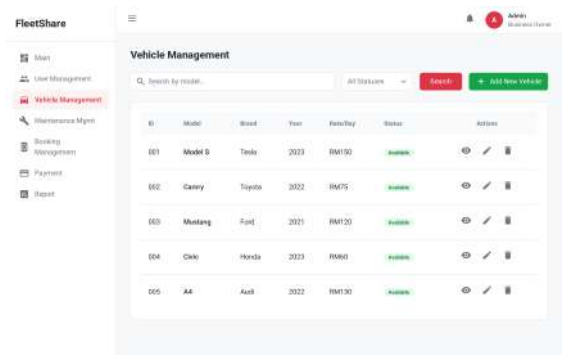


Figure 5.18: Vehicle Management: Central registry of all fleet vehicles.

The vehicle management dashboard (Figure 5.18) facilitates the administration of the fleet inventory. It features a search bar and status filter for efficient asset retrieval, alongside a prominent control to register new vehicles. The central data grid lists key specifications for each unit, providing quick access to view, edit, or remove records.

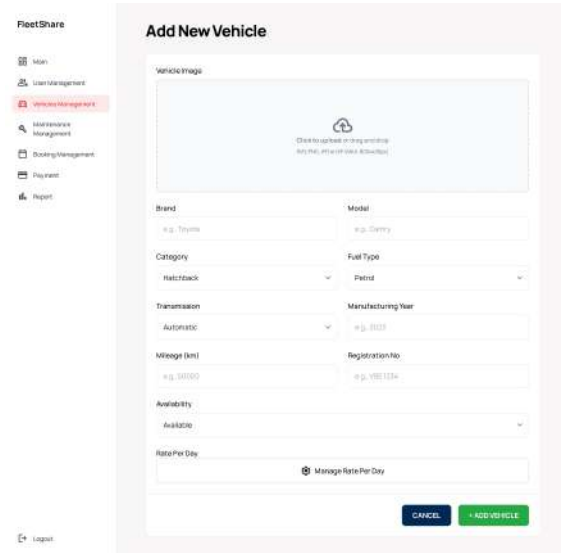


Figure 5.19: Add Vehicle: Form for onboarding new vehicles to the fleet.

The vehicle registration interface (Figure 5.19) facilitates the onboarding of new fleet assets. It features a prominent media upload section for vehicle imagery, followed by a comprehensive form for capturing technical specifications such as brand, model, and fuel type. The layout includes dedicated controls for defining rental rates and availability status prior to final submission.

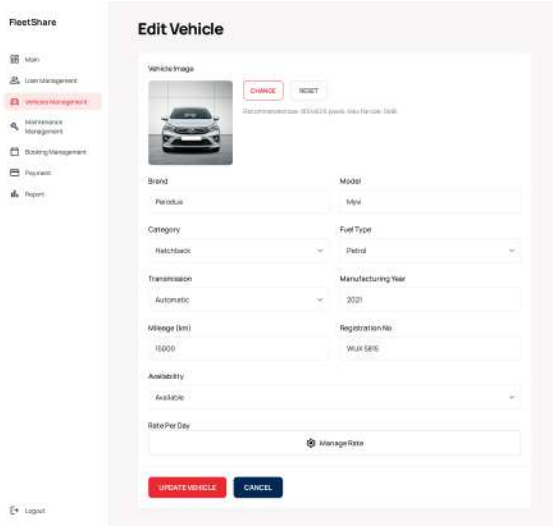


Figure 5.20: Edit Vehicle: Interface for updating vehicle specs and status.

The vehicle modification interface (Figure 5.20) permits administrators to update the specifications and operational status of existing fleet assets. It features controls for managing the vehicle image, editable fields for technical attributes such as mileage and registration data and a dedicated section for adjusting rental rates before committing changes.

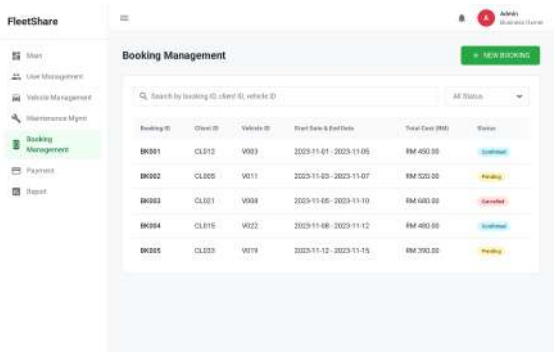


Figure 5.21: Booking Management: Control panel for overseeing all rental reservations.

The booking management dashboard (Figure 5.21) provides a centralized registry of all reservations. It features a multi-criteria search bar and status filter to streamline record retrieval. The data grid presents essential booking metrics including reservation dates, total cost, and color-coded status tags allowing administrators to monitor fleet utilization efficiently.

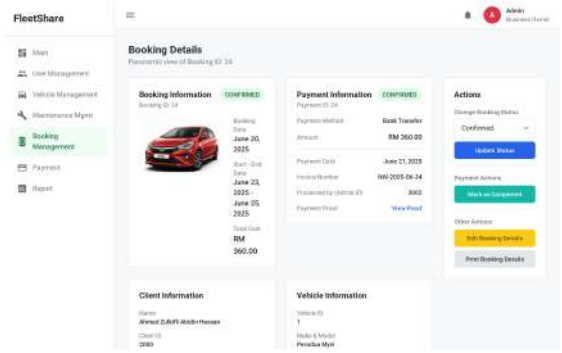


Figure 5.22: Booking Detail: Granular view of specific reservation information.

The booking detail interface (Figure 5.22) presents a consolidated record of a specific reservation, organizing data into segmented panels for booking logistics, payment status, client identity, and vehicle information. It includes a dedicated command sidebar that allows administrators to modify booking states, verify payment proofs, and generate documentation.

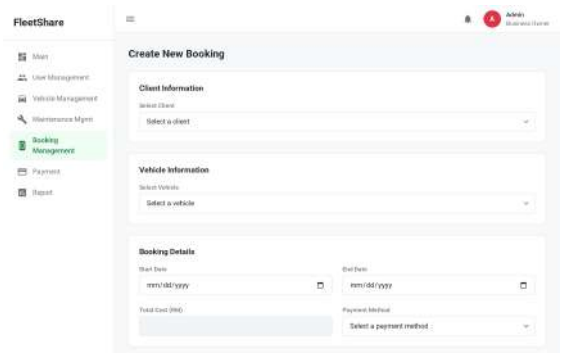


Figure 5.23: Create Booking: Admin-initiated manual booking interface.

The manual booking creation interface (Figure 5.23) allows administrators to generate reservations on behalf of clients. It is structured into three segments: client selection via dropdown, vehicle assignment, and scheduling controls using

date pickers. The form also includes fields for defining the payment method and viewing the calculated total cost.

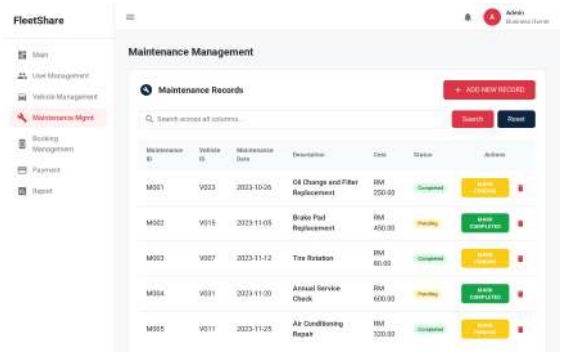


Figure 5.24: Maintenance Management: Tracking system for vehicle repairs and service.

The maintenance tracking interface (Figure 5.24) enables administrators to monitor the service history of the fleet. It features a search-enabled registry listing repair details, costs, and scheduling dates. The grid provides direct status controls, allowing operators to toggle records between "Pending" and "Completed" states or remove entries as needed.

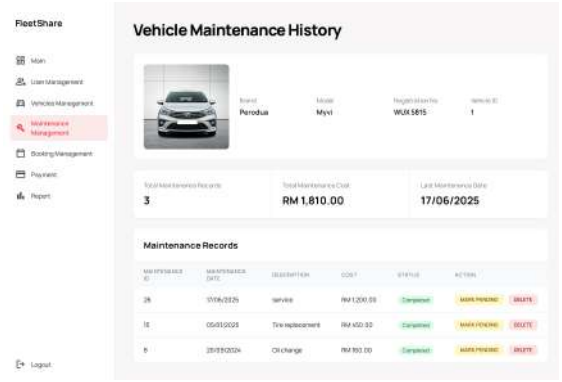


Figure 5.25: View Maintenance: Detailed logs of maintenance history.

The maintenance history interface (Figure 5.25) provides a comprehensive log of service activities for a specific vehicle. It features a summary dashboard highlighting total costs and recent activity dates, followed by a detailed data grid that allows administrators to track repair specifics and manage the status of individual records.

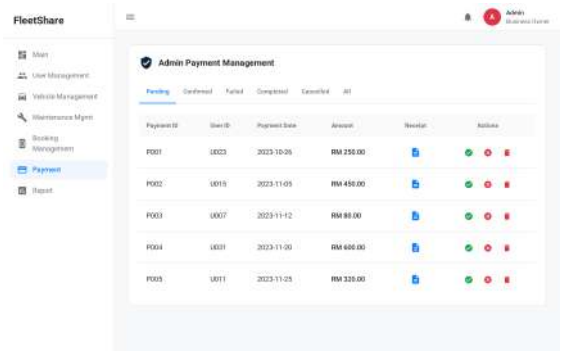


Figure 5.26: Payment Management: Oversight of all financial transactions.

The payment administration interface (Figure 5.26) facilitates the oversight of financial transactions through a tabbed categorization system. It lists payment records with access to proof-of-payment documents and provides direct controls for verifying, rejecting, or deleting transactions based on their current status.

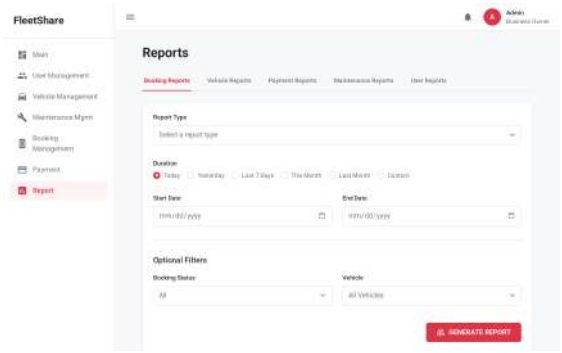


Figure 5.27: Generate Report: Tools for creating custom system reports.

The report generation interface (Figure 5.27) enables administrators to extract system analytics across various categories, such as bookings and maintenance. It provides configuration controls for selecting report types, defining temporal scopes via preset or custom date ranges, and applying specific filters before generating the final output.

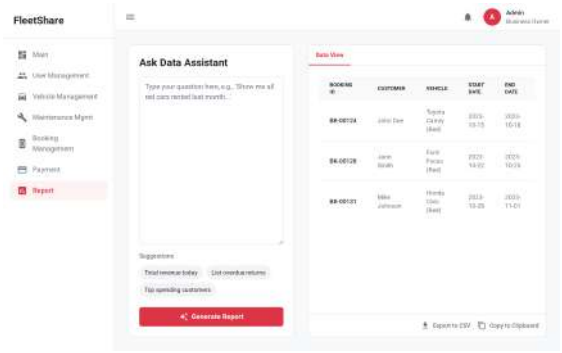


Figure 5.28: Data Assistant: Tool for quick data retrieval and system queries.

The data assistant interface (Figure 5.28) leverages natural language processing to facilitate dynamic system queries. It features a text input field for custom questions, supplemented by quick-access suggestion chips for common metrics. Query results are instantly rendered in a tabular "Data View" panel, which includes utilities for exporting the dataset to CSV or the clipboard.

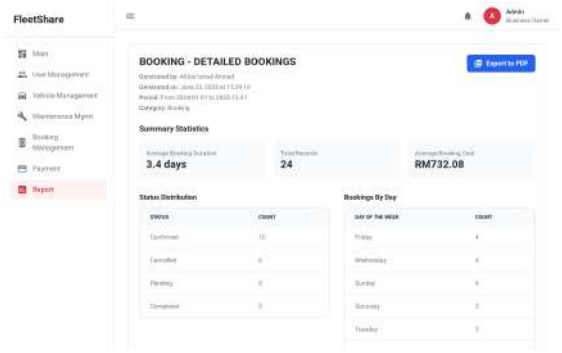


Figure 5.29: Generated Report: Preview of finalized system analytics documents.

The generated report interface (Figure 5.29) displays the finalized analytical output based on user-defined parameters. It features a header with metadata and PDF export options, followed by summary cards for key metrics such as average booking duration and total records. The data is further segmented into detailed distribution tables, organizing metrics by status and temporal frequency.

5.5.3 Screen Objects and Actions

The FleetShare user interface comprises distinct screen objects designed to facilitate specific user tasks. To ensure a seamless user experience, these objects incorporate immediate feedback mechanisms and validation logic. The following subsections detail the key interactive elements categorized by module.

General User Objects

These elements control system access and account creation, available to all visitors. The application enforces standard web layout conventions to streamline user onboarding paths (Krug, 2014).

Global Navigation Bar

A persistent top-level element containing links to Home, Dashboard, and Profile.

- **Action:** Clicking navigates to the respective module.
- **Behavior:** Dynamic visibility based on login status (Garrett, 2010).

Authentication Forms (Login)

Input fields for Email and Password with a primary submission button.

- **Action:** Validates credentials against the database.
- **Feedback:** Displays inline error messages for failed attempts or redirects to the Dashboard upon success.

Registration Inputs

A multi-field form including a Role Selection dropdown (Renter vs. Fleet Owner).

- **Action:** Real-time validation checks email format and password strength.

- **Feedback:** The submit button remains disabled until all required fields satisfy validation criteria (Bargas-Avila et al., 2010).

Client Interaction Objects

These objects manage the vehicle discovery, booking, and payment workflows specific to Renters.

Search & Filter Controls

A combination of a keyword search bar, category dropdowns, and price range sliders.

- **Action:** Modifying a filter triggers a request to refresh the vehicle grid.
- **Behavior:** Results update dynamically without a full page reload (Hearst, 2009).

Vehicle Card & Booking Button

Summary cards displaying vehicle thumbnail, rate, and transmission type.

- **Action:** Clicking "Book Now" transitions the user to the specific Vehicle Summary and Booking Form.

Interactive Date Picker

Calendar controls for selecting Pickup and Return dates.

- **Action:** Selecting a date range triggers a calculation script.
- **Feedback:** The **Total Price** field updates instantly based on the daily rate $\times$ duration (Norman, 2013).

Payment Upload Mechanism

A file input field specifically for bank transfer receipts.

- **Action:** Opens the native OS file dialog to select an image or PDF.

- **Feedback:** Displays a preview of the uploaded filename or thumbnail prior to final submission.

Admin Management Objects

These objects provide high-level control over system data, users, and reporting.

Dashboard Metric Cards

Summary widgets displaying real-time counts.

- **Action:** Clicking a card acts as a shortcut, redirecting the admin to the detailed management view for that specific metric (Few, 2013).

Management Data Grids

Tabular views used in User, Vehicle, and Booking management screens.

- **Action:** columns serve as sort controls; rows contain specific "Edit" or "Delete" action buttons.
- **Behavior:** Pagination controls at the footer load subsequent datasets.

Maintenance Status Toggles

Interactive status indicators (e.g., Pending/Completed) within the Maintenance log.

- **Action:** Toggling updates the record status in the database.
- **Feedback:** Visual color change (e.g., Yellow to Green) confirms the state change.

Data Assistant Query Input

A natural language text field for generating custom reports.

- **Action:** Pressing "Enter" processes the query via NLP.
- **Feedback:** Returns a generated data view or specific answer within the chat interface (Katsogiannis-Meimarakis and Koutrika, 2023).

Report Export Controls

Buttons for downloading analytics (PDF/CSV).

- **Action:** Generates a file based on current filters.
- **Feedback:** A spinner indicates processing time before the browser initiates the file download.

5.6 Discussion

The technical blueprint developed for the FleetShare platform represents a strategic synthesis of modern architectural, database, and user interface paradigms engineered specifically to address the operational bottlenecks faced by small and medium-sized enterprises (SMEs) and independent vehicle operators. To appreciate the validity of the chosen design strategies, it is essential to critically evaluate the architectural trade-offs, multi-tenant security controls, and relational integrity structures implemented across the platform.

5.6.1 Architectural Decoupling and N-Tier Maintainability

The adoption of a Layered N-Tier Architectural Pattern driven by the Java Spring Boot framework fundamentally resolves the structural challenges of system scalability and software maintainability (Walls, 2022). Traditional monolithic web applications frequently suffer from high coupling, where business rules are directly embedded within user interfaces or database queries, causing severe code regression during updates (Fowler, 2002).

By decomposing the FleetShare codebase into explicit, highly cohesive packages (`controller`, `service`, `repository`, `entity`), an impermeable separation of concerns is established. The presentation layer is entirely insulated from data persistence mechanics through the Data Transfer Object (`dto`) layer, which filters out internal schema mutations and prevents sensitive entity data leakage across

public interfaces. This granular decoupling is critical for the platform's long-term lifecycle; should future requirements necessitate transitioning from server-side Thymeleaf templates to a standalone mobile application client (e.g., Android via Jetpack Compose), the entire core business logic residing within the `service` package remains completely untouched and seamlessly reusable via newly exposed RESTful API endpoints.

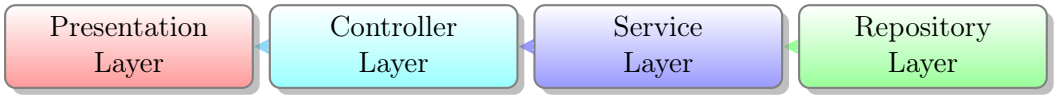


Figure 5.30: Linear Data Flow Demonstrating N-Tier Decoupling

Architecture Principle: DTO Isolation

The Data Transfer Object (DTO) layer serves as a structural firewall. By mapping internal database entities (e.g., `Vehicle.java`) to lightweight external structures (e.g., `VehicleDTO.java`) before transmission to the Presentation layer, the system strictly enforces the **Separation of Concerns**. This prevents sensitive database metadata from leaking to the frontend and shields the presentation logic from breaking if the underlying database schema is modified.

5.6.2 Multi-Tenancy Trade-offs: Shared-Schema vs. Isolation Costs

A pivotal architectural decision within FleetShare is the selection of a logical Shared-Schema Multi-Tenant Model over a physically isolated database-per-tenant framework (Krebs et al., 2012). While a database-per-tenant architecture offers complete data security out of the box, it incurs massive operational and financial overhead due to the compounding costs of cloud hosting, redundant resource utilization, and complex cross-database schema migration workflows. For the targeted demographic of independent owners and small commercial branches, these high infrastructural costs represent an immediate barrier to digital adoption.

The shared-schema paradigm selected for FleetShare completely eliminates these financial bottlenecks by pooling computational resources within a single MySQL database instance. However, this optimization shifts the critical responsibility of multi-tenant security entirely onto the application layer (Bezemer and Zaidman, 2010). To mitigate the severe risk of cross-tenant data leakage or competitor corporate espionage, the platform implements strict logical data isolation enforced within the business layer. Utilizing Spring Data JPA, every database operation handles context-aware entity parameters tied directly to the authenticated user's `fleetOwnerId`. By intercepting queries at the `service` layer and restricting data retrieval based on verified user context, the architecture guarantees a secure, isolated tenant workspace while preserving the low-cost operational economics necessary for SME sustainability (Kabbedijk et al., 2012).

Architectural Decision: Logical Data Isolation

In a Shared-Schema model, physical database separation does not exist. Therefore, security is entirely reliant on the Application layer. FleetShare achieves this by implicitly injecting the authenticated session's `fleetOwnerId` into every JPA repository method. A generic query to fetch vehicles is automatically constrained: `SELECT * FROM vehicles WHERE fleet_owner_id = ?`, completely preventing cross-tenant data access.

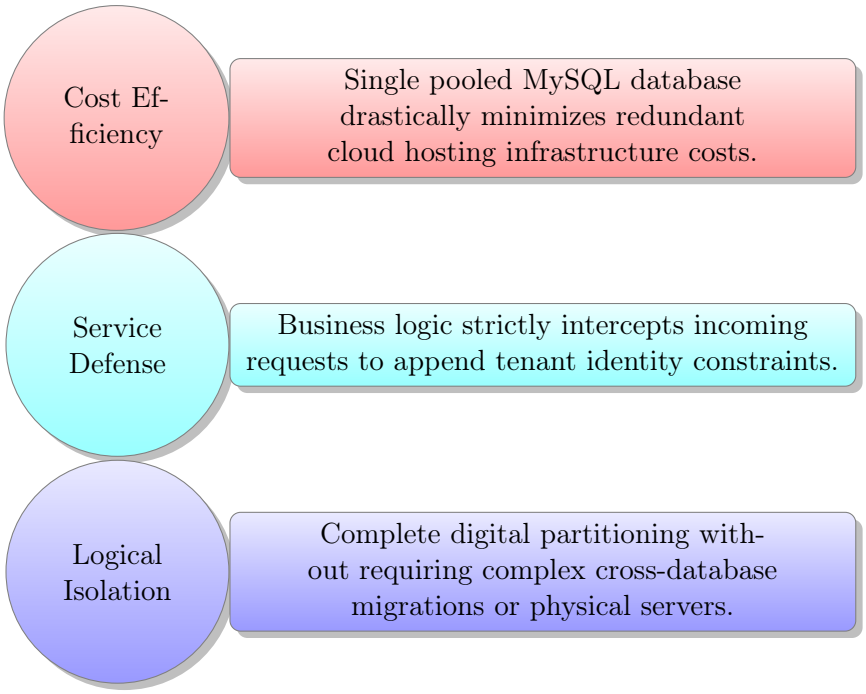


Figure 5.31: Shared-Schema Multi-Tenancy Implementation Strategy

5.6.3 Relational Integrity and the Temporal Data Dilemma

While classical relational design models mandate strict adherence to the Third Normal Form (3NF) to eliminate structural redundancies and update anomalies (Codd, 1970), empirical software engineering challenges often emerge when pure normalization meets mutable business realities (Date, 2004). FleetShare inherently exposes and resolves this paradigm conflict within its transactional booking and financial schemas.

In a purely normalized relational schema, an invoice would compute its total cost dynamically by querying the current daily rate of a vehicle via standard foreign key joins. However, this introduces a catastrophic business anomaly: if a fleet owner updates a vehicle's daily rental fee to accommodate peak seasonal demand, all past, completed financial invoices tied to that vehicle would retroactively recalculate, destroying financial record integrity.

FleetShare programmatically resolves this architectural conflict through the design of the `BookingPriceSnapshot` table. By immutably capturing and lock-

ing the precise temporal variables (`rate_per_day` and `days_rented`) at the exact timestamp of transaction finalization, the system effectively shields historical accounting records from subsequent inventory state updates.

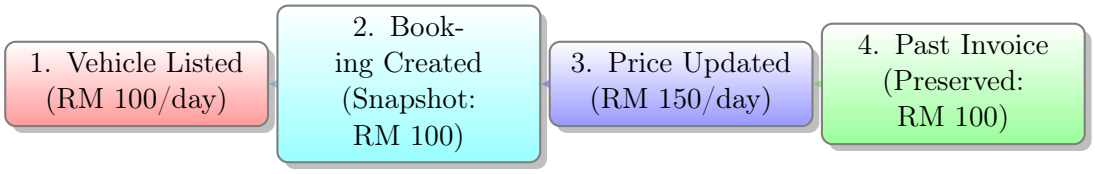


Figure 5.32: Temporal Isolation Flow demonstrating Financial Integrity

Database Design: Append-Only Audit Trails

To maintain strict regulatory compliance, FleetShare never overwrites the status of a booking or payment. Instead, the system inserts new records into append-only log tables (`BookingStatusLog`, `PaymentStatusLog`). This guarantees a complete, traceable timeline of who modified the state and when.

Furthermore, the implementation of append-only audit trails (`BookingStatusLog`, `PaymentStatusLog`, and `VehicleMaintenanceLog`) ensures completely traceable state transitions without perpetually destroying historical data via string mutation operations, providing the rigorous level of regulatory auditability required by modern commercial operations (Elmasri and Navathe, 2016).

5.6.4 Cognitive Load Optimization in Role-Based Interface Design

Finally, the front-end layout strategies map these underlying complex data flows into clean, minimalistic interfaces designed to actively minimize user cognitive load (Shneiderman et al., 2016). SME operators often face administrative fatigue from navigating cluttered enterprise tools (Few, 2013). FleetShare resolves this by leveraging dynamic, conditional menu fragments via Thymeleaf decoration. The system strategically strips away extraneous input controls, presenting users exclusively with action objects relevant to their immediate operational goals (e.g.,

streamlined ToyyibPay checkout workflows for renters, asset tracking dashboards for owners).

UX Principle: Progressive Disclosure

By utilizing Thymeleaf template decoration, FleetShare implements **Progressive Disclosure**. Instead of overwhelming the user with a monolithic enterprise menu, the system evaluates the authenticated user's role and conditionally renders only the navigation links and action buttons immediately relevant to their workflow.

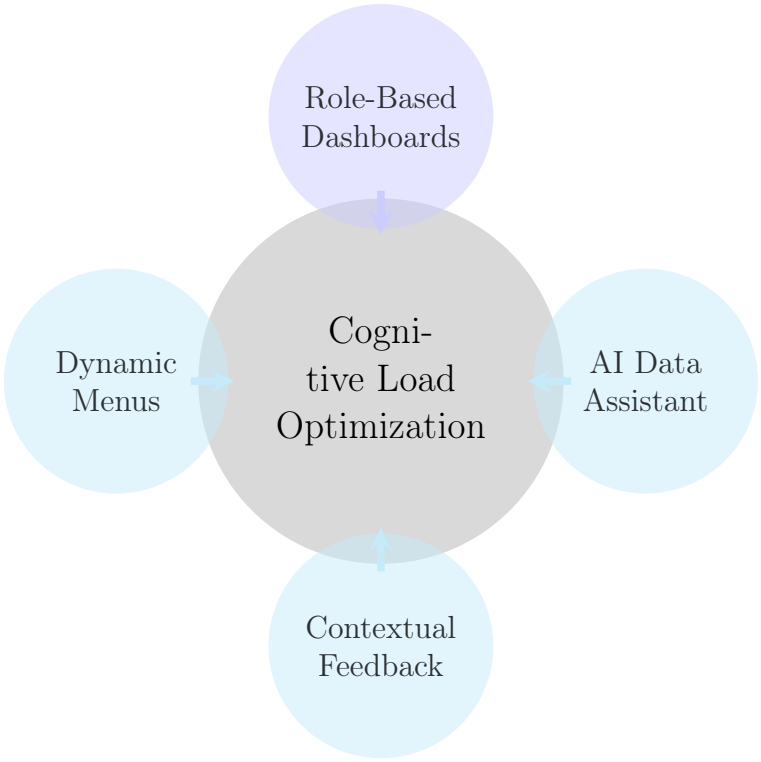


Figure 5.33: UI/UX Strategies for Reducing Cognitive Friction

Coupled with immediate UI feedback loops like contextual toasts and live KPI chart updates, the system design ensures high usability and rapid transaction execution speeds. Furthermore, the integration of advanced tools like the natural language AI Data Assistant empowers administrators to extract complex analytics without learning intricate reporting interfaces. Collectively, these UX methodolo-

gies successfully bridge the gap between advanced multi-tenant backend complexity and a frictionless, intuitive consumer application experience (Norman, 2013).

5.7 Summary

In summary, this chapter has presented the comprehensive, granular technical design of the FleetShare multi-tenant vehicle rental management platform. By establishing a robust, Layered N-Tier Architectural Pattern driven by the Java Spring Boot framework, the system achieves a clean separation of concerns and strict subsystem modularity. The shared-schema relational database model systematically normalized to the Third Normal Form (3NF) and fortified with immutable temporal price snapshot mechanisms ensures optimal resource pooling while maintaining secure data partitioning for independent fleet operators. Furthermore, the role-centric UI/UX design paradigms, leveraging Bootstrap 5 and Thymeleaf rendering, guarantee that these advanced backend mechanics (including AI-driven analytics and secure payment gateways) are translated into frictionless, responsive presentation layouts. Collectively, the explicit architectural package mappings, normalized data dictionaries, and user interaction architectures compiled in this chapter provide a definitive, executable blueprint. This structural roadmap directly governs the physical implementation, system deployment, and rigorous testing evaluation phases presented in the subsequent chapters of this thesis.

CHAPTER 6

SYSTEM IMPLEMENTATION

6.1 Introduction

This chapter discusses the system implementation phase of the FleetShare platform, representing the realization of the architectural blueprints and database schemas defined in Chapter 5 into a functional, executable software system. The implementation phase bridges the critical gap between conceptual design and physical code, detailing the configuration of the development environment, the translation of the Model-View-Controller (MVC) architecture into Java Spring Boot components, and the structural realization of the multi-tenant database using Spring Data JPA. Furthermore, this chapter outlines the programmatic execution of the system's hierarchical navigation and highlights how specific technical complexities—such as resolving temporal data anomalies and integrating third-party asynchronous services—were successfully engineered.

6.2 System Hierarchical Menu and Navigation Architecture

The system hierarchical menu dictates the structural taxonomy and navigational flow of the FleetShare web application (Rosenfeld et al., 2015). Because FleetShare operates on a strict multi-tenant, Role-Based Access Control (RBAC) architecture managed by Spring Security, the hierarchical menu is dynamically rendered and deeply contextualized based on the authenticated user's session role (Platform

Admin, Fleet Owner, or Renter) (Sandhu et al., 1996). This dynamic rendering explicitly ensures robust authorization while minimizing cognitive overload by isolating unrelated application modules (Shneiderman et al., 2016). Furthermore, restricting interface visibility based on authenticated privileges serves as a fundamental defense-in-depth mechanism, actively preventing users from enumerating restricted system capabilities (Stuttard and Pinto, 2011).

- **Renter (Client) Hierarchy:** The primary entry level acts as a consumer-facing interface (Sundararajan, 2016). Upon successful authentication, the top-level navigation exposes the *Home Dashboard*, the *Vehicle Discovery Catalog* (for browsing and dynamically filtering assets), *My Bookings* (tracking active, upcoming, and historical reservations), *Payment Management* (handling outstanding invoices and uploading payment receipts), and a *User Profile* configuration module.

- **Fleet Owner Hierarchy:** Engineered specifically for SME operators, the owner navigation serves as a comprehensive, B2B back-office portal (Yigitbasioglu and Velcu, 2012). The primary hierarchical nodes include an overarching *Overview Dashboard*, *Fleet Management* (encapsulating Vehicle Inventory and Maintenance Logs), *Booking Operations* (handling reservation approvals and a localized Customer CRM), *Financials* (tracking specific Invoices, Payments, and Payout Dashboards), *AI Analytics & Reports*, and finally, the *Business Profile* configuration.

- **Platform Admin Hierarchy:** The administrative hierarchy is strictly provisioned for global oversight and platform governance (Tiwana, 2013). Its top-level taxonomy includes a *Global Overview Dashboard*, *User Management* (governing the suspension and verification of all tenants), comprehensive *System Oversight* (global views of all Vehicles, Bookings, and Maintenance logs for dispute resolution), *Global Financials* (tracking platform commission and

initiating Owner payouts), and *AI-Assisted System Analytics*.

Figure 6.1 illustrates the complete navigational sitemap and branching logic of the FleetShare platform, demonstrating how the monolithic web interface is securely partitioned.

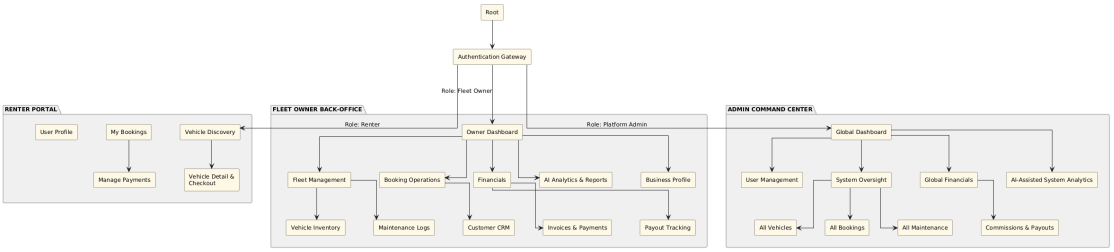


Figure 6.1: Hierarchical Menu and Navigation Sitemap of the FleetShare Platform

6.3 System Development

The core development of the FleetShare platform was executed systematically, adhering strictly to the Layered N-Tier Architectural Pattern (Richards, 2015). To realize this architecture within a multi-tenant environment, the development phase leveraged a cohesive technology stack anchored by the Java Spring Boot framework, MySQL, and Thymeleaf. The following subsections detail the specific implementation strategies applied to the backend persistence layer, the frontend presentation layer, and the complex external service integrations. Employing such structured decoupling across these technologies fundamentally enhances the long-term maintainability, security, and evolutionary capabilities of the software system (Sommerville, 2015).

6.3.1 Backend Application Engineering (Spring Boot)

The backend engine was architected using Java 17 and the Spring Boot 3 framework, providing a robust, standalone application container governed by an Inversion of Control (IoC) architecture (Walls, 2022). Through Spring's core Depen-

dency Injection mechanism, the lifecycle and wiring of application components such as database repositories, email workers, and payment clients are managed automatically, drastically reducing tight coupling across the codebase.

The system's core business logic is encapsulated entirely within the `@Service` layer. This layer acts as the authoritative boundary for executing algorithmic operations, applying business rules, and managing database transactions via the `@Transactional` annotation, which ensures atomic rollbacks in the event of partial execution failures.

To protect the integrity of this business layer, the presentation layer is rigorously insulated from data persistence mechanics through the Data Transfer Object (DTO) interception pattern (Fowler, 2002). Spring `@RestController` and `@Controller` classes interact exclusively with DTOs. Before any payload is accepted, incoming HTTP requests undergo strict constraint validation (e.g., `@Valid`, `@NotNull`) via the Jakarta Validation API. Conversely, outgoing responses are sanitized; sensitive JPA proxy metadata and Bcrypt-hashed passwords are mathematically stripped from the payload before serialization, guaranteeing strict internal data security (Provos and Mazières, 1999).

Security Warning: Mass Assignment Vulnerabilities

Failure to implement the Data Transfer Object (DTO) pattern exposes the application to **Mass Assignment** or **Over-Posting** attacks. If raw JPA entities (e.g., `User.java`) are directly bound to incoming HTTP requests, malicious actors could inject unauthorized fields into the JSON payload (e.g., `"role": "ADMIN"`). By utilizing strictly defined DTOs, the framework implicitly drops any parameters not explicitly mapped, serving as a structural firewall against payload injection.

Finally, backend stability is fortified through a centralized `@ControllerAdvice` global exception handler. Rather than allowing the application to crash or leak

raw stack traces to the user, this interceptor catches runtime anomalies such as database constraint violations or missing resources and gracefully maps them into standardized error views or structured HTTP response codes, ensuring a resilient and professional user experience.

Listing 6.1: Example of centralized error interception using @ControllerAdvice

```
@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(
            ResourceNotFoundException ex) {
        ErrorResponse error = new ErrorResponse(
                404, ex.getMessage());
        return new ResponseEntity<>(
                error, HttpStatus.NOT_FOUND);
    }

    // Prevents database constraint errors from leaking stack traces
    @ExceptionHandler(DataIntegrityViolationException.class)
    public ResponseEntity<ErrorResponse> handleDatabaseError(
            Exception ex) {
        ErrorResponse error = new ErrorResponse(
                409, "Conflict: Record already exists.");
        return new ResponseEntity<>(
                error, HttpStatus.CONFLICT);
    }
}
```

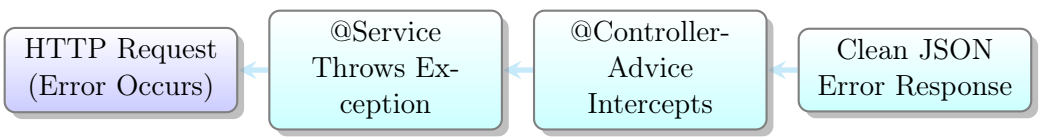


Figure 6.2: Execution Flow of the Global Exception Handler

6.3.2 Security and Persistence Realization (Spring Data JPA)

To materialize the logical database design, Spring Data JPA and Hibernate were utilized as the primary Object-Relational Mapping (ORM) layer connecting to a MySQL 8.0 instance (Bauer et al., 2015). Java Entity classes were mapped directly to MySQL tables using explicit relational constraints (e.g., `@Entity`, `@OneToMany`, and `@ManyToOne`). To enforce database normalization and referential integrity at the code level, foreign keys were strictly defined utilizing the `@JoinColumn` annotation. Furthermore, automated auditing mechanisms were implemented using JPA timestamp annotations to consistently track record lifecycles (e.g., `created_at`) without manual intervention.

Crucially, the multi-tenant data isolation was enforced programmatically within the `Service` layer rather than relying on manual SQL string concatenations, which are highly susceptible to SQL injection (Halfond et al., 2006). Every repository query triggered by a Fleet Owner inherently utilizes Spring Data JPA's parameterized methods to append their specific `fleetOwnerId` (e.g., `findByFleetOwnerId`) (Krebs et al., 2012). To optimize database performance and prevent the common N+1 query performance bottleneck, entity relationships were configured with lazy loading fetch strategies (`FetchType.LAZY`). This ensures that associated datasets are only retrieved into memory when explicitly requested by the business logic.

Figure 6.3 illustrates the physical relational schema realized in the database, highlighting the complex interconnectivity between Users, Vehicles, and the critical `BookingPriceSnapshot` entity.

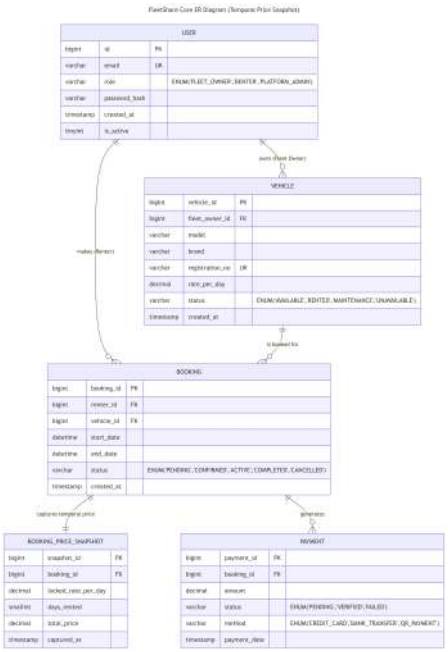


Figure 6.3: Core Physical Entity-Relationship Diagram (ERD) Realized via Hibernate ORM

6.3.3 Frontend Implementation (Thymeleaf & Bootstrap)

The Presentation Layer was engineered utilizing Thymeleaf, a server-side Java template engine seamlessly integrated with the Bootstrap 5 CSS framework and the Spring Security dialect (`thymeleaf-extras-springsecurity6`) (Walls, 2022). Rather than duplicating raw HTML structures across the platform's numerous interfaces, the Thymeleaf Layout Dialect (`layout:decorate`) was implemented to create modular, highly reusable UI fragments (e.g., standardizing the global navigation headers and sidebars) (Kappel et al., 2006).

To successfully bridge the backend business logic with the frontend interface, dynamic data binding was orchestrated using Thymeleaf's structural attributes (e.g., `th:each`, `th:object`, and `th:if`). This architecture allows Spring `@Controller` classes to inject populated Java Data Transfer Objects directly into the HTML Document Object Model (DOM) prior to server dispatch.

Listing 6.2: Dynamic DOM Generation utilizing Thymeleaf Structural Attributes

```
<!-- Iterates through a List<VehicleDTO> injected by the Spring Controller -->
<tr th:each="vehicle : ${vehicleList}">
    <td th:text="${vehicle.licensePlate}">Plate Number</td>
    <td th:text="${vehicle.dailyRate}">RM 0.00</td>

    <!-- Conditional rendering based on Vehicle Status -->
    <td>
        <span th:if="${vehicle.status == 'AVAILABLE'}"
              class="badge bg-success">Active</span>
        <span th:if="${vehicle.status == 'MAINTENANCE'}"
              class="badge bg-warning">Repair</span>
    </td>
</tr>
```

To further optimize the user experience and reduce unnecessary server round-trips, the frontend heavily utilizes Vanilla JavaScript for localized client-side interactivity. Features such as real-time search filtering, localized mathematical formatting, and asynchronous view toggling (e.g., seamlessly switching between visual data grids and tabular layouts) execute instantaneously on the client browser. Ultimately, these views are rendered server-side as highly responsive, secure HTML5 grids, ensuring rapid initial page loads, optimal SEO discoverability, and fluid presentation across both mobile and desktop viewports.

Best Practice: Client-Side DOM Manipulation

By strategically offloading non-critical logic (such as dynamic table filtering or currency string formatting) to **Vanilla JavaScript** executing on the client's local browser, FleetShare drastically reduces unnecessary asynchronous HTTP requests. This prevents the Spring Boot server from executing repetitive, computationally expensive `SELECT` queries on the database for simple UI updates, significantly optimizing overall cloud resource consumption.

6.3.4 Complex Service Integrations

To elevate the platform beyond standard CRUD capabilities, several advanced software integrations were engineered. Because external APIs inherently introduce network latency and unpredictability, these services were designed with rigorous edge-case handling and fault tolerance.

ToyyibPay Payment Gateway

Implementation: Spring's `RestTemplate` handles asynchronous HTTP POST requests to the ToyyibPay API (Lauret, 2019). The integration dynamically routes between a Central Payout model (split payment commission) and a BYOK (Bring Your Own Key) fallback depending on the Fleet Owner's configuration.

Edge Case Mitigation: Webhook Spoofing

To prevent malicious actors from spoofing payment webhooks, the system implements a strict MD5 cryptographic hash validation algorithm (`validateCallbackHash`) against the incoming payload's secret key. This ensures transaction statuses are only updated by authorized ToyyibPay servers.

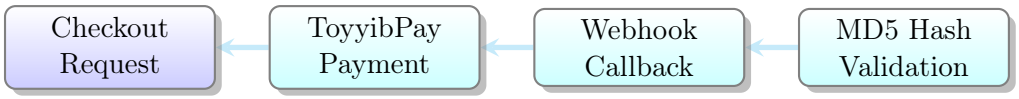


Figure 6.4: Asynchronous Payment Webhook Verification Flow

Asynchronous Email Notification Service

Implementation: Using `JavaMailSender` and `MimeMessageHelper`, the system parses dynamic Thymeleaf templates into rich HTML emails, appending dynamic PDF documents via `ByteArrayResource`.

Edge Case Mitigation: SMTP Latency & Failures

Because SMTP dispatching is inherently slow, these tasks are strictly offloaded to background threads via Spring's `@Async` annotation to prevent UI thread blocking (Goetz et al., 2006). Furthermore, robust `MessagingException` catch blocks ensure that if the SMTP server times out, the primary database transaction (e.g., confirming a booking) still succeeds without crashing the user's browser.

Listing 6.3: Fault-Tolerant Asynchronous Email Dispatch

```
@Async("mailTaskExecutor")
public void sendBookingConfirmation(String recipient, BookingDTO booking) {
    try {
        MimeMessage message = javaMailSender.createMimeMessage();
        MimeMessageHelper helper = new MimeMessageHelper(message, true);

        helper.setTo(recipient);
        helper.setSubject("Booking Confirmed: " + booking.getId());
        helper.setText(buildHtmlTemplate(booking), true);

        javaMailSender.send(message);
    } catch (MessagingException | MailException e) {
        log.error("SMTP Delivery Failed for Booking {}: {}",
```

```
                booking.getId(), e.getMessage());
            // Exception is caught; main DB transaction will NOT rollback
        }
    }
```

AI Data Analytics Engine

Implementation: This engine aggregates live relational data into an optimized text context, dispatching it to a Large Language Model (LLM) via Java's native `HttpClient`.

Edge Case Mitigation: LLM Throttling & Unpredictable Output

Because third-party LLMs are prone to traffic throttling, the `AiAssistantService` implements an automated exponential backoff and retry mechanism to gracefully handle HTTP 429 (Rate Limit Exceeded) and 504 (Gateway Timeout) errors. Additionally, robust regex sanitation algorithms were implemented to strip rogue Markdown code blocks (e.g., ```` ```json ````) from the LLM's unpredictable responses before parsing the payload via Jackson Databind (Katsogiannis-Meimarakis and Koutrika, 2023).

Listing 6.4: Regex Sanitization of Unpredictable LLM JSON Payloads

```
// Third-party LLMs frequently wrap JSON inside markdown ticks
String rawResponse = llmHttpClient.send(request, BodyHandlers.ofString());

// Strip rogue ```json and ``` from the payload before parsing
String sanitizedJson = rawResponse
        .replaceAll("^```json\\s*", "")
        .replaceAll("\\s*```$", "")
        .trim();
```

```
AnalyticsDTO data = objectMapper.readValue(sanitizedJson, AnalyticsDTO.class);
```

Dynamic Document Generation

Implementation: Utilizing OpenHTMLtoPDF and Apache POI Java libraries, the system dynamically injects live transactional Data Transfer Objects (DTOs) into backend HTML templates, rendering them physically into downloadable PDF invoices and Excel spreadsheets.

Edge Case Mitigation: PDF Rendering & Page Breaks

Automating the generation of these operational documents significantly reduces administrative overhead and mitigates human error in financial reporting (Romney and Steinbart, 2017). To prevent UI rendering bugs during PDF compilation, strict CSS print-media queries and page-break parameters were implemented. This ensures dynamic tabular data cascades cleanly across multiple pages without structural overflow.

Listing 6.5: CSS Print-Media Queries for PDF Page-Break Enforcement

```
<style>
  @media print {
      /* Prevent table rows from splitting across multiple PDF pages */
      table tr {
          page-break-inside: avoid;
      }

      /* Force a page break before the Terms and Conditions section */
      .terms-section {
          page-break-before: always;
      }
  }
</style>
```

Table 6.1 summarizes the specific fault tolerance strategies engineered to insulate the core application from these external integration risks. Furthermore, Figure 6.5 provides a comprehensive visual synthesis of the complete, high-level N-Tier architecture, illustrating exactly how these asynchronous services and third-party APIs interact seamlessly with the primary Spring Boot engine.

Table 6.1: Fault Tolerance Strategies per Integration

Integration	**Risk Addressed**	**Mitigation Technique**
ToyyibPay	Webhook spoofing	MD5 hash validation, secret key check
Email Service	SMTP latency / failure	`@Async` offloading, exception isolation
AI Analytics	API throttling (429/504)	Exponential backoff, regex sanitization
Document Generation	Page overflow / broken layout	CSS print-media queries, page-break rules

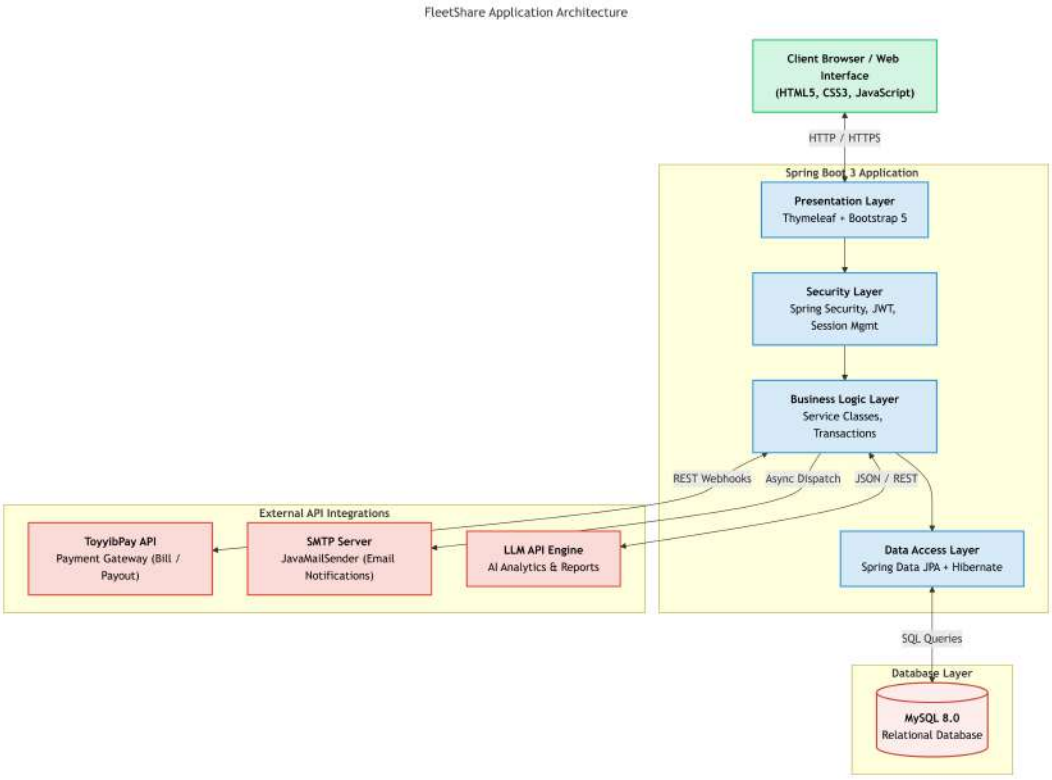


Figure 6.5: High-Level N-Tier System Architecture and External Integrations

6.4 Discussion

The physical implementation phase successfully validated the theoretical architectural models (Taylor et al., 2009). However, translating conceptual paradigms into a production-ready environment surfaced specific operational challenges, particularly concerning data integrity and asynchronous workflow orchestration (Kleppmann, 2017).

6.4.1 Resolving Temporal Data Anomalies in Financial Records

One of the most critical challenges encountered was managing the temporal mutability of vehicle pricing within a normalized schema (Snodgrass, 1999). In a standard relational design, an invoice calculates its total by dynamically joining the `Vehicle` table to retrieve the `rate_per_day`. However, this introduces a severe data corruption risk: if a Fleet Owner updates a vehicle's daily rate to accommodate peak demand, all historically completed financial invoices linked via foreign keys would retroactively recalculate. This would effectively destroy the integrity of past financial records (Kimball and Ross, 2013).

As illustrated in Figure 6.6, this anomaly was successfully resolved by designing the `BookingPriceSnapshot` entity. Rather than relying on a dynamic join, the Spring Service layer was engineered to permanently capture, detach, and persist the specific `rate_per_day`, `days_rented`, and the final `total_calculated_cost` variables into an immutable snapshot table at the exact millisecond the checkout is finalized.

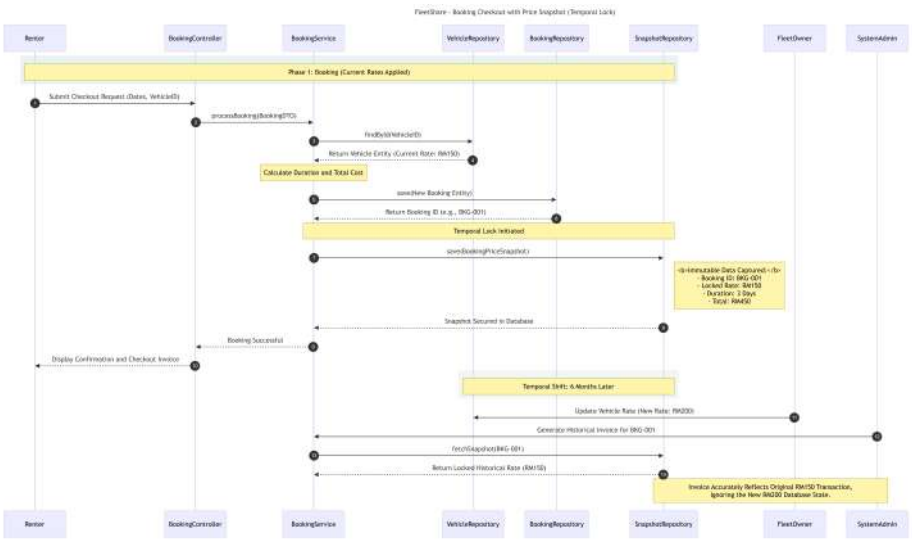


Figure 6.6: Sequence Diagram illustrating the Temporal Price Snapshot Mechanism

Furthermore, to accommodate real-world business edge cases such as manual price negotiations or promotional discounts, the snapshot table utilizes a `remarks` column to serialize these dynamic adjustments as immutable JSON payloads (e.g., `[{"description":"Owner Discount","amount":-10}]`). This effectively freezes the entire mathematical context of the transaction, decoupling historical accounting records from subsequent inventory state mutations and ensuring absolute regulatory financial compliance. Implementing explicit temporal patterns is recognized as a fundamental requirement for enterprise systems that must maintain strict auditability (Fowler, 2002).

6.4.2 Orchestrating Asynchronous External APIs

A final significant implementation hurdle involved integrating high-latency external third-party services, specifically ToyyibPay, the SMTP Email provider, and the Large Language Model (LLM) analytics engine, without compromising the application's primary execution thread (Hohpe and Woolf, 2004). Initial synchronous implementations caused severe UI blocking, where users experienced frozen browser screens while waiting for external servers to respond.

This architectural bottleneck was systematically resolved by leveraging Spring Boot's `@Async` capabilities and managed thread pooling (Walls, 2022). By offloading high-latency operations to background worker threads, the Presentation Layer could immediately return success views to the user, while the Business Layer negotiated with external APIs concurrently (Goetz et al., 2006).

Crucially, this asynchronous decoupling introduced essential fault-tolerance for unpredictable edge cases. Because the background threads execute outside the primary HTTP request lifecycle, localized network failures (e.g., an SMTP `MessagingException` timeout) are gracefully caught and logged without triggering a rollback of the core JPA database transaction.

Furthermore, to mitigate the volatility of third-party LLM endpoints, the background workers were programmed with explicit network timeout durations and exponential backoff algorithms. This ensures that transient HTTP 429 (Rate Limit Exceeded) or 504 (Gateway Timeout) errors automatically trigger a delayed retry, guaranteeing background service reliability while drastically reducing perceived system latency for the end user.

6.5 Summary

In summary, this chapter has detailed the physical construction and deployment architecture of the FleetShare platform. By successfully integrating a Layered N-Tier Spring Boot backend with a shared-schema, multi-tenant MySQL database and a responsive Thymeleaf frontend, the theoretical system designs have been accurately translated into a robust, secure, and executable web application.

Crucially, this development phase resolved significant technical engineering challenges. It successfully mitigated temporal pricing anomalies through the serialization of immutable data within the `BookingPriceSnapshot` entity, and it enforced rigorous cross-tenant data security through parameterized queries and DTO interception. The implementation of dynamically rendered hierarchical menus en-

sures an optimized user experience. Furthermore, complex external integrations, fortified by asynchronous fault-tolerance mechanisms and cryptographic webhook validations, equip the platform with highly resilient, enterprise-level operational capabilities.

The successful completion of this rigorous development phase has established a stable, highly cohesive software build. This foundation now serves as the authoritative baseline, ready to undergo comprehensive functional evaluation and User Acceptance Testing (UAT) in the subsequent evaluation phases of this project.

CHAPTER 7

CONCLUSION

7.1 Introduction

This chapter concludes the research and development journey of the FleetShare platform by summarizing the core findings and evaluating the achievement of the initial project objectives. It provides a critical reflection on the system's current limitations and development constraints encountered during the project lifecycle. Finally, this chapter proposes actionable recommendations for future enhancements to further optimize and scale the vehicle rental ecosystem.

7.2 Project Contributions

The development of the FleetShare system has successfully addressed the primary research gap identified at the outset of this study: the distinct lack of accessible, unified digital infrastructure for small and medium-sized enterprises (SMEs) and independent vehicle owners within the Malaysian vehicle rental market. By transitioning these operators away from fragmented, semi-digitized methods, such as standalone spreadsheets and instant messaging applications, the project established a cohesive, centralized management ecosystem. Overcoming such technological fragmentation is a well-documented imperative for SMEs seeking to enhance operational resilience and market responsiveness (Matt et al., 2015). This transition is visually represented in Figure 7.1.

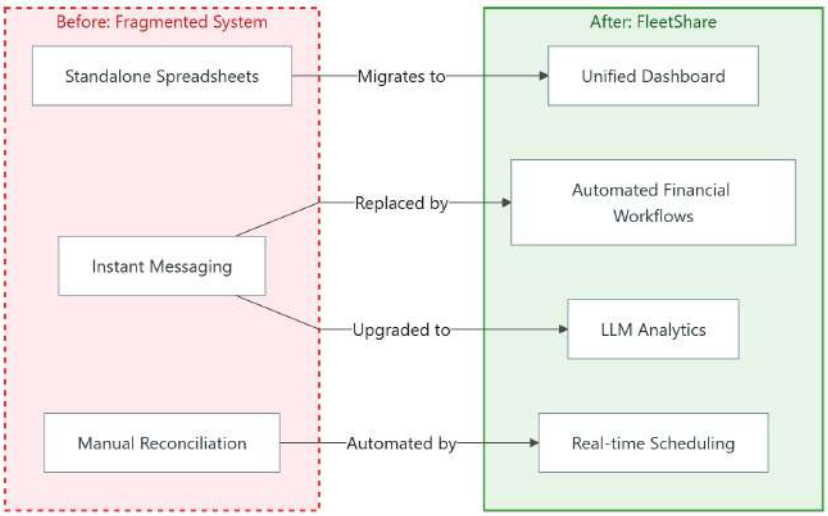


Figure 7.1: Transition from fragmented SME tools to the integrated FleetShare platform.

From a technical perspective, the implementation of a Layered N-Tier architecture utilizing the Model-View-Controller (MVC) design pattern provided a highly maintainable and robust foundation (Fowler, 2002). The engineering of a shared-schema multi-tenant database effectively pooled computational resources to ensure economic scalability while enforcing rigorous application-level data isolation, thereby guaranteeing that proprietary competitor data remains securely partitioned (Krebs et al., 2012). Furthermore, the integration of advanced integrations, specifically the Large Language Model (LLM) analytics engine, democratizes access to enterprise-level data insights for micro-operators who typically lack the capital for complex business intelligence tools. The application of LLMs as natural language interfaces significantly lowers the technical barrier to entry for complex relational database querying (Katsogiannis-Meimarakis and Koutrika, 2023).

Practically, FleetShare delivers substantial operational contributions to its stakeholders. For fleet administrators, the system drastically reduces administrative fatigue by automating financial workflows via the ToyyibPay gateway and centralizing inventory tracking into a single, intuitive dashboard (Few, 2013). The implementation of real-time scheduling mechanisms effectively eliminates opera-

tional bottlenecks, such as double-bookings and temporal scheduling conflicts. Simultaneously, renters are provided with a streamlined, transparent, and frictionless booking experience. By successfully delivering these technical and functional milestones, the project has fully realized its core objectives, empowering independent operators to compete more efficiently in the modern digital economy. This alignment of scalable technical architecture with practical business utility is recognized as a fundamental driver of sustainable competitive advantage in modern platform ecosystems (Tiwana, 2013).

7.3 Project Constraints

While the FleetShare platform successfully fulfills its core operational requirements, a critical evaluation reveals specific constraints and limitations encountered during its development. Architecturally, the system is constrained by its dependency on external third-party Large Language Model (LLM) APIs for analytics, subjecting the system to external rate-limiting parameters and inherent network latency (Ozkaya, 2023). Furthermore, to circumvent the severe financial overhead of deploying isolated databases for every operator (Database-per-Tenant) within the strict Final Year Project (FYP) academic schedule, a Shared-Schema approach was adopted (Krebs et al., 2012). An overview of these architectural and scoping constraints is presented in Figure 7.2.

To ensure the timely delivery of a stable, secure web-based MVP (Minimum Viable Product), several resource-intensive and peripheral modules were explicitly excluded from the initial project scope (Lenarduzzi and Taibi, 2016). The specific functional and technical boundaries of the current iteration include:

- **Advanced Algorithmic Integrations:** Complex mechanisms such as AI-driven predictive vehicle recommendations, blockchain ledgers for immutable transactions, and automated third-party insurance API linkages are not included in this iteration.

- **Hardware and IoT Telematics:** The system acts strictly as a digital management layer. It does not interface with physical Internet of Things (IoT) hardware, meaning it currently lacks live GPS tracking, real-time vehicle telemetry, or remote engine immobilizers for enhanced asset security.

- **Physical and Logistical Operations:** The platform facilitates digital bookings and financial clearing but does not manage or execute physical vehicle inspections, delivery logistics, or on-site mechanical support services.

- **Native Mobile Application Deployment:** The platform was developed strictly as a responsive, browser-based web application. Dedicated, native mobile applications for iOS and Android environments were deferred due to temporal resource constraints.

- **Global Financial Compliance:** The financial, legal, and regulatory scope is strictly localized. International multi-currency conversion mechanisms and global compliance features beyond basic Malaysian regulations (and Ringgit Malaysia processing) are excluded.

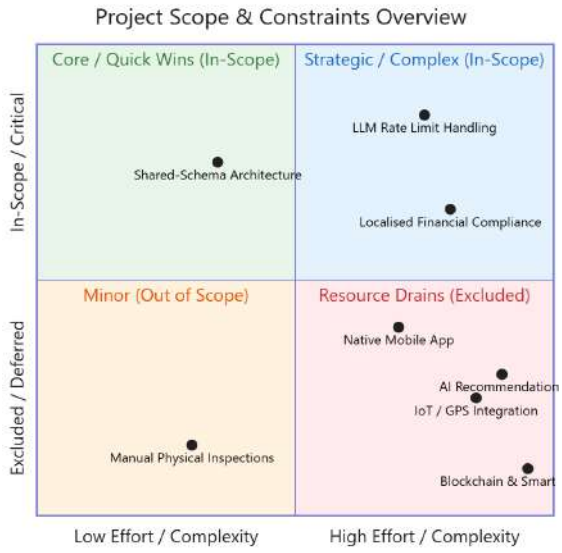


Figure 7.2: Project Constraint Overview

7.4 Future Work

To further elevate the capabilities of the FleetShare ecosystem and directly address the constraints identified during this project, several actionable enhancements are recommended for future iterations. A relative timeline for implementing these enhancements is presented in Figure 7.3.

First, while the current web-based application is fully responsive, the development of a dedicated native Android application would significantly enhance the user experience for both renters and fleet owners. Constructing this client utilizing modern frameworks such as Kotlin and Jetpack Compose, structured upon clean architecture principles, would allow the platform to leverage native device capabilities (Martin, 2017). This includes utilizing hardware-level push notifications for instant booking updates and localized biometric authentication for accelerated, secure system access.

Secondly, the integration of an automated e-KYC (Know Your Customer) module is highly recommended. By interfacing with national identity databases or utilizing AI-driven optical character recognition (OCR), the system could automatically validate renter identities and driving licenses during registration, thereby mitigating fraud risks without adding manual verification burdens to the fleet administrators (Sullivan, 2018).

Thirdly, extending the system architecture to support real-time IoT telematics would provide transformative operational value (Gubbi et al., 2013). Integrating live GPS tracking and onboard diagnostic sensors would allow fleet administrators to monitor vehicle geolocation, detect unauthorized cross-border travel, and automatically synchronize vehicle mileage records directly within their centralized dashboards.

Finally, future iterations should explore advanced algorithmic and financial integrations. Implementing AI-driven dynamic pricing models would allow fleet

owners to automatically optimize their rental rates based on real-time seasonal demand and localized market scarcity (den Boer, 2015). Furthermore, migrating the core booking agreements onto a blockchain ledger utilizing smart contracts could provide immutable, cryptographically secure transaction histories, streamlining automated interactions with third-party insurance APIs and expediting dispute resolutions (Zheng et al., 2017).

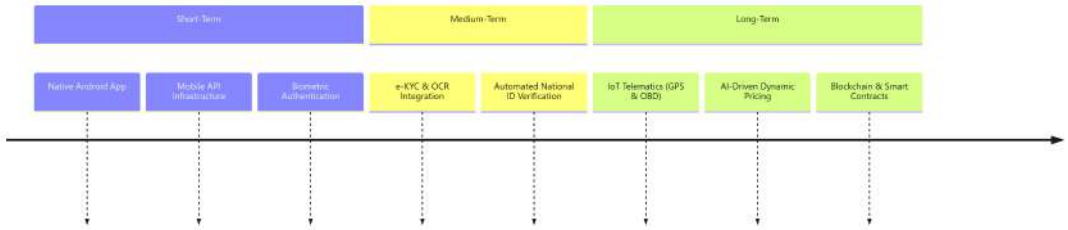


Figure 7.3: Future work roadmap (short-term, medium-term, long-term)

7.5 Summary

The rapid expansion of the sharing economy has fundamentally reshaped mobility demands, necessitating a modernized, digital approach for independent and SME vehicle rental operators in Malaysia (Sundararajan, 2016). By systematically transitioning these businesses away from fragmented, error-prone manual processes, this research has successfully engineered a cohesive digital solution (Matt et al., 2015). The design, implementation, and rigorous testing of the FleetShare centralized management system have provided a robust, scalable, and secure operational foundation. By successfully integrating a strict multi-tenant N-Tier architecture (Krebs et al., 2012) with advanced AI analytics and automated financial routing, the platform transcends basic management utilities. Ultimately, this project achieves its overarching goal of democratizing access to enterprise-grade tools, empowering independent operators to maximize asset utilization, streamline administrative workflows, and thrive within a highly competitive digital transportation landscape (Tiwana, 2013).

REFERENCES

Alam, S. S. and Noor, M. K. M. (2009). Ict adoption in small and medium enterprises: an empirical evidence of service sectors in malaysia. *International Journal of Business and Management*, 4(2):112–125.

Alur, D., Crupi, J., and Malks, D. (2003). *Core J2EE Patterns: Best Practices and Design Strategies*. Prentice Hall, 2nd edition.

Ambler, S. W. (2004). *The Object Primer: Agile Model-Driven Development with UML 2.0*. Cambridge University Press, 3rd edition.

Bardhi, F. and Eckhardt, G. M. (2012). Access-based consumption: The case of car sharing. *Journal of Consumer Research*, 39(4):881–898.

Bargas-Avila, J. A., Oberholzer, G., Schmutz, P., de Vito, M., and Opwis, K. (2010). Web form design: Guidelines and analysis of end-user preferences. *Interacting with Computers*, 22(5):331–339.

Bass, L., Clements, P., and Kazman, R. (2021). *Software Architecture in Practice*. Addison-Wesley Professional, Boston, MA, 4th edition.

Bauer, C., King, G., and Gregory, G. (2015). *Java Persistence with Hibernate*. Manning Publications, 2nd edition.

Belk, R. (2014). You are what you can access: Sharing and collaborative consumption online. *Journal of Business Research*, 67(8):1595–1600.

Bezemer, C.-P. and Zaidman, A. (2010). Multi-tenant saas applications: maintenance dream or nightmare? In *Proceedings of the Joint ERCIM Workshop on Software Evolution (EVOL) and International Workshop on Principles of Software Evolution (IWPSE)*, pages 88–92. ACM.

Boehm, B. W. (1984). Software engineering economics. *IEEE Transactions on Software Engineering*, SE-10(1):4–21.

Chong, F. and Carraro, G. (2006). Architecture strategies for catching the long tail. *Microsoft Developer Network (MSDN) Library*.

Cimperman, R. (2006). *UAT Defined: A Guide to Practical User Acceptance Testing*. Addison-Wesley Professional, Boston, MA.

Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6):377–387.

Cooper, A., Reimann, R., Cronin, D., and Noessel, C. (2014). *About Face: The Essentials of Interaction Design*. John Wiley & Sons, 4th edition.

Courage, C. and Baxter, K. (2005). *Understanding Your Users: A Practical Guide to User Requirements Methods, Tools, and Techniques*. Morgan Kaufmann, San Francisco, CA.

Date, C. J. (2004). *An Introduction to Database Systems*. Pearson/Addison Wesley, 8th edition.

den Boer, A. V. (2015). Dynamic pricing and learning: historical origins, current research, and new directions. *Surveys in Operations Research and Management Science*, 20(1):1–18.

Dennis, A., Wixom, B. H., and Tegarden, D. (2021). *Systems Analysis and Design*. John Wiley & Sons, 7th edition.

Element Fleet Management (2026). Why physical ai is about to reshape fleet technology. Industry insight, Element Fleet Management.

Elmasri, R. and Navathe, S. B. (2016). *Fundamentals of Database Systems*. Pearson, Hoboken, NJ, 7th edition.

Ferraiolo, D. F., Sandhu, R., Gavrila, S., Kuhn, D. R., and Chandramouli, R. (2001). Proposed nist standard for role-based access control. *ACM Transactions on Information and System Security (TISS)*, 4(3):224–274.

Few, S. (2013). *Information Dashboard Design: Displaying Data for At-a-Glance Monitoring*. Analytics Press, 2nd edition.

Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional.

Fowler, M. (2003). *UML Distilled: A Brief Guide to the Standard Object Modeling Language*. Addison-Wesley Professional, 3rd edition.

Garrett, J. J. (2010). *The Elements of User Experience: User-Centered Design for the Web and Beyond*. New Riders, 2nd edition.

Glinz, M. (2007). On non-functional requirements. In *15th IEEE International Requirements Engineering Conference (RE 2007)*, pages 21–26. IEEE.

Goetz, B., Peierls, T., Bloch, J., Bowbeer, J., Holmes, D., and Lea, D. (2006). *Java Concurrency in Practice*. Addison-Wesley Professional.

Gubbi, J., Buyya, R., Marusic, S., and Palaniswami, M. (2013). Internet of things (iot): A vision, architectural elements, and future directions. *Future Generation Computer Systems*, 29(7):1645–1660.

Halfond, W. G. J., Viegas, J., and Orso, A. (2006). A classification of sql-injection attacks and countermeasures. In *Proceedings of the IEEE International Symposium on Secure Software Engineering (ISSSE)*, pages 13–15. IEEE.

Hamari, J., Sjöklint, M., and Ukkonen, A. (2016). The sharing economy: Why people participate in collaborative consumption. *Journal of the Association for Information Science and Technology*, 67(9):2047–2059.

Hearst, M. A. (2009). *Search User Interfaces*. Cambridge University Press.

Hohpe, G. and Woolf, B. (2004). *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley Professional.

Hove, S. E. and Anda, B. (2005). Experiences from conducting semi-structured interviews in empirical software engineering research. In *11th IEEE International Software Metrics Symposium (METRICS'05)*, pages 23–32. IEEE.

IEEE Computer Society (2009). Ieee standard for information technology–systems design–software design descriptions.

Kabbedijk, J., Brinkkemper, S., Jansen, S., and van der Veld, B. (2012). Defining multi-tenancy: A taxonomy of multi-tenant software architectures. In *International Conference on Software Business*, pages 39–52. Springer.

Kappel, G., Pröll, B., Reich, S., and Retschitzegger, W. (2006). *Web Engineering*. John Wiley & Sons.

Katsogiannis-Meimarakis, G. and Koutrika, G. (2023). A survey on natural language interfaces to databases: An llm perspective. *The VLDB Journal*, 32(6):1355–1381.

Ken Research (2023). Malaysia car rental and mobility solutions market. Industry report, Ken Research.

Kerzner, H. (2017). *Project Management: A Systems Approach to Planning, Scheduling, and Controlling*. John Wiley & Sons, Hoboken, NJ, 12th edition.

Kiczales, G., Lamping, J., Mendhekar, A. P., Maeda, C., Lopes, C. V., Loingtier, J.-M., and Irwin, J. (1997). Aspect-oriented programming. In *Proceedings of the 11th European Conference on Object-Oriented Programming (ECOOP)*, pages 220–242. Springer.

Kimball, R. and Ross, M. (2013). *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. John Wiley & Sons, 3rd edition.

Kleppmann, M. (2017). *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media.

Kramer, D. and Schmidt, M. (2024). Architectural patterns for multi-tenant software-as-a-service applications using java spring boot. *IEEE Transactions on Software Engineering*, 50(2):412–428.

Krasner, G. E. and Pope, S. T. (1988). A cookbook for using the model-view-controller user interface paradigm in smalltalk-80. *Journal of Object-Oriented Programming*, 1(3):26–49.

Krebs, R., Momm, C., and Kounev, S. (2012). Architectural concerns for multi-tenant saas applications. In *Proceedings of the 2nd International Conference on Cloud Computing and Services Science (CLOSER 2012)*, pages 426–431. SciTePress.

Krug, S. (2014). *Don't Make Me Think, Revisited: A Common Sense Approach to Web Usability*. New Riders, 3rd edition.

Larman, C. and Basili, V. R. (2003). Iterative and incremental developments: A brief history. *Computer*, 36(6):47–56.

Laudon, K. C. and Laudon, J. P. (2020). *Management Information Systems: Managing the Digital Firm*. Pearson, New York, NY, 16th edition.

Lauret, A. (2019). *The Design of Web APIs*. Manning Publications.

Leff, A. and Rayfield, J. T. (2001). Web-application development using the model/view/controller design pattern. In *Proceedings Fifth IEEE International Enterprise Distributed Object Computing Conference*, pages 118–127. IEEE.

Lenarduzzi, V. and Taibi, D. (2016). Mvp explained: A systematic mapping study on the definitions of minimum viable product. In *42nd Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*, pages 112–119. IEEE.

Marcotte, E. (2011). *Responsive Web Design*. A Book Apart, New York, NY.

Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.

Matt, C., Hess, T., and Benlian, A. (2015). Digital transformation strategies. *Business & Information Systems Engineering*, 57(5):339–343.

Mourad, A., Puchinger, J., and Chu, C. (2019). A survey of models and algorithms for optimizing shared mobility. *Transportation Research Part B: Methodological*, 123:323–346.

Myers, G. J., Sandler, C., and Badgett, T. (2011). *The Art of Software Testing*. John Wiley & Sons, Hoboken, NJ, 3rd edition.

Newman, S. (2021). *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media, 2nd edition.

Nielsen, J. (1993). *Usability Engineering*. Morgan Kaufmann.

Norman, D. (2013). *The Design of Everyday Things*. Basic Books, revised and expanded edition.

OECD (2021). *The Digital Transformation of SMEs*. OECD Publishing, Paris.

Ozkaya, I. (2023). Application of large language models to software engineering tasks: Opportunities, risks, and implications. *IEEE Software*, 40(3):4–8.

Pohl, K. (2010). *Requirements Engineering: Fundamentals, Principles, and Techniques*. Springer, Berlin, Heidelberg.

Pressman, R. S. and Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach*. McGraw-Hill Education, New York, NY, 9th edition.

Provos, N. and Mazières, D. (1999). A future-adaptable password scheme. In *Proceedings of the 1999 USENIX Annual Technical Conference*, pages 81–92. USENIX Association.

Richards, M. (2015). *Software Architecture Patterns*. O'Reilly Media.

Romney, M. B. and Steinbart, P. J. (2017). *Accounting Information Systems*. Pearson, 14th edition.

Rosenfeld, L., Morville, P., and Arango, J. (2015). *Information Architecture: For the Web and Beyond*. O'Reilly Media, 4th edition.

Rumbaugh, J., Jacobson, I., and Booch, G. (2004). *The Unified Modeling Language Reference Manual*. Pearson Higher Education, Boston, MA, 2nd edition.

Rüping, A. (2003). *Agile Documentation: A Practical Guide to Advanced Methods in Software Documentation*. John Wiley & Sons.

Sandhu, R. S., Coyne, E. J., Feinstein, H. L., and Youman, C. E. (1996). Role-based access control models. *IEEE Computer*, 29(2):38–47.

Satzinger, J. W., Jackson, R. B., and Burd, S. D. (2015). *Systems Analysis and Design in a Changing World*. Cengage Learning, Boston, MA, 7th edition.

Seethamraju, R. (2015). Adoption of software as a service (saas) enterprise resource planning (erp) systems in small and medium sized enterprises (smes). *Information Systems Frontiers*, 17(3):475–492.

Shneiderman, B., Plaisant, C., Cohen, M., Jacobs, S., Elmqvist, N., and Diakopoulos, N. (2016). *Designing the User Interface: Strategies for Effective Human-Computer Interaction*. Pearson, Boston, MA, 6th edition.

Silberschatz, A., Korth, H. F., and Sudarshan, S. (2019). *Database System Concepts*. McGraw-Hill, 7th edition.

Snodgrass, R. T. (1999). *Developing Time-Oriented Database Applications in SQL*. Morgan Kaufmann.

Sommerville, I. (2015). *Software Engineering*. Pearson, 10th edition.

Sommerville, I. (2020). *Software Engineering*. Pearson, 11th edition.

Stuttard, D. and Pinto, M. (2011). *The Web Application Hacker's Handbook: Finding and Exploiting Security Flaws*. John Wiley & Sons, 2nd edition.

Sullivan, C. (2018). Digital identity - from emergent legal concept to new reality. *Computer Law & Security Review*, 34(4):723–731.

Sundararajan, A. (2016). *The Sharing Economy: The End of Employment and the Rise of Crowd-Based Capitalism*. MIT Press.

Taylor, R. N., Medvidovic, N., and Dashofy, E. M. (2009). *Software Architecture: Foundations, Theory, and Practice*. John Wiley & Sons.

Tiwana, A. (2013). *Platform Ecosystems: Aligning Architecture, Governance, and Strategy*. Morgan Kaufmann.

Walls, C. (2022). *Spring in Action*. Manning Publications, Shelter Island, NY, 6th edition.

Wiegers, K. and Beatty, J. (2013). *Software Requirements*. Microsoft Press, Redmond, WA, 3rd edition.

Wilson, J. M. (2003). Gantt charts: A centenary appreciation. *European Journal of Operational Research*, 149(2):430–437.

Yigitbasioglu, O. M. and Velcu, O. (2012). A review of dashboards in performance management: Implications for design and research. *International Journal of Accounting Information Systems*, 13(1):41–59.

Zheng, Z., Xie, S., Dai, H., Chen, X., and Wang, H. (2017). An overview of blockchain technology: Architecture, consensus, and future trends. In *Proceedings of the 2017 IEEE International Congress on Big Data (BigData Congress)*, pages 557–564. IEEE.